

Agent Harness: A Practitioner's Textbook

A fact-based synthesis of contemporary writing on harness engineering, drawn from the [Awesome Harness Engineering](#) reading list and the [OpenReview](#) survey [Agent Harness Engineering: A Survey](#). Every substantive claim is referenced inline.

22 sections · generated from Markdown sources

Preface

This book examines the system that surrounds a language model when the model is asked to do real work. That system now has a name — *the harness* — and a small but rapidly maturing body of literature on how to build it. The chapters that follow bring that literature together into a single narrative. Each claim cites the relevant original article so that readers can trace any thread back to its source.

The field starts from a simple premise. As Vivek Trivedy of LangChain puts it: "Agent = Model + Harness. **If you're not the model, you're the harness.**" ([LangChain — The Anatomy of an Agent Harness](#)). In this framing, everything around the model — system prompts, tools, sandboxes, memory, sub-agents, control flow, and evaluation infrastructure — belongs to the harness. This book studies how to design that surrounding system well.

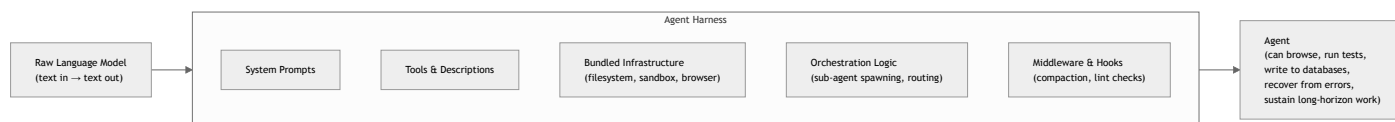
Throughout the book, think of the model as a component that consumes tokens and emits tokens. Its output may be text for a user or structured text that requests an action, but surrounding software always carries out the action itself. This distinction between model output and real-world effect explains why later chapters devote so much attention to context, tools, state, tests, sandboxes, and evaluation.

The companion stage-1 volume covers the model internals this book relies on: tokens, attention and the KV-cache, the context window, sampling, retrieval, and the tool-call protocol. Here, we briefly name those concepts and build the harness-engineering layer on top of them. Most of the new material is software engineering: context, tools, state, sandboxes, and evaluation.

Before You Start

This is the second book in a two-part sequence. It assumes you have read *LLM Foundations for Harness Engineering* and are comfortable with tokens, attention and the KV-cache, the context window, sampling, post-training, retrieval, the agent loop, the tool-call protocol, prompt injection, and pass@k/pass^k. When this book uses a Foundations concept, it points back to the relevant chapter (for example, "Foundations ch 9") instead of deriving the concept again. Readers who have not completed the stage-1 material can still follow the narrative, but should treat the one-line recaps as pointers rather than complete explanations.

The Core Equation



Key Takeaways

- A language model alone cannot maintain state, execute code, or access real-time knowledge — these are all harness-level features.
- Harness engineering is distinct from prompt engineering: it improves the entire system, not just individual prompts.
- The field is young — most canonical articles were published in 2025 and 2026 — but is maturing rapidly.
- Every claim in this textbook is cited so readers can follow threads back to primary sources.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>

Chapter 1: What Is an Agent Harness?

1.1 The Model + Harness Equation

A raw language model takes text in and produces text out. The companion volume covers its native capabilities and limitations (see Foundations ch 1). Turning that model into an agent requires building every additional capability around it: browsing a codebase, running tests, writing to a database, talking to a user, recovering from errors, and sustaining progress across hours of work. LangChain describes the resulting harness as a combination of system prompts; tools and their descriptions; bundled infrastructure such as a filesystem, sandbox, and browser; orchestration logic such as sub-agent spawning and model routing; and hooks or middleware for deterministic operations such as compaction, continuation, and lint checks ([LangChain — The Anatomy of an Agent Harness](#)).

This framing makes the design problem explicit. On its own, a model cannot maintain durable state across interactions, execute code, access real-time knowledge, or set up environments and install packages. All of these are harness-level features ([LangChain — The Anatomy of an Agent Harness](#)). Even basic chat relies on a harness pattern: a while-loop tracks previous messages and adds new ones to the context, creating the appearance that the model "remembers" what was just said.

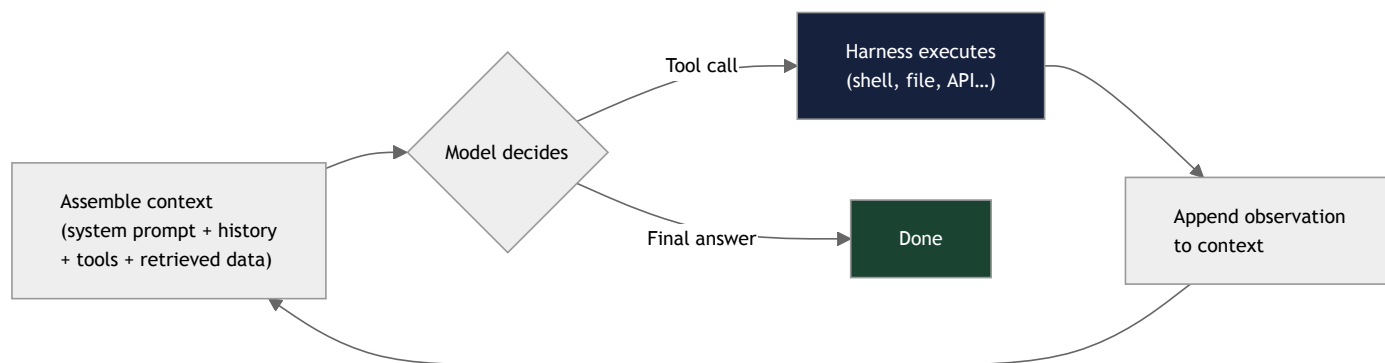
HumanLayer offers essentially the same equation from the perspective of someone configuring a coding agent: "coding agent = AI model(s) + harness." It describes the harness as the agent's runtime or peripherals — the components through which the model interacts with its environment ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

1.2 The Agent Loop

The companion volume introduces the agent loop and a key principle behind it: a tool call is structured output for the harness to execute, not a direct action taken by the model (see Foundations ch 1 and ch 12). Briefly, a raw model call is one-shot: text goes in and text comes out. An *agent* wraps that call in a loop. The harness assembles the context, and the model emits either a final answer or a *tool call* — structured output, typically JSON, that names a tool and supplies its arguments. Deterministic harness code then executes the call, appends the observation to the context, and starts the next iteration with that expanded context.

HumanLayer captures the same idea in the phrase "tools are just structured outputs" ([HumanLayer — 12-Factor Agents](#)). File edits, shell commands, browser clicks, and database writes become real only when the harness accepts the model's request and executes it. The model at the centre of this loop is what Anthropic calls the *augmented LLM* and what the research literature describes as *ReAct* (reason + act) (mechanics in Foundations ch 12).

Two consequences of this loop shape the rest of the book. First, **context grows monotonically**: each turn appends a tool call and its observation, so an N-step task accumulates N rounds of history. Chapter 2 therefore treats context as a finite resource; compaction, sub-agents, and memory all help manage it. Second, **the model never executes anything itself**. It emits a request, and deterministic harness code decides whether and how to honour it. The gap between request and execution is where the later chapters place guardrails, sandboxes, hooks, and approval gates. The harness occupies that gap, sitting between what the model asks for and what actually happens.



1.3 Bounded Contexts: Inner and Outer Harness

The word "harness" is used loosely, and Thoughtworks writers have noted that its boundaries vary depending on whom you ask. Birgitta Böckeler proposes three concentric rings: the model at the core; the coding agent's *builder harness* in the middle, including the system prompt and tools shipped by Anthropic, OpenAI, and others; and the *user harness* on the outside, including the AGENTS.md files, hooks, skills, and review agents that a team adds for its

codebase ([Thoughtworks / Martin Fowler — Harness Engineering](#)). Most working engineers operate primarily in this outer ring.

The OpenReview paper *Agent Harness Engineering: A Survey* defines this boundary more formally. It describes a harness as the software and interface substrate that governs how foundation models perceive context, call tools, act over time, and remain auditable in a deployment environment ([OpenReview — Agent Harness Engineering: A Survey](#)). The paper treats the harness as a potential *binding constraint*: for long-horizon agents, reliability often depends as much on the surrounding substrate as on raw model quality.

1.4 ETCLOVG: A Seven-Layer Map

The survey organizes the harness design space using the acronym **ETCLOVG**: Execution environment, Tool interface, Context, Lifecycle, Observability, Verification, and Governance ([OpenReview — Agent Harness Engineering: A Survey](#)). This map is useful because it prevents the concept of a harness from being reduced to prompts or tools alone.

- **Execution environment**: the sandbox, browser, operating system, code runner, or managed cloud where actions become real effects.
- **Tool interface**: protocols, schemas, registries, function calling, MCP/A2A-style boundaries, and tool-selection policies.
- **Context**: prompt assembly, retrieval, memory, compaction, state compression, and what the model is allowed to see.
- **Lifecycle**: task startup, planning, checkpoint/resume, failure recovery, handoff, session termination, and long-running state.
- **Observability**: traces, telemetry, cost attribution, latency, token accounting, and failure forensics.
- **Verification**: eval harnesses, graders, task suites, outcome checks, and readiness gates.
- **Governance**: permissions, policy languages, audit trails, human approval, constitutional or rule-based controls, and cross-layer security.

The chapters that follow can largely be read as a tour through these seven layers. Context and memory dominate Chapters 2-3; tools and execution appear in Chapters 4-5; lifecycle appears in Chapters 7-9; and verification, observability, and governance are developed in Chapters 10-12 and revisited in the outlook.

1.5 Why Harnesses Exist: Working Backwards from Model Deficits

LangChain offers a useful way to derive the components of a harness: list the agent behaviors you want, then identify what models cannot natively do. The necessary harness components follow from those gaps ([LangChain — The Anatomy of an Agent Harness](#)).

For example, models can operate only on information available in the context window. A filesystem therefore provides durable storage, a place to offload information, and a shared workspace for agents and humans. Bash and code execution address a different limitation: it is impractical to define in advance every tool an agent might need, so a general-purpose execution channel allows the model to create tools as needed. Sandboxes give that execution a safe environment. Memory and search inject information that is absent from the model's weights and current context. Compaction, tool-result offloading, and skills help manage a finite context window whose performance degrades as it fills.

Each component addresses a specific limitation. The harness is the system formed by those responses.

1.6 The Historical Arc: From Prompt Engineering to Harness Engineering

Anthropic presents the recent shift as a natural progression. In the early days of LLM applications, the dominant practice was *prompt engineering*: writing and organizing instructions for one-shot tasks. As applications developed into multi-turn agents operating over longer time horizons, the focus expanded to *context engineering*. This means curating and maintaining the most useful set of tokens during inference, including information that enters the context outside the prompts themselves ([Anthropic — Effective Context Engineering for AI Agents](#)).

Harness engineering sits one level above context engineering. As Mitchell Hashimoto has put it, the practice means taking the time to engineer a solution whenever the agent makes a mistake so that it does not make the same mistake again ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#) quoting Hashimoto). Prompt engineering tunes an individual prompt; harness engineering improves the entire system in which prompts run.

A further reframing arrived in 2026 under the name *loop engineering*. As agents began to run unattended over longer horizons, the main unit of work shifted again: from the prompt, to the context, and then to the *loop* that decides what to prompt, when to prompt it, and whether the result is good enough ([Addy Osmani — Loop Engineering](#)). Loop engineering is not so much a rival to harness engineering as an operator-facing view of its outer control loop: the triggers, verifiers, and stop rules that surround the agent. Chapter 8 develops this idea in full.

1.7 Frameworks, Runtimes, and Harnesses

These three terms are sometimes used interchangeably. LangChain's Harrison Chase distinguishes them as follows ([LangChain — Agent Frameworks, Runtimes, and Harnesses, Oh My!](#)):

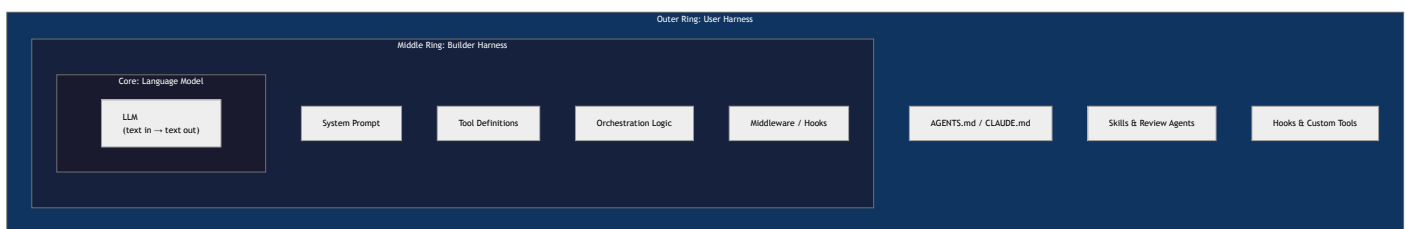
A *framework* — such as LangChain, Vercel's AI SDK, CrewAI, the OpenAI Agents SDK, or Google ADK — provides abstractions that help developers get started and standardize how applications are built. A *runtime* — such as LangGraph, Temporal, or Inngest — handles infrastructure concerns including durable execution, streaming, human-in-the-loop support, and thread-level and cross-thread persistence. A *harness* — such as LangChain's DeepAgents or Anthropic's Claude Agent SDK — sits one level higher. It includes default prompts, opinionated tool handling, planning tools, filesystem access, and other "batteries included" features. The boundaries are not absolute; for example, LangGraph can reasonably be described as both a runtime and a framework. Even so, the distinction is useful when deciding what to adopt.

1.8 Will Better Models Make Harnesses Obsolete?

One question shadows the harness-engineering frame: will better models make the surrounding system less important? HumanLayer argues in "Skill Issue" that teams often blame the model — "GPT-6 will fix it," or "we just need better instruction-following" — when the real problem is harness configuration ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)). Better models will eliminate some current failure modes, but they will also be assigned harder problems and will continue to fail in unexpected ways. Such failures are a fundamental property of non-deterministic systems. The implication is that harness engineering is ongoing work, not temporary scaffolding to discard once models become capable enough.

LangChain reaches a similar conclusion. As models become more capable, some features that live in the harness today will be absorbed into the model. Harness engineering will nevertheless remain useful, both for addressing model deficiencies and for building systems that make model intelligence more effective ([LangChain — The Anatomy of an Agent Harness](#)).

Diagram: Model → Harness Layers



Key Takeaways

- **Agent = Model + Harness:** every capability beyond raw text I/O must be engineered into the surrounding system.
- **The agent loop is the foundation:** assemble context, the model emits a tool call, the harness executes it, the result is appended — and the cycle repeats, growing context every turn.
- **Tool calls are structured requests:** the model proposes actions in text; the harness decides which requests become real effects.
- **Three concentric layers:** the LLM core, the builder harness (shipped by the AI lab), and the user harness (built by the team).
- **ETCLOVG gives a systems map:** execution, tools, context, lifecycle, observability, verification, and governance are all harness layers.
- **Harness components derive from model deficits:** filesystem, sandbox, memory, compaction each address a specific limitation.

- **Harness engineering is ongoing work:** as models improve, harder problems are tackled and new failure modes emerge.
- **Framework ≠ Runtime ≠ Harness:** understanding the distinction helps teams make adoption decisions.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, *Oh My!*, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 2: Context as a Finite Resource

2.1 Context Rot and the Attention Budget

The most important operating constraint for an agent harness is that useful context is not the same as the largest context a model can accept. The underlying failure mode is *context rot*: model output can degrade as the input grows. The companion volume introduces a two-layer model of this effect—degradation from length itself, compounded by the accumulation of irrelevant material (see Foundations ch 9). Here, context rot serves as the chapter's governing design constraint. Anthropic connects the same failure mode to needle-in-a-haystack benchmarks: as a context contains more tokens, the model may become less accurate at recalling information from it ([Anthropic — Effective Context Engineering for AI Agents](#)). The effect varies by model, task, and the position of the relevant information, so treat it as a probabilistic tendency rather than a hard law for every prompt.

Anthropic's mechanistic explanation is not that transformers literally "run out" of attention. Rather, long contexts create many more pairwise token relationships for the model to represent, while training data and positional mechanisms are typically stronger at shorter, more local dependencies. Position-encoding interpolation and other long-context techniques let models process sequences longer than those seen during training, but they can still reduce positional resolution or retrieval reliability ([Anthropic — Effective Context Engineering for AI Agents](#)).

The practical conclusion is that context is a finite resource with diminishing marginal returns. Anthropic describes it as an "attention budget" that every additional token consumes. HumanLayer puts the point more bluntly: even as models support longer context windows, production systems should usually favor small, focused prompts and contexts. In its experience, open-ended "tool-calling loops" often become difficult to recover after roughly 10–20 turns ([HumanLayer — 12-Factor Agents](#)).

A larger context window helps when the model genuinely needs access to more information, but it does not automatically improve attention or instruction-following. As HumanLayer notes, extended-context releases often use techniques such as YaRN to increase sequence length rather than the model's effective instruction budget. For needle-in-a-haystack tasks, a larger window can simply create a larger haystack ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

2.2 The Anatomy of Effective Context

Given this budget constraint, the goal, in Anthropic's words, is "the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome" ([Anthropic — Effective Context Engineering for AI Agents](#)). In practice, that goal applies to several parts of the harness:

System prompts should operate at the right *altitude*. They should avoid both hardcoded if-else logic for every edge case and vague, high-level advice. Instead, they should be specific enough to guide behavior while leaving the model room to apply strong heuristics. Anthropic recommends organizing prompts into sections (`<background_information>`, `<instructions>`, `## Tool guidance`, etc.) with XML or Markdown delimiters, although formatting becomes less important as models improve.

Tools define the contract between the agent and its environment. Each tool should be self-contained and clearly described, with as little overlap as possible. The most common failure Anthropic sees is a bloated tool set that covers too much functionality and creates ambiguous decision points: "if a human engineer can't definitively say which tool should be used in a given situation, an AI agent can't be expected to do better."

Few-shot examples should be diverse and canonical, not a catalog of every edge case. Anthropic's analogy is that, for an LLM, examples are the "pictures" worth a thousand words.

2.3 The KV-Cache: Why Stable Prefixes Pay Off

The companion volume explains the KV-cache and the prefix-stability rule: change an early token, such as a timestamp in the system prompt, and every token after that point must be recomputed (see Foundations ch 9). Here, the focus is on the cache as a production cost lever. Although it receives little attention in the academic literature on context engineering, the KV-cache is central to production agent design. Manus argues that its hit rate is "the single most important metric for a production-stage AI agent" ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

The cache dominates the cost model because of how a typical agent loop works. The agent receives input, selects and executes an action from its tool space, then appends the action and observation to the context for the next turn. The context grows at every step while the output remains short, producing a highly skewed prefill-to-decoding ratio;

Manus reports an average input-to-output token ratio of 100:1. When the prefix is unchanged, the KV-cache can serve it again, reducing both time-to-first-token and inference cost. Manus cites Claude Sonnet pricing in which cached input tokens cost \$0.30 per million, compared with \$3 per million for uncached input—a 10x difference.

Manus offers three rules for keeping the cache hot. First, keep the prompt prefix stable: a one-token difference invalidates the cache from that point onward, so seemingly minor changes such as adding a timestamp with second-level precision to the system prompt are costly. Second, make the context append-only and use deterministic JSON serialization; some libraries do not guarantee key ordering, which can silently break the cache. Third, mark cache breakpoints explicitly when the inference framework requires them.

2.4 Mask, Don't Remove

Manus's second principle concerns the action space. As the number of tools grows—and MCP (the Model Context Protocol; see Chapter 4) makes it easy to connect hundreds of them—it is tempting to load and unload tools dynamically during an agent loop. Manus's experiments suggest a clear rule: avoid doing so. Tool definitions appear near the beginning of the context, so changing them invalidates the cache for everything that follows. Earlier turns may also refer to tools that are no longer available, causing schema violations or hallucinated calls ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

The alternative is action masking: keep a stable tool surface in the context, but constrain which actions the agent may select in a given state. Depending on the provider and harness, the implementation may use logit constraints, tool-choice controls, response prefill, or a runtime validator that rejects disallowed actions ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). Manus gives related actions consistent name prefixes— `browser_*` for browser tools and `shell_*` for shell tools—so a simple constraint can allow or exclude an entire group.

2.5 The File System as the Ultimate Context

Even with a 128K-token window (Manus's figure at the time; current windows are larger, but the underlying dynamic is unchanged), real agentic work regularly exceeds the available context. Web pages and PDFs can produce enormous observations, performance may degrade well before the technical limit, and long inputs remain expensive even when cached ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

The solution proposed by Manus, and echoed by Anthropic and LangChain, is to use the filesystem as the agent's working memory. It is far larger than the context window, persists across turns, can be manipulated directly by the agent, and gives stored artifacts paths that the agent can reference. However, it is not "memory" in the human sense. Without clear filenames, summaries, indexes, and sound retrieval habits, an agent can still fail to find what it wrote. Manus therefore favors compression strategies that are deliberately *restorable*: a web page can leave the context if its URL remains, and a document's contents can be omitted if the agent retains its path.

LangChain calls the filesystem "arguably the most foundational harness primitive." It provides a workspace for data, code, and documentation; lets agents offload work incrementally instead of keeping everything in context; and creates a natural collaboration surface for coordination among agents and humans ([LangChain — The Anatomy of an Agent Harness](#)). Adding git provides versioning, rollback, and branching.

For coding agents, this principle becomes stronger: the repository should be the system of record. OpenAI's Codex harness guidance argues that successful agent-first repositories keep the context an agent needs close to the code. An `AGENTS.md` file directs the agent to the right places, short design documents explain the local architecture, and structural checks encode the rules that matter ([OpenAI — Harness Engineering](#)). A decision that exists only in Slack, a ticket, or a senior engineer's memory is not reliable context for a fresh agent session.

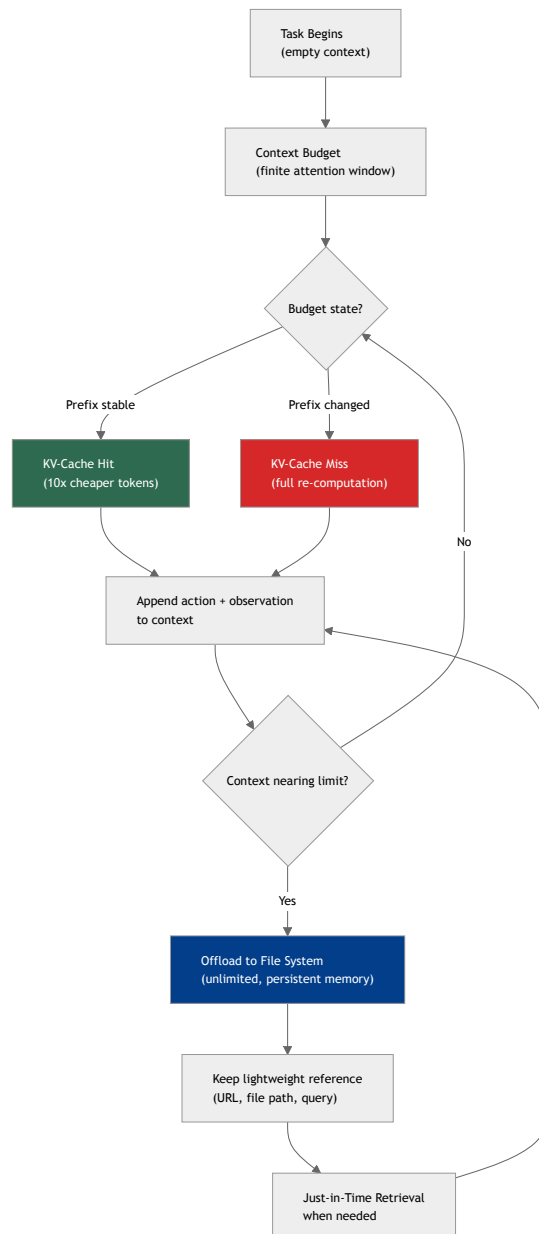
This does not mean putting more text into the prompt. It means making repository facts discoverable. One useful cold-start test is to open a new agent session without an oral briefing and ask five questions: What is this system? How is it organized? How do I run it? How do I verify it? What is its current state? If repository files and standard commands cannot supply the answers, the harness has a knowledge-visibility gap. Closing that gap usually requires small documents near the code, stable entry points, progress files, and verification commands—not one enormous root instruction file.

2.6 Just-in-Time Retrieval

The traditional pattern—embed everything, retrieve the top-k chunks, and prepend them to the context (the RAG pipeline; see Foundations ch 11)—is increasingly supplemented by a *just-in-time* approach. Instead of pre-processing everything in advance, agents retain lightweight identifiers such as file paths, queries, and links, then load the underlying data into context when needed ([Anthropic — Effective Context Engineering for AI Agents](#)).

Anthropic's Claude Code uses this pattern when working with large codebases. The model writes targeted queries, stores their results, and uses tools such as `head` and `tail` to inspect large datasets without loading them in full. File-path metadata is informative in its own right: `test_utils.py` in a `tests/` folder suggests a different role from a file with the same name in `src/core_logic/`. Folder hierarchies, names, and timestamps all become navigation signals. The trade-off is that runtime exploration is slower than retrieving precomputed data, and an agent without clear tool guidance can waste context on dead ends. A hybrid pattern is therefore common: provide a small amount of high-value context up front, such as project instructions in `CLAUDE.md` or `AGENTS.md`, and let the agent explore everything else on demand.

Diagram: Context Budget → KV-Cache → File System as Memory



Key Takeaways

- **Context rot is real enough to design around:** larger contexts can help, but they do not remove the need for curation.
- **The KV-cache is a major production metric:** stable prefixes can matter as much as model quality for latency and cost.
- **Mask tools when possible, don't churn them casually:** dynamically adding/removing tools mid-run can break cache locality and cause schema violations.

- **The filesystem is working memory, not magic memory:** it is larger and persistent, but it still needs paths, summaries, and retrieval discipline.
- **The repository is the agent's system of record:** critical project facts need to be discoverable from repo files and commands during a cold start.
- **Just-in-time retrieval often beats pre-loading:** maintain lightweight identifiers and load data only when needed.

Further Reading

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>

Chapter 3: Compaction, Memory, and the Sub-Agent Pattern

Even with disciplined context engineering, a long-horizon task can exceed any single context window. The literature converges on three techniques for continuing beyond that limit.

3.1 Compaction

Compaction summarizes a conversation as it approaches the context-window limit, then starts a new context window from that summary ([Anthropic — Effective Context Engineering for AI Agents](#)). In Claude Code, the harness passes the message history to the model with instructions to preserve architectural decisions, unresolved bugs, and implementation details while discarding redundant tool output. The agent then continues from the compressed context and the files it accessed most recently. The compaction trigger—usually a token threshold or a percentage of context usage—must itself be tuned. Trigger too late, and the context may overflow in the middle of a turn; trigger too early, and the summary may discard details that are still useful.

Anthropic recommends tuning compaction prompts on real, complex traces. Start by maximizing recall so that the summary captures all relevant information; then improve precision by removing material that does not help the agent continue. The lightest form of compaction is tool-result clearing: after the agent has acted on a tool result, the raw output can usually be discarded.

The key caveat is that compaction is lossy. It is well suited to preserving decisions, goals, constraints, and pointers to artifacts, but not every detail of a long debugging trace. A good harness therefore pairs compaction with restorable references—commit hashes, file paths, issue IDs, URLs, and short notes—so that a later agent can reload the primary evidence when the summary is too thin.

3.2 Structured Note-Taking

The complementary pattern is *agentic memory*: the agent regularly writes structured notes to disk and reloads them later. Anthropic illustrates the pattern with Claude playing Pokémon. Across thousands of game steps, the agent maintains progress tallies ("for the last 1,234 steps I've been training my Pokémon in Route 1, Pikachu has gained 8 levels toward the target of 10"), develops maps of regions, and records combat strategies. Those notes let it resume multi-hour training sequences after a context reset ([Anthropic — Effective Context Engineering for AI Agents](#)).

Manus's `todo.md` technique is a specialized form of structured note-taking, with an additional purpose explored in the next section.

3.3 Recitation: Manipulating Attention Through the End of Context

Manus reports that its agent creates a `todo.md` file for complex tasks, then rewrites the file step by step as work progresses and checks off completed items. The purpose is not merely organizational. A typical Manus task averages about 50 tool calls, and over a long context the model can drift off task or lose sight of earlier goals. Rewriting the todo list repeatedly recites those goals at the *end* of the context. This keeps the global plan in the model's recent attention span and helps counter the well-known "lost-in-the-middle" problem (the companion volume, *LLM Foundations* ch9, covers the underlying effect) ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

3.4 Sub-Agents and the Context Firewall

The third pattern—and the one with the greatest architectural consequences—is sub-agent decomposition. A specialized sub-agent receives a focused task and its own context window. It may consume tens of thousands of tokens while working, but returns only a condensed summary of 1,000–2,000 tokens to the parent ([Anthropic — Effective Context Engineering for AI Agents](#)). HumanLayer calls this boundary the *context firewall*. Because the parent thread handles orchestration without seeing the sub-agent's intermediate noise, its context remains useful for much longer and is less likely to enter the "dumb zone" ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

HumanLayer distinguishes sharply between role-playing and context control. Assigning generic "frontend engineer" and "backend engineer" personas to sub-agents is not the useful pattern; delegating bounded work to protect the parent's context is. The best sub-agent tasks have a compact final answer but require many intermediate tool calls—for example, locating a definition in a codebase, tracing information flow across services, or conducting broad research.

Sub-agents can also control cost. HumanLayer uses an expensive model (Opus) for orchestration and cheaper models (Sonnet or Haiku) for delegated work. A simple operation such as `grep` does not require the orchestrator's most capable model.

Anthropic's multi-agent research system is the canonical example of this pattern at scale. A lead agent analyzes a query and asks specialized sub-agents to investigate different aspects in parallel. Each sub-agent works in its own context window and returns a compressed result; the lead then synthesizes those results into a final report. On Anthropic's internal research evaluation, a configuration with Opus as the lead and Sonnet as the sub-agents outperformed single-agent Opus by 90.2%—a relative improvement ([Anthropic — How We Built Our Multi-Agent Research System](#)). The mechanism is largely one of token economics. In Anthropic's analysis, three factors explained 95% of the performance variance on the BrowseComp benchmark, and token usage alone explained 80%.

The catch is cost. In Anthropic's data, single-agent runs use roughly 4× as many tokens as a chat, while multi-agent systems use roughly 15×. A multi-agent system therefore costs on the order of 4× as much as a single agent—a derived ratio, not a figure stated directly by the source. This architecture makes economic sense only for high-value work that benefits from parallelism. It is a poor fit for tightly coupled subtasks that share mutable state, a category that includes many implementation-heavy coding tasks. It can still help coding workflows when delegated tasks are read-only, investigative, or separated by clear ownership boundaries. Because current models are also weak at real-time coordination across agents, the coordinator must define those boundaries explicitly.

3.5 Don't Few-Shot Yourself Into a Rut

Manus offers a counterintuitive principle: too much consistency in the context can be harmful. Models are strong mimics of the patterns they see. If a trace contains many similar action-observation pairs, the model may continue the pattern after it stops being useful, leading to drift, overgeneralization, or hallucination. In Manus's example, an agent reviewing a batch of 20 résumés falls into a rhythm and begins repeating actions for their own sake ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

The proposed fix is to introduce small, structured variations: different serialization templates, alternate phrasing, minor reordering, or controlled noise. Diversity in the trace prevents one pattern from dominating the model's attention.

3.6 Keep the Wrong Stuff In

The complementary principle is to retain useful errors. The natural impulse is to retry a failed action and hide its trace, but Manus argues that doing so removes the evidence the model needs to avoid the same mistake ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). Keeping the most recent failed action and its relevant stack trace in context allows the model to adjust its next attempt. This does not mean preserving unlimited logs indefinitely. Repeated, identical failures should be compacted into a short diagnosis and a retry counter. Manus calls error recovery "one of the clearest indicators of true agentic behavior" and notes that academic benchmarks underrepresent it because they tend to measure success under ideal conditions.

HumanLayer formalizes this idea as Factor 9: compact errors into context. An agent's ability to read an error and adjust its next call—its *self-healing* property—is one of the genuine benefits of LLM agents, and it works only when the error remains visible ([HumanLayer — 12-Factor Agents](#)). Combined with a counter that limits consecutive identical errors, the pattern is robust.

3.7 Cost, Latency, and Model Routing

The sub-agent pattern already treats cost as a design variable: an expensive model orchestrates while cheaper models perform the supporting work. The broader principle is worth stating explicitly. Cost and latency are first-class harness concerns, not afterthoughts.

Three levers recur across the literature:

- **Model routing.** Not every step requires the strongest model. A harness can send inexpensive, high-volume work—a `grep`, a classification, or a short summary—to a small, fast model and reserve the frontier model for reasoning-intensive steps. HumanLayer uses Opus for the orchestrator and Sonnet or Haiku for sub-agents ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)); Anthropic's research system uses the same lead-agent / sub-agent split (chapter 7).
- **The KV-cache.** Stable context prefixes can be served from the cache at roughly one-tenth the price and a fraction of the latency of uncached tokens (the companion volume, *LLM Foundations* ch9, explains the mechanics; see also chapter 2). In production agents, cache discipline is often the largest single cost lever.

- **Token accounting.** Agentic workloads are prefill-heavy—Manus reports an input-to-output ratio of about 100:1—and cost grows with context length. Multi-agent systems can consume roughly 15× the tokens of a single chat, so they pay off only on high-value tasks.

Latency has a different structure. Time-to-first-token is dominated by prefill and therefore by cache hits; end-to-end latency is dominated by the number of *sequential* model round-trips. Parallel tool calls and sub-agents can reduce wall-clock time substantially—by up to 90% on Anthropic's research workloads (chapter 7)—without reducing total token cost. The general rule is to treat tokens, dollars, and seconds as separate, explicit budgets and understand which design lever affects each one.

3.8 Named Memory Architectures

Sections 3.1–3.3 presented compaction, note-taking, and recitation as harness *techniques*. The research literature also combines these ideas into named memory *systems* that are useful to recognize.

- **MemGPT** uses an explicit operating-system analogy. It treats the context window as fast "main memory" and external stores as "disk," then lets the model issue function calls that page information into and out of a fixed-size context. This *virtual context management* makes a bounded window appear much larger ([Packer et al. — MemGPT: Towards LLMs as Operating Systems](#)). The system is now productized as Letta.
- **Mem0** is a memory layer that dynamically *extracts* salient facts from a conversation, *consolidates* them with existing stored information, and *retrieves* them on later turns. A graph variant also captures relationships between entities. Its reported advantage is operational: much lower token cost and latency than replaying the complete history of long, multi-session dialogues ([Chhikara et al. — Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#)).
- **Sleep-time compute** uses the idle periods between requests. The agent processes its context offline, anticipating likely future questions and precomputing inferences so that later queries require less test-time compute. On some benchmarks, this approach reduced the inference budget needed for a given level of accuracy by roughly 5× ([Lin et al. — Sleep-time Compute: Beyond Inference Scaling at Test-time](#)).

The harness-level caveat is the same one introduced in §3.1: managed memory is powerful but lossy, and it creates new failure modes. The system may retrieve the wrong memory, consolidate information incorrectly, or assert a stale fact with confidence. These architectures justify their complexity when sessions are long and cross-session recall genuinely matters. For short tasks, they may add overhead without providing more value than the structured notes described in §3.2.

3.9 Multi-Agent Topologies and Why They Fail

Section 3.4 introduced sub-agents as context firewalls and described the orchestrator-worker configuration; Chapters 6 and 7 develop orchestration in more detail. Practitioners also use several other multi-agent *topologies*:

- **Orchestrator–worker (supervisor):** a lead agent decomposes a task, delegates parts to workers, and synthesizes their results (Ch 6, 7).
- **Hierarchical:** supervisors manage other supervisors when a task decomposition is too deep for one lead to hold in context.
- **Blackboard / shared memory:** agents coordinate by reading and writing to a common workspace instead of messaging one another directly. This works well when many agents contribute to a single, evolving artifact.
- **Debate / voting:** several agents debate or vote to improve reliability. This is the multi-agent form of the parallelization-voting pattern (§6.3).

It is tempting to equate more agents with greater capability, but the empirical record is more sobering. The MAST study manually annotated more than 200 tasks across seven popular multi-agent frameworks and produced a taxonomy of 14 failure modes. It groups them into three categories: **specification issues** (underspecified roles and prompts), **inter-agent misalignment** (agents talking past one another, dropping information, or diverging from the shared goal), and **task verification** (weak or absent checks of the final result) ([Cemri et al. — Why Do Multi-Agent LLM Systems Fail?](#)). Many failures therefore come not from a lack of raw model capability, but from breakdowns in *coordination and verification*—precisely the surfaces that the harness controls.

The taxonomy leads directly to practical guidance. Prefer the simplest topology that works (§6.1). Define task boundaries explicitly so that workers neither duplicate nor omit work (§3.4, Ch 7). Treat verification as a first-class responsibility rather than an afterthought—the generator–evaluator split described in Ch 7—because MAST identifies weak verification as one of the three main families of failure.

3.10 Memory Poisoning and Trust Escalation

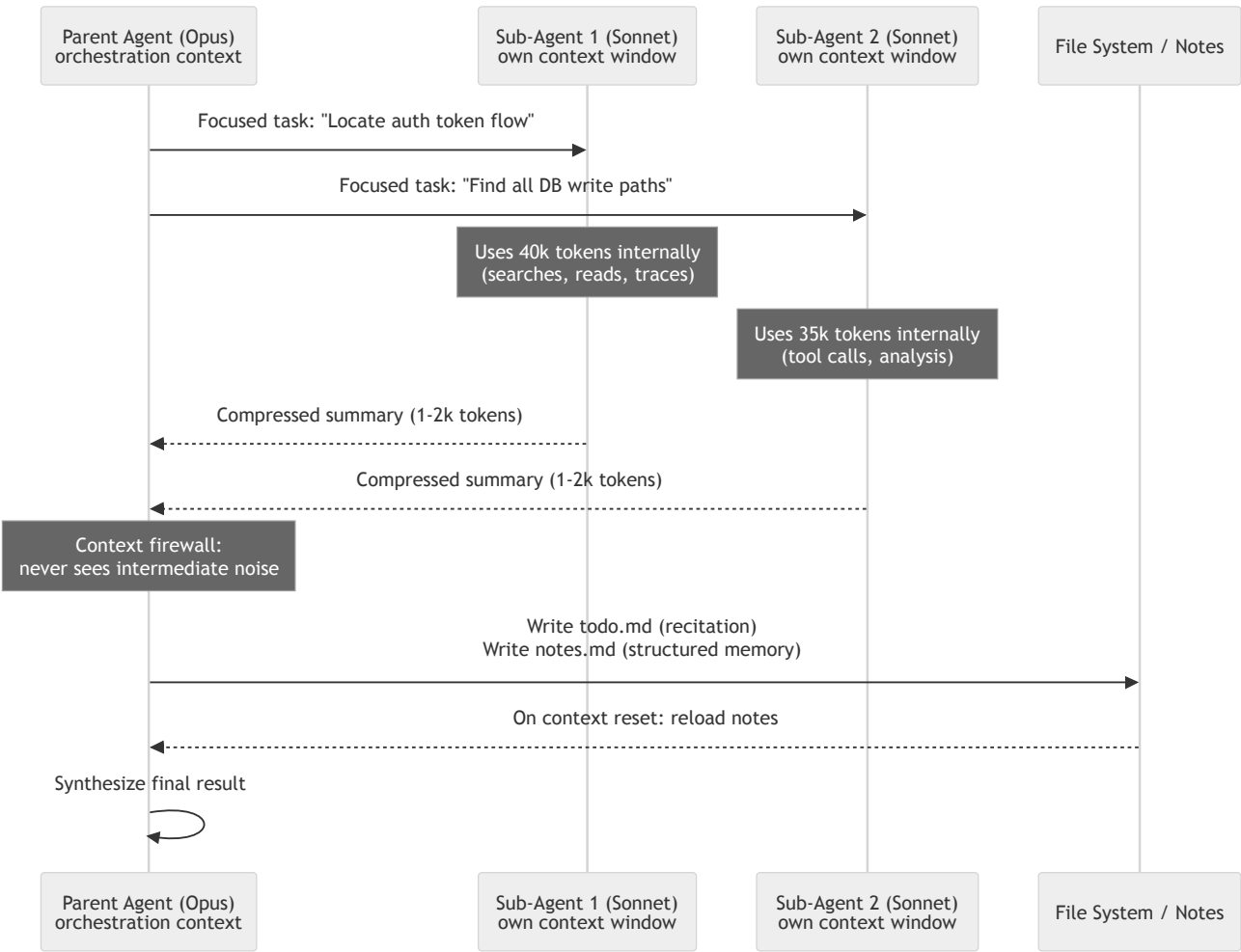
Persistent memory changes the security model. An injected instruction on a web page would normally threaten only one run. If the agent incorporates that instruction into a durable note, preference store, or shared blackboard, however, the attack can survive a context reset and influence later runs. Anthropic calls this **persistent memory poisoning**: untrusted content crosses a write boundary into state that future agents treat as trusted ([Anthropic — How We Contain Claude](#)).

The defense is to treat memory as a provenance-bearing store, not as a neutral extension of context:

- label every memory with its source, the identity that authorized the write, its creation time, and its trust class;
- separate observations from instructions, and never promote retrieved content to policy automatically;
- require review or deterministic validation before untrusted findings enter durable, shared memory;
- support expiry, supersession, and rollback so that a poisoned fact can be removed from all future contexts;
- check authorization again at retrieval time, because a fact visible to the writer is not necessarily one that the reader may access.

Multi-agent systems introduce another risk: **trust escalation**. A low-privilege worker can return a plausible summary to a higher-privilege coordinator, which may then act with authority the worker never possessed. Compression makes this path especially dangerous because provenance and uncertainty are often the first details to disappear. A sub-agent result should therefore include citations or pointers to artifacts, a confidence level and any unresolved questions, and the identity and permissions under which the work was performed. Before turning that result into a privileged action, the parent must validate the evidence. Chapter 5 develops the relevant containment controls; Chapter 18 describes the identity and control-plane model that makes them enforceable across a fleet.

Diagram: Parent Agent → Sub-Agent → Compressed Result (Context Firewall)



Key Takeaways

- **Compaction extends task horizons but loses detail:** preserve key decisions and restorable references, not every raw observation.
- **Structured note-taking enables multi-session continuity:** agents that write progress to disk can resume work after context resets.
- **Recitation defeats "lost-in-the-middle":** repeatedly rewriting a todo list pushes goals into the model's recent attention span.
- **The context firewall is the sub-agent pattern's key value:** the parent never sees intermediate noise; it receives only condensed results.
- **Leave useful errors in context:** self-healing only works when the relevant error trace is visible, but repeated failures should be compacted.
- **Cost and latency are design variables:** route cheap work to small models, keep the KV-cache warm, and parallelize for wall-clock speed.
- **Named memory systems package the memory patterns:** MemGPT (OS-style virtual context), Mem0 (extract-consolidate-retrieve), and sleep-time compute (offline pre-processing) are useful reference points, but each introduces its own retrieval and staleness failure modes.
- **More agents multiply coordination failures, not just cost:** the MAST taxonomy finds that specification gaps, inter-agent misalignment, and weak verification—all surfaces owned by the harness—dominate multi-agent failures. Prefer the simplest topology and make verification first-class.
- **Durable memory is a trust boundary:** retain provenance, separate data from instructions, and validate before promoting untrusted findings into shared state; otherwise one injected page can steer many future runs.
- **Delegation must not silently escalate authority:** a parent should verify a worker's evidence before acting with privileges the worker did not possess.

Further Reading

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
- Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
- Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>

Chapter 4: Tools and the Agent–Computer Interface

4.1 Why Tool Design Is Different

Anthropic uses the term *agent–computer interface* (ACI) by analogy with HCI: designing how an agent uses tools deserves as much engineering attention as designing how a human uses a screen ([Anthropic — Building Effective Agents](#)). It offers three concrete recommendations for tool formats:

- Give the model enough tokens to "think" before it must commit to syntax that is difficult to revise.
- Keep formats close to what the model has seen in training data.
- Avoid unnecessary formatting work, such as exact line counting in diff headers or excessive string escaping in code embedded in JSON.

When Anthropic built its SWE-bench agent, the team spent more time optimizing tool schemas than optimizing the prompt itself. One change illustrates the impact: switching from relative to absolute file paths eliminated nearly all path-related errors that occurred after the agent moved out of the repository root ([Anthropic — Building Effective Agents](#)).

4.2 Choosing the Right Tools — and the Right Number

Anthropic's later "Writing Effective Tools for Agents" develops a central warning: giving an agent more tools does not necessarily improve its results ([Anthropic — Writing Effective Tools for Agents](#)). A common mistake is to wrap every API endpoint as a tool, whether or not that endpoint suits the way an agent works. Agents need different *affordances* from traditional software. For example, an agent using `list_contacts` to search an address book must scan every contact token by token, consuming limited context. A purpose-built `search_contacts` or `message_contact` tool gives it a more direct path.

Good tools also consolidate operations that agents frequently chain together. Instead of exposing `list_users`, `list_events`, and `create_event` separately, provide `schedule_event`. Replace `read_logs` with `search_logs`. Combine `get_customer_by_id` + `list_transactions` + `list_notes` into `get_customer_context`.

4.3 Where Tools Come From: The Model Context Protocol

The rest of this chapter assumes that an agent has access to a well-designed set of tools. In practice, many of those tools are delivered through a standard interface: the *Model Context Protocol* (MCP). Briefly, MCP is an open client–server standard. An *MCP server* exposes tools—and optionally resources and reusable prompts—through a uniform protocol. Any compatible *client*, such as Claude Code, an IDE, or a custom agent, can then discover and call them without a bespoke integration. Foundations ch 12 covers the transport mechanics and the risks of untrusted results; here, the focus is MCP as a surface for harness design ([Anthropic — Code Execution with MCP](#)).

MCP's main value is composability. A team can connect independently maintained Google Drive, Salesforce, and internal database servers to the same agent, which sees them as one combined tool surface. This is why MCP recurs throughout this book: it provides the substrate for the namespacing, masking, and code-execution patterns discussed below.

The downside is that MCP also makes it trivial to provide too many tools. As Chapter 2 noted, users can connect hundreds of them, leaving the model's context crowded with tool definitions. That is exactly the kind of bloat addressed by Chapter 2's *mask, don't remove* rule and by the consolidation advice above. MCP is plumbing, not a substitute for tool design: a poorly designed MCP server merely delivers poorly designed tools at scale. The practices in this chapter—consolidating frequently chained operations, namespacing tools, capping responses, and writing descriptions like onboarding documentation—apply whether a tool is hand-written or delivered through MCP.

MCP also creates a deployment boundary. An internal server should not have to become public simply because a hosted agent needs to reach it. OpenAI's secure MCP tunnel places an **outbound-only client** inside the private network. The client connects to an explicitly configured destination, preserves streaming and authentication, and remains an inspectable process under the customer's control; no inbound port is required ([OpenAI — Connect Private MCP Servers to OpenAI Products](#)). The broader pattern goes beyond networking: access to a private capability should be initiated by the side that owns it, restricted to named destinations, and audited as delegated authority.

4.4 Four Integration Boundaries

The OpenReview survey argues that tool and protocol standards are easiest to compare by the boundary they cross, rather than by vendor or release date ([OpenReview — Agent Harness Engineering: A Survey](#)):

- **Model -> Function:** structured invocation such as function calling, where the model emits a machine-readable request for deterministic code.
- **Agent -> External capability:** MCP-style decoupling, where an agent runtime discovers tools, resources, and prompts exposed by external servers.
- **Agent -> Agent:** A2A-style delegation, where one agentic application hands work to another opaque agent with its own state and tools.
- **Agent -> Repo/environment:** version-controlled policy and affordances, such as AGENTS.md, local skills, repo commands, and environment-specific tool rules.

This view explains why MCP, A2A, OpenAPI, function calling, and AGENTS.md are not direct substitutes: each addresses a different integration problem. A harness designer must first identify the boundary involved, then choose a protocol and governance model that preserves provenance, permissions, cost information, and evidence of failure across it.

4.5 Namespacing

When agents can access dozens of MCP servers and hundreds of tools, name collisions and unclear purposes become serious failure modes. Anthropic recommends grouping related tools under shared prefixes: service-level prefixes such as `asana_*` and `jira_*`, followed by resource-level prefixes such as `asana_projects_*` and `asana_users_*`. Its evaluations show that choosing prefix or suffix namespacing can materially affect tool-use performance, and that the best scheme depends on the workload ([Anthropic — Writing Effective Tools for Agents](#)).

Manus applies the same pattern to control the agent's action space. By prefixing all browser tools with `browser_` and all shell tools with `shell_`, the harness can mask entire tool groups at once using simple logit constraints ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

4.6 Returning Meaningful Context

Tool responses should favor relevance over flexibility and human-readable identifiers over opaque technical ones. Anthropic finds that replacing alphanumeric UUIDs with meaningful labels—or even zero-indexed IDs—significantly improves Claude's precision and reduces hallucinations ([Anthropic — Writing Effective Tools for Agents](#)). When an agent needs readable names but downstream calls still require technical IDs, a `response_format` enum with `concise` and `detailed` modes works well. In Anthropic's Slack examples, concise responses can be one-third the size of detailed ones.

4.7 Token-Efficient Responses

Tool responses are a major source of context bloat. Anthropic limits Claude Code's tool responses to 25,000 tokens by default and recommends combining sensible defaults with pagination, range selection, filtering, and truncation ([Anthropic — Writing Effective Tools for Agents](#)). When a response is truncated, it should guide the agent toward a more efficient strategy—for example, several targeted searches instead of one broad search. Errors should likewise offer useful next steps rather than only an opaque traceback.

HumanLayer applies this idea through "back-pressure": its build and test hooks stay silent on success and surface only errors. Earlier, the team had the agent run the full test suite after every change. Four thousand lines of passing test output would flood the context window, causing the agent to lose track of its task and begin hallucinating about test files ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

4.8 Prompt-Engineering Tool Descriptions

Anthropic describes tool descriptions as one of the most effective design levers. It reports that refining them was a key factor in achieving state-of-the-art results on SWE-bench Verified with Claude Sonnet 3.5 ([Anthropic — Writing Effective Tools for Agents](#)). Write each description as if you were onboarding a junior engineer. Spell out context that might otherwise remain implicit, including specialized query formats, domain-specific terminology, and relationships between resources. Prefer unambiguous parameter names such as `user_id` over `user`. Then run many examples in a workbench, study the mistakes, and iterate.

One debugging example shows the value of this approach. When Anthropic launched Claude's web search tool, traces revealed that Claude was unnecessarily appending `2025` to the `query` parameter and biasing the results. The fix required no model retraining, only a clearer tool description.

4.9 Code Execution as a Meta-Tool

A newer approach is to expose MCP tools through a code API rather than as direct calls, allowing the agent to invoke them by writing code. Anthropic's "Code Execution with MCP" argues that the standard pattern becomes wasteful when an agent has hundreds of tools spread across dozens of MCP servers: every tool definition is loaded into context up front, and every intermediate result passes through the model ([Anthropic — Code Execution with MCP](#)).

The alternative is to represent MCP servers as a filesystem of TypeScript files, with one typed `callMCPTool` wrapper per tool. The agent discovers tools by listing directories and reads only the files it needs. In Anthropic's Google Drive → Salesforce example, this reduces token use from 150,000 to 2,000—a 98.7% saving.

This design has several reinforcing benefits:

- **Progressive disclosure:** tools are loaded only when needed, addressing the up-front context cost.
- **Context-efficient results:** the agent can filter a 10,000-row spreadsheet to five matching rows in the execution environment before any data crosses into the model's context.
- **Better control flow:** loops, conditionals, and error handling use familiar code patterns. The runtime, rather than the model, evaluates conditions, reducing time to first token.
- **Privacy-preserving operations:** intermediate results stay in the execution environment by default; only what the agent explicitly logs reaches the model. With a correctly designed proxy, PII can be tokenized at the MCP-client boundary so raw values need not reach the model.
- **State persistence and skills:** agents can save working code as reusable functions backed by `SKILL.md` files, building up a toolbox over time.

Cloudflare reported similar findings under the name "Code Mode." Together, these results support a straightforward conclusion: LLMs are good at writing code, so tool interfaces should make use of that ability ([Anthropic — Code Execution with MCP](#)).

OpenAI's **programmatic tool calling** applies the same idea within a Responses API turn. The model writes a short program that calls permitted tools, filters or joins their results, and returns only the compact output to the model. This works well for bounded dataflow operations—filtering, joining, ranking, deduplicating, aggregating, and validating—because deterministic runtime control replaces many model round trips. It works poorly when each observation substantially changes the next judgment, when every action requires separate approval, or when all source results must remain visible for citation or review ([OpenAI — Programmatic Tool Calling](#)). Use code to compress mechanical orchestration, not to conceal consequential decisions.

Code execution does introduce a tradeoff: it requires sandboxing infrastructure, with its own operational and security costs. It also changes what must be audited. The harness needs to record the program, the tools it was permitted to call, and the side effects it produced—not just the final compact result.

4.10 Iterative Tool Refinement With Evals

Anthropic's recommended workflow for tool development has four stages ([Anthropic — Writing Effective Tools for Agents](#)):

1. **Prototype** the tools in a local MCP server, test by hand, collect intuition.
2. **Build an evaluation** with realistic tasks (multiple tool calls, real data, no toy sandboxes), each paired with verifiable success criteria.
3. **Run the evaluation** programmatically, capturing traces that include planning summaries, tool calls, tool results, runtime, token counts, and tool errors. Where a model exposes a visible thinking mode, that can help debug behavior, but the eval should not depend on access to hidden chain-of-thought.
4. **Analyze results** by reading transcripts, paying attention to what agents *do not* say (LLMs do not always state what they mean), and refactor the tools accordingly.

Anthropic applied this loop to its internal Slack and Asana tools. On held-out test sets, versions optimized with Claude outperformed expert-built implementations. The result both supports the workflow and offers an early example of agents improving their own tools.

4.11 The Agent-to-Agent Boundary: A2A

The preceding sections cover three of the four integration boundaries in §4.4: model-to-function, agent-to-external-capability (MCP), and agent-to-repo/environment. The fourth, *agent-to-agent*, has an emerging standard of its own. The **Agent2Agent (A2A) protocol**, introduced by Google in April 2025 and donated to the Linux Foundation in June

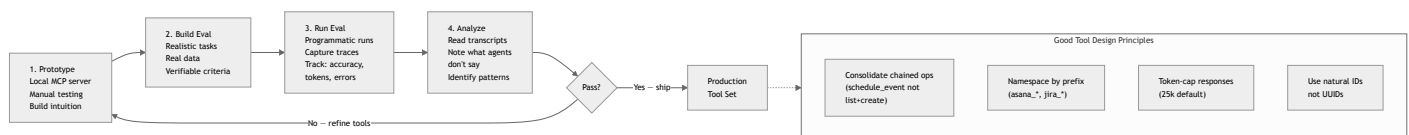
2025, is an open standard that lets one agentic application delegate work to another *opaque* agent—a peer with its own model, tools, memory, and hidden internal state ([Agent2Agent \(A2A\) Protocol](#); [Linux Foundation — Agent2Agent Protocol Project](#)).

The protocol deliberately uses conventional web mechanics. Communication runs over JSON-RPC 2.0 and HTTP(S). Each agent publishes an *Agent Card* that describes its capabilities for discovery. A task lifecycle then covers submission, negotiation of an interaction format—text, files, or structured data—and streaming results back to the caller.

The contrast with MCP is what matters. MCP connects an agent *downward* to tools and resources that it controls and can inspect. A2A connects an agent *across* to a peer that it cannot control or inspect. The primary engineering concern therefore shifts from tool-schema design to *trust and provenance across an organizational boundary*. Because the caller cannot inspect the peer's context or sandbox, an A2A response is untrusted content in the sense defined by Chapter 5. The lethal-trifecta discipline applies to peer agents just as it does to web pages, and governance—scoped identity, delegated authorization, and audit—must cover the A2A call rather than end at the local process (Ch 5, Ch 17). A2A does not eliminate the trust problem; it standardizes the boundary at which that problem must be solved.

This boundary also suggests a practical topology rule. Native multi-agent execution is most useful when tasks are **independent, bounded, and mergeable**, as with parallel research threads, isolated reviews, or separate artifacts. It is usually a poor fit for sequential dependencies or for several writers changing the same state, where coordination overhead and race conditions dominate. In those cases, retain a single owner and use ordinary tools or a deterministic workflow. When peer agents are appropriate, the control plane described in Chapter 18 must still resolve identity, delegated authority, version, lineage, and revocation for every call.

Diagram: Tool Design Pipeline — Prototype → Eval → Iterate



Key Takeaways

- **Tool design deserves as much care as prompt design:** the ACI (agent–computer interface) is to agents what HCI is to people.
- **More tools can hurt rather than help:** consolidate frequently chained operations into purpose-built tools.
- **MCP standardizes where tools come from:** it makes tools composable across independent servers, but also makes over-supply easy—tool-design discipline still applies.
- **Tool protocols cross different boundaries:** function calling, MCP, A2A, and repo-local policy are complementary, not interchangeable.
- **Namespacing is not cosmetic:** it enables logit-level masking of tool groups and prevents collisions in large MCP environments.
- **Tool responses are a major source of context bloat:** cap, paginate, filter, and truncate by default.
- **Code execution as a meta-tool is a step change:** exposing MCP tools as a typed code API cut token use by 98.7% in Anthropic's example.
- **Programmatic tool calling is for bounded dataflow:** filter, join, rank, deduplicate, aggregate, and validate in code; keep adaptive judgment, approvals, and citation-bearing evidence visible to the agent.
- **Private MCP connectivity should be outbound-only:** initiate the connection from the network that owns the capability, constrain its destination, and audit it as delegated authority.
- **The four-stage eval loop is the recommended workflow:** prototype → build eval → run eval → analyze transcripts → iterate.
- **A2A standardizes the agent-to-agent boundary:** it lets one agent delegate to an opaque peer through JSON-RPC and Agent Cards, shifting the concern from tool-schema design to trust and provenance across an organizational boundary, where the lethal-trifecta and governance rules of Ch 5 apply.
- **Parallel agents need independent ownership:** shared mutable state and sequential dependencies usually call for one owner plus deterministic orchestration.

Further Reading

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
- Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
- OpenAI, *Connect Private MCP Servers to OpenAI Products*, Jun 2026. <https://developers.openai.com/blog/connect-private-mcp-servers-to-openai-products>
- OpenAI, *Programmatic Tool Calling*, 2026. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>

Chapter 5: Sandboxing, Guardrails, and Safe Autonomy

5.1 The Agent Security Threat Model

Most of this chapter covers *mitigations*: sandboxes, hooks, and approval gates. Before discussing them, we need to be explicit about the risks they address. An agent that can read untrusted content and act on external systems has a distinctive threat profile ([Anthropic — Beyond Permission Prompts](#)):

- **Prompt injection** (see Foundations ch 8, "Prompt Injection as Context Confusion," and ch 12) — the model cannot reliably distinguish data from instructions. Untrusted content that the agent reads—a web page, issue comment, source file, or tool result—can therefore steer it as if that content were a command.
- **Data exfiltration** — a compromised or steered agent with network access can send secrets, such as SSH keys, API tokens, or proprietary source code, to an attacker-controlled destination.
- **Destructive action** — a steered agent with filesystem or shell access can delete or corrupt files, or commit and push harmful code.
- **Tool and supply-chain risk** — a malicious or compromised MCP server, package, or dependency can introduce hostile tools or instructions that the agent may then trust.

The combination practitioners worry about most is sometimes called the *lethal trifecta*: access to private data, exposure to untrusted content, and the ability to communicate externally ([Simon Willison — The lethal trifecta for AI agents](#)). Each capability can be manageable on its own. When one agent has all three, however, a single injected instruction can read a secret and send it to an attacker. Most controls in this chapter work by breaking one leg of the trifecta. Network isolation removes external communication, filesystem isolation removes access to private data, and approval gates place a human in the path of consequential actions.

The key premise for the rest of the chapter is that the model is not a trusted component. It is capable, but it can be steered. The harness is what stands between a hostile instruction and a real-world consequence.

Anthropic's later containment work sharpens this threat model along two dimensions ([Anthropic — How We Contain Claude](#)). Risk can originate from a **misusing user**, from **model misbehavior**, or from an **external attacker** steering the model through content. Defenses, in turn, can be placed in the **model**, the **execution environment**, or at the **external-content boundary**. This matrix matters because no single defensive layer covers every source of risk. Alignment cannot guarantee that a model will ignore an injection, while a sandbox cannot determine whether a permitted email is semantically harmful. Production containment therefore requires defense in depth across all three layers.

5.2 The Permission Fatigue Problem

Coding agents that operate without oversight are dangerous, but agents that request permission for every action are impractical. Anthropic describes the resulting problem as approval fatigue: "Constantly clicking 'approve' slows down development cycles and can lead to 'approval fatigue,' where users might not pay close attention to what they're approving, and in turn making development less safe" ([Anthropic — Beyond Permission Prompts: Making Claude Code More Secure and Autonomous](#)). The solution is structural: establish boundaries within which the agent can act freely, and request permission only when an action crosses one of them.

In Anthropic's internal use, sandboxing safely reduced permission prompts by 84% ([Anthropic — Beyond Permission Prompts](#)).

5.3 Sandbox as Cage, Reset Button, and License

The OpenReview survey presents the sandbox as more than a security mechanism. In an agent system, it serves three purposes at once ([OpenReview — Agent Harness Engineering: A Survey](#)):

- **Security**: it bounds the blast radius of unpredictable model-generated actions and prompt-injection-driven behavior.
- **Reproducibility**: it gives evals, training trajectories, and long-running sessions a resettable baseline. A container or microVM can be destroyed and rebuilt; a developer workstation cannot.
- **Liveness**: it defines a region where the agent is allowed to act without asking a human on every file write, package install, or network call.

The third purpose is distinctive to agent systems. A sandbox is not only a cage; it is also a license to act. By moving permission decisions from individual actions into session-level configuration, it makes long-horizon autonomy

practical without producing constant approval prompts.

Containment should scale with the work. Three recurring patterns form a useful ladder ([Anthropic — How We Contain Claude](#)):

- An **ephemeral container** is disposable, cheap, and appropriate for bounded tasks where the agent needs broad freedom inside a small blast radius.
- A **human-in-the-loop sandbox** supports interactive work: safe operations proceed automatically, while boundary crossings suspend for approval.
- A **sealed virtual machine** isolates higher-risk workloads behind a stronger kernel and network boundary, at greater startup and operational cost.

The useful question is not simply "Is it sandboxed?" Instead, ask which resources remain reachable, which state survives a reset, and what authority can cross the boundary. Resettable compute does not neutralize a credential mounted inside the sandbox, a poisoned memory written outside it, or an egress path capable of transmitting private data.

5.4 Filesystem and Network Isolation Must Be Paired

Claude Code's sandbox enforces filesystem and network boundaries together, and Anthropic argues that both are necessary. Filesystem isolation prevents a prompt-injected agent from modifying sensitive files. Network isolation prevents it from leaking data or downloading malware. Without the network boundary, a compromised agent could exfiltrate SSH keys. Without the filesystem boundary, it could escape the sandbox and gain network access ([Anthropic — Beyond Permission Prompts](#)).

The implementation builds on OS-level primitives—Linux bubblewrap and macOS seatbelt—and applies to subprocesses as well as direct Claude Code interactions. Network traffic passes through a Unix domain socket to a proxy, which enforces domain restrictions and asks the user to confirm newly requested domains. The runtime is open source.

Claude Code on the web extends the design to a cloud sandbox in which sensitive credentials, including git credentials and signing keys, never reside inside the sandbox with the agent. A custom proxy handles git interactions and attaches scoped credentials only after confirming that an operation is permitted—for example, that a push targets the configured branch.

An egress allowlist is a **capability grant**, not merely a list of harmless destinations. Access to a package registry permits the agent to download executable code. Access to a source host may permit it to publish content. Access to a general web endpoint can complete the exfiltration leg of the lethal trifecta. Policies should therefore bind network access to a destination, protocol, operation, identity, and task—not to a domain alone—and log the rule that authorized each connection.

Containment must also begin **before trust is established**. Opening a repository can trigger configuration loading, dependency discovery, language servers, hooks, or local listeners before the user sees a trust dialog. Treat project-open and configuration-load paths as hostile. Parse without executing where possible, disable automatic hooks and listeners, and withhold credentials and egress until the workspace has been explicitly trusted ([Anthropic — How We Contain Claude](#)).

5.5 Governance: Identity, Policy, and Audit

Sandbox boundaries are necessary, but they are not sufficient. The governance layer asks who the agent represents, what authority it holds, how that authority changes with the task context, and what evidence remains after an action ([OpenReview — Agent Harness Engineering: A Survey](#)).

Three design moves matter in production:

- **Identity and delegated auth**: the agent should act through scoped credentials or delegated identity, not with a user's full ambient authority. Credential vaults and proxies should attach secrets only at the boundary where the operation is authorized.
- **Context-dependent permission policy**: static allow/deny lists are inspectable but blunt. Task-aware policies can evaluate tool name, arguments, session state, target repo, network domain, and user role before each invocation, while a deterministic checker enforces the result.
- **Audit-grade traces**: logs must capture not only the tool call, but the identity, permission decision, policy version, arguments, output summary, and whether a human approved escalation.

This is also where supply-chain attacks become harness concerns. MCP tool poisoning, tool squatting, rug-pull updates, hallucinated packages, and retrieval-source poisoning all cross the boundary between tool interfaces and governance. A safe harness needs provenance and integrity checks for tools, packages, datasets, and retrieval sources; a prompt that merely says "be careful" is not enough.

5.6 Hooks and Middleware as Programmatic Enforcement

A sandbox is one kind of programmatic guardrail. Hooks and middleware provide a finer-grained alternative. Claude Code supports user-defined commands or scripts that run automatically at lifecycle events, such as agent start, after a tool call, or on stop ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)). LangChain's middleware follows a similar structure. Some hooks are fully deterministic scripts; others are procedural checkpoints that inject context back into the model. Their reliability comes from automatic execution by the harness, not from the model remembering a rule.

Common uses include notifications, automated approvals or denials, integrations, and verification. A hook might play a sound when an agent finishes, deny a database migration and ask the user to run it, post a Slack message, open a pull request, or run type checking and a build before the agent stops. HumanLayer's example runs Biome and TypeScript in parallel whenever Claude attempts to stop. It exits silently on success; on failure, it returns only the errors with exit code 2, causing the harness to re-engage the agent.

LangChain reports that this kind of middleware helped move deepagents-cli from the Top 30 to the Top 5 on Terminal-Bench 2.0. Its `PreCompletionChecklistMiddleware` intercepts the agent before exit and prompts it to verify its work against the task specification. `LocalContextMiddleware` runs at startup to map the working directory and discover available tools. `LoopDetectionMiddleware` counts edits to each file and, after N edits to the same file, asks the agent to reconsider its approach. This can break "doom loops" in which the agent repeatedly tries small variations of a failing strategy ([LangChain — Improving Deep Agents with Harness Engineering](#)).

The survey's governance taxonomy places these hooks within a broader enforcement pipeline. Pre-invocation checks can deny unsafe tool calls. Post-invocation hooks can mark or redact untrusted outputs before they enter context. Stop hooks can require verification or audit updates, and escalation hooks can route ambiguous cases to humans. The more consequential an action is, the less its safety should depend on the model remembering an instruction.

5.7 Feedforward and Feedback: A Cybernetic View

Thoughtworks' Birgitta Böckeler offers a higher-level taxonomy for these controls ([Thoughtworks — Harness Engineering](#)). Outer-harness controls operate in two directions:

- **Guides (feedforward)** anticipate the agent's behavior and steer it before it acts. Examples include instructions in AGENTS.md, skills, reference documentation, and language-server hints. Their goal is to increase the probability of a good result on the first attempt.
- **Sensors (feedback)** observe the agent's work after it acts and help it self-correct. Tests, linters, type checkers, and AI code review all serve this role.

A harness with only feedforward guides keeps issuing rules but never determines whether they work. A harness with only feedback sensors repeatedly catches mistakes without helping prevent them. Both are necessary.

Each direction can be divided along a second axis:

- **Computational** controls — linters, type checkers, structural tests — are deterministic, run in milliseconds to seconds, and produce reliable results.
- **Inferential** controls — semantic analysis, AI code review, LLM-as-judge — handle nuance but are slower, more expensive, and non-deterministic.

The two axes are independent. Coding conventions in AGENTS.md are inferential feedforward; ArchUnit tests that check module boundaries on commit are computational feedback. A `/code-review` skill provides inferential feedback, while a pre-bootstrap script that creates the project structure is computational feedforward. A well-engineered harness combines all four types.

5.8 Three Regulation Categories

Böckeler also distinguishes harnesses by the qualities they regulate ([Thoughtworks — Harness Engineering](#)):

- **Maintainability harness** — regulates internal code quality, duplication, complexity, coverage, and style. This is the easiest category because it can draw on a long history of existing tools.

- **Architecture fitness harness** — regulates performance, observability, and debuggability. It captures the application's cross-cutting "fitness functions."
- **Behavior harness** — asks whether the application behaves as intended. This remains the least-solved category. Most teams currently use functional specifications as feedforward and AI-generated test suites as feedback, sometimes supplemented by mutation testing. Böckeler is candid that trusting AI-generated tests "is not good enough yet."

These categories make harness coverage easier to assess. A harness that is strong on maintainability but weak on behavior can create a false sense of safety.

5.9 Timing: Keep Quality Left

Continuous integration teaches that the earlier a problem is found, the cheaper it is to fix. The same principle applies to harness design. Fast computational sensors, such as linters and quick tests, should run before commit. More expensive computational and inferential sensors, such as mutation testing and broader code review, can run after integration in the pipeline. Continuous-drift sensors—including dead-code detection, dependency scanners, and log-anomaly judges—operate outside the change lifecycle altogether ([Thoughtworks — Harness Engineering](#)).

As Böckeler notes, the OpenAI Codex team's harness follows the same pattern: custom linters and structural tests enforce a layered architecture, while recurring "garbage collection" passes scan for drift and ask agents to suggest fixes.

5.10 Harnessability, Agentic Readiness, and Ambient Affordances

Not every codebase is equally easy to harness. A strongly typed language provides type-checking sensors by default. Clear module boundaries make architectural constraints enforceable. Opinionated frameworks such as Spring hide details that the agent would otherwise need to manage ([Thoughtworks — Harness Engineering](#)).

The term *ambient affordances*, which Böckeler credits to Ned Letcher, captures these properties of the environment that make it legible, navigable, and manageable for agents ([Thoughtworks — Harness Engineering](#)). Greenfield teams can design these affordances from the start. Legacy teams face the opposite situation: a harness is often most necessary where it is hardest to build.

Looking ahead, Böckeler proposes *harness templates*: bundles of guides and sensors for specific service topologies, such as a JVM CRUD service, a Go event processor, or a Node dashboard. These could accompany existing service templates. She invokes Ashby's Law of Requisite Variety to formalize the argument: a regulator must have at least as much variety as the system it governs. Choosing a constrained topology reduces that variety and makes a comprehensive harness more achievable ([Thoughtworks — Harness Engineering](#)).

The broader production concept is **agentic readiness**: can an autonomous caller safely understand, invoke, observe, retry, and, when necessary, undo an operation? A service may be easy for a human developer to use yet hostile to an agent if its APIs conceal side effects, return ambiguous errors, or lack stable operation identifiers. Readiness improves when:

- mutations accept idempotency keys and expose operation status;
- APIs distinguish read, propose, commit, and compensate rather than hiding them behind one opaque call;
- machine identity and delegated authorization are first-class;
- errors state what failed, whether a retry is safe, and what evidence would prove recovery;
- state changes are observable, attributable, and reversible through compensating actions where literal undo is impossible.

These features act as ambient affordances for autonomous software. They reduce the amount of probabilistic reasoning required from the model and give the Chapter 18 control plane stable surfaces for policy, lifecycle management, and audit.

5.11 Operational Safety: Circuit Breakers, Kill Switches, Budgets, and Canaries

Sandboxes, governance, and hooks (§5.3–5.6) constrain what an agent *may* do. A second family of controls limits the consequences when the agent or its tools *misbehave at runtime*. These controls come almost unchanged from distributed-systems reliability and security operations. They belong in the harness because an agent that has been steered or caught in a loop cannot be trusted to apply them to itself.

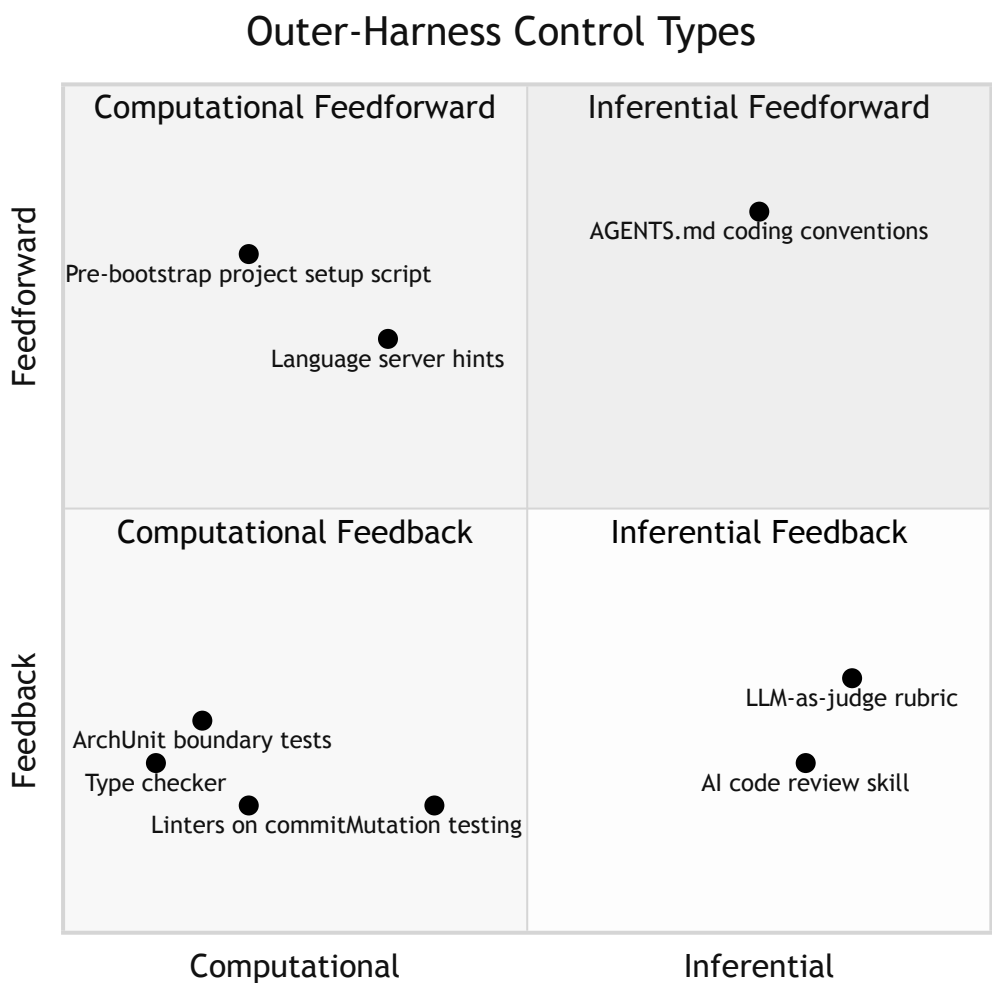
- **Circuit breakers.** Wrap a flaky or expensive dependency—a tool, downstream service, or sub-agent—so that the breaker trips after a threshold of failures. Subsequent calls fail fast instead of hanging or producing a retry storm

(Fowler — CircuitBreaker). For agents, this limits the blast radius of an erroring tool or an agent that repeatedly retries the same failing action. It complements the Chapter 3 advice to "keep the wrong stuff in": that pattern keeps one failure visible, while the breaker prevents the failure from recurring without limit.

- **Kill switches.** Provide a human- or policy-triggered mechanism that can halt one agent or an entire fleet immediately, independently of the agent's own control flow. A prompt-injected agent may be actively working against its instructions, so the switch must live in the harness—as a supervisor process, revocable credential, or sandbox teardown—not in a prompt that says "stop if asked."
- **Action budgets, iteration caps, and cost governors.** Set hard limits on tool calls, tokens, wall-clock time, or spending. When a limit is reached, the loop stops and escalates rather than continuing unchecked. This is the operational form of the loop stop rules in Chapter 8 and the per-task budgets in Chapter 17: an unbounded loop creates both an unbounded bill and an unbounded blast radius.
- **Canary tokens.** Place fake secrets—an unused API key, decoy file, or tripwire URL—where a prompt-injected agent might try to read or exfiltrate them. A callback from a canary is a high-signal warning that the agent has been steered toward data it should not touch (Thinkst — Canarytokens). Unlike a sandbox, a canary does not *prevent* the exfiltration leg of the lethal trifecta (§5.1); it *detects* it. That makes it a useful final line of defense when prevention is imperfect.

The principle is consistent throughout the chapter: the more consequential a failure is, the less its prevention should depend on the model choosing to avoid it. Prevention through sandboxes and policy works together with detection through canaries and the drift alerts discussed in Ch 17. Neither is sufficient alone.

Diagram: Feedforward/Feedback × Computational/Inferential Quadrant



Key Takeaways

- **The threat model comes first:** prompt injection, data exfiltration, destructive action, and supply-chain risk all become especially dangerous when private data, untrusted content, and external communication form the *lethal trifecta*.

- **Sandboxing reduces permission prompts by 84%** while maintaining safety: structural boundaries are more effective than repeated approval dialogs.
- **Sandboxing has three jobs:** security, reproducibility, and liveness.
- **Filesystem and network isolation must be paired:** each addresses a different attack vector, and either alone is insufficient.
- **Containment is a matrix, not a single sandbox:** defend against misuse, model misbehavior, and external attack at the model, environment, and content boundaries; choose ephemeral containers, interactive sandboxes, or sealed VMs according to risk.
- **Egress is authority:** an allowed destination grants a real capability, so bind network access to the operation, identity, and task—and grant no ambient trust merely because a project has been opened.
- **Governance is more than approvals:** identity, scoped credentials, policy checks, provenance, and audit trails must compose across tools and sessions.
- **Hooks and middleware are programmatic enforcement:** they run regardless of model memory, making them more reliable than prompt-only constraints.
- **Feedforward and feedback are both required:** guides without sensors have no learning loop; sensors without guides react but don't prevent.
- **Three categories of harness coverage:** maintainability (well-tooled), architecture fitness (achievable), and behavior (the unsolved problem).
- **Ambient affordances matter:** strongly-typed languages and opinionated frameworks make harnessing easier from day one.
- **Agentic readiness is an API property:** idempotency, explicit operation status, machine identity, retry semantics, observable state, and compensating actions make systems safer for autonomous callers.
- **Runtime safety also requires operational controls:** circuit breakers, kill switches, action and cost budgets, and canary tokens limit misbehavior when it occurs. They belong in the harness because a steered agent cannot be trusted to stop itself.

Further Reading

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>

Chapter 6: Agentic Workflow Patterns

6.1 Workflows vs. Agents

Anthropic distinguishes two architectures within the broader category of "agentic systems" ([Anthropic — Building Effective Agents](#)):

- **Workflows** orchestrate LLMs and tools through predefined code paths.
- **Agents** dynamically direct their own processes and tool usage.

Anthropic's first principle is to use the simplest solution that works and add complexity only when the task demands it. Many use cases do not need agents at all: a single LLM call, supplemented by retrieval and in-context examples, is often enough. Workflows provide predictability and consistency for well-defined tasks. Agents are a better fit when the path cannot be specified in advance and the system needs model-driven decisions at scale.

6.2 The Augmented LLM

The basic building block is the *augmented LLM*: a model connected to retrieval, tools, and memory. Modern models can use these capabilities actively by generating queries, selecting tools, and deciding what information to retain ([Anthropic — Building Effective Agents](#)). MCP (see Ch 4) is an increasingly common way to make these capabilities available to the model.

6.3 Compositional Workflow Patterns

The following patterns progress from the simplest and most constrained to the most flexible:

Prompt chaining breaks a task into sequential steps. Each LLM call processes the previous call's output, with optional programmatic checks between steps. Use it when the task decomposes cleanly and simpler calls can improve accuracy enough to justify the added latency. For example, one call might draft marketing copy and a second translate it. The main failure mode is propagation: latency accumulates across the chain, and an early error can contaminate every later step. Add gates at important boundaries.

Routing classifies an input and dispatches it to a specialized path. Use it when inputs fall into distinct categories that benefit from different handling, and when those categories can be identified reliably. A customer-service system, for example, might route requests to refund, technical-support, or general-question pipelines. Because misclassified and ambiguous inputs can fail silently, instrument the classifier and monitor its error rate.

Parallelization runs LLM calls simultaneously and then aggregates their outputs. It has two common variants: *sectioning* divides the work into independent subtasks, while *voting* runs the same task multiple times. Use parallelization to reduce latency or when multiple perspectives improve confidence, as in vulnerability reviews with several prompts or content moderation with multiple votes. Voting multiplies token cost by the number of runs, however, and correlated errors can make unanimous results falsely reassuring.

Orchestrator-workers uses a central LLM to break down a task dynamically, delegate subtasks to worker LLMs, and synthesize their results. Unlike basic parallelization, the subtasks are not defined in advance. This pattern suits complex work whose decomposition depends on the input, such as a coding task that spans many files or research across many sources. Its main risk is uncontrolled fan-out, so cap both the number of workers and the total token budget.

Evaluator-optimizer places generation and critique in a loop, usually with one LLM producing an answer and another evaluating it. Use this pattern when the evaluation criteria are clear and iterative refinement produces measurable gains. Two signals indicate a good fit: human feedback reliably improves the output, and an LLM can plausibly provide similar feedback. Examples include literary translation with a critic and multi-round research with a relevance evaluator. Because the loop may never converge, set a maximum number of iterations; every round adds latency and cost.

6.4 Three Principles for Agent Implementation

Anthropic concludes with three implementation rules ([Anthropic — Building Effective Agents](#)):

1. **Maintain simplicity** in agent design.
2. **Prioritize transparency** by explicitly showing the agent's planning steps.
3. **Carefully craft the agent-computer interface** through tool documentation and testing.

Frameworks can accelerate early development, but their abstraction layers may obscure the prompts and tool calls that determine system behavior. Anthropic recommends starting with direct API calls while learning the shape of the problem. Adopt a framework once repeated patterns and operational requirements make the additional abstraction worthwhile.

6.5 The Micro-Agent Pattern

HumanLayer offers a pragmatic version of the same idea ([HumanLayer — 12-Factor Agents](#)). The "loop until done" pattern tends to hit a wall after roughly 10–20 turns. Beyond that point, accumulated context and per-turn errors cause agents to lose coherence (see Ch 2, Context as a Finite Resource).

A more reliable design embeds small, focused agents within a deterministic DAG—a directed acyclic graph of steps defined in code. LangChain describes the same harness shape ([LangChain — The Anatomy of an Agent Harness](#)). In its deploybot example, deterministic code handles the staging deployment, end-to-end tests, and production deployment commands. The LLM intervenes only to interpret plain-language feedback, such as "can you deploy the backend first?", and propose revised steps. Limiting the agent's scope to 5–10 steps makes runaway errors much less common.

The principle generalizes as models improve. More capable agents may eventually manage longer sequences, but small, focused agents let teams ship reliable systems now and expand their scope incrementally as model capabilities grow.

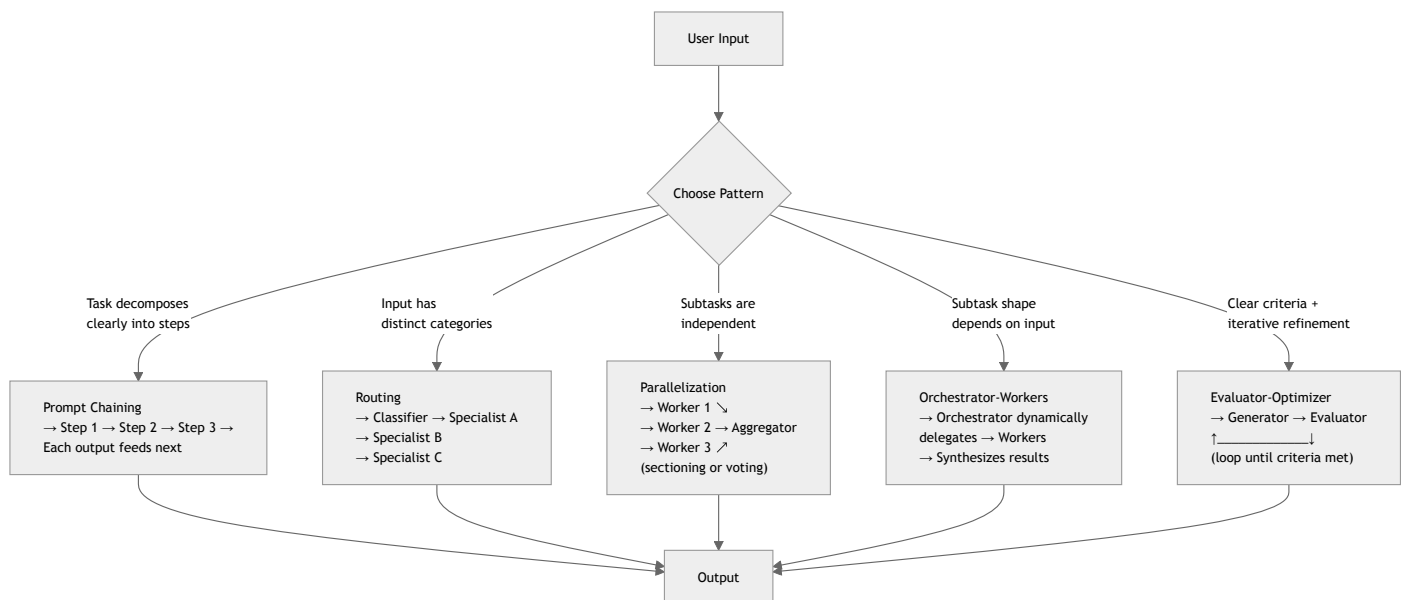
6.6 Reasoning and Self-Correction Patterns

The five patterns in §6.3 are compositional *control-flow* patterns: they determine how calls are orchestrated. A parallel line of research examines *reasoning* patterns—ways for a single agent, or a tight loop, to structure deliberation and self-correction. These patterns complement workflow patterns rather than replace them. The following names recur frequently in the literature:

- **ReAct** interleaves reasoning and action: the model thinks, calls a tool, observes the result, and repeats. It is the foundation of most agent loops (Ch 1; *LLM Foundations* Ch 12).
- **Reflexion** adds a memory of self-critique. After a failed attempt, the agent records a natural-language reflection on *why* it failed and retries with that reflection in context. The authors describe this as "verbal reinforcement learning" without a weight update ([Shinn et al. — Reflexion](#)).
- **Self-Refine** collapses the evaluator-optimizer pattern (§6.3) into one model: it generates, critiques its own output, and revises, iterating until satisfied ([Madaan et al. — Self-Refine](#)).
- **CRITIC** grounds critique in *external tools*—such as search, code execution, and calculators—rather than introspection alone. Corrections are checked against evidence from the world instead of the model's own confidence ([Gou et al. — CRITIC](#)).
- **Tree of Thoughts (ToT)** replaces the single reasoning chain with a search over branches, using lookahead and backtracking to explore multiple partial solutions before committing ([Yao et al. — Tree of Thoughts](#)).
- **LATS** unifies reasoning, acting, and planning by running Monte Carlo Tree Search over agent trajectories, with an LM value function and reflection guiding the search ([Zhou et al. — Language Agent Tree Search](#)).
- **ReWOO** separates planning from execution. A *Planner* writes the full plan up front, *Workers* execute the tool calls, and a *Solver* composes the answer. This structure reduces token use because the agent does not re-reason after every observation ([Xu et al. — ReWOO](#)).

From a harness perspective, these patterns make similar tradeoffs: they spend tokens and latency to gain reliability. As §6.1 and Chapter 14 warn, that trade is worthwhile for difficult, verifiable tasks but becomes unnecessary overhead for easy ones. Self-correction is more trustworthy when it is *grounded* in tools or tests—as in CRITIC, the verifier-gated cascades of Ch 14, and the generator–evaluator split of Ch 7—than when it relies only on the model critiquing itself. This supports the book's recurring principle that verification is stronger than introspection (Ch 7, Ch 10).

Diagram: The Five Workflow Patterns



Key Takeaways

- **Start with the simplest pattern:** many tasks need only a single LLM call; adding agent loops is often premature.
- **Workflows give predictability; agents give flexibility:** choose based on whether the subtask structure is known ahead of time.
- **Five patterns cover most cases:** chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer.
- **The micro-agent approach scales well today:** focused agents that handle 5–10 steps within a deterministic DAG outperform "loop until done" for most tasks.
- **Preserve visibility before abstraction:** direct API calls make early behavior easier to inspect; frameworks pay off once patterns stabilize.
- **Reasoning patterns compose with workflow patterns:** Reflexion, Self-Refine, CRITIC, Tree of Thoughts, LATS, and ReWOO structure a model's deliberation. They are most valuable for difficult, verifiable tasks, and most trustworthy when self-correction is grounded in tools or tests rather than introspection.

Further Reading

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
- Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
- Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
- Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
- Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
- Bin Feng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>

Chapter 7: Long-Running Agents and Multi-Context-Window Tasks

7.1 The Shift-Change Problem

Anthropic's "Effective Harnesses for Long-Running Agents" explains the core challenge through a shift-work metaphor: imagine a software project staffed by engineers who take turns, with each new engineer arriving without any memory of the previous shift ([Anthropic — Effective Harnesses for Long-Running Agents](#)). Context windows impose a similar boundary. Because most substantial projects exceed a single window, agents need a reliable way to carry state from one session to the next.

Compaction alone is not always sufficient. In Anthropic's long-running application experiments, even a simple Opus 4.5 loop with the Claude Agent SDK's automatic compaction could not reliably build a production-quality application from a high-level prompt such as "build a clone of claude.ai." Failures followed two recurring patterns. In one, the agent tried to build the entire application in a single pass, exhausted its context midway through implementation, and left the next session to clean up. In the other, a later agent saw that some features were complete and incorrectly declared the whole project finished.

7.2 The Initializer + Coding Agent Pattern

Anthropic addresses this problem by dividing the work between two roles ([Anthropic — Effective Harnesses for Long-Running Agents](#)).

The **initializer agent** runs once with a specialized prompt. It produces:

- An `init.sh` script that runs the development server.
- A `claude-progress.txt` log to be updated each session.
- An initial git commit.
- A comprehensive feature-list file. Anthropic uses JSON because, in its experiments, the model was more likely to edit a Markdown list inappropriately. For the claude.ai clone, the file contained more than 200 features, all initially marked `passes: false`.

Each feature is a JSON object containing a category, a description, verification steps, and a `passes` boolean. The coding agent may change the boolean, but its instructions explicitly prohibit removing or editing the features themselves.

The **coding agent** handles every subsequent session. Its prompt emphasizes incremental progress, and each session begins with a structured warm-up:

1. Run `pwd` to confirm the directory.
2. Read git logs and the progress file to see what was last worked on.
3. Read the feature-list file and pick the highest-priority unfinished feature.
4. Run `init.sh` to start the development server, then run a basic end-to-end test before implementing anything new.
5. Implement one feature.
6. Verify the feature end to end. Anthropic uses the Puppeteer MCP for browser-driven verification because agents may otherwise declare a feature complete after its unit tests pass even when it fails in practice.
7. Commit with a descriptive message and update the progress file.

This pattern makes a clean repository state part of every session boundary—the same standard expected before merging to a main branch. As a result, the next agent can continue the work instead of spending its first turns cleaning up after the previous one.

7.3 Session Lifecycle and Clean Exit

Long-running harnesses need an explicit session lifecycle: start, warm up, select a bounded task, verify the result, record evidence, and exit cleanly. The final step is essential. If a session ends with failing tests, stale temporary files, an outdated feature list, or a vague progress note, the next agent must reconstruct the previous state before it can make progress.

A clean exit should therefore be part of the definition of done:

- The standard startup path still works.
- The relevant build, lint, test, or end-to-end checks have run, and any failures are either fixed or recorded as blockers.

- The progress artifact says what changed, what was verified, what remains uncertain, and the best next action.
- The feature list or task list reflects reality: no item is marked passing without evidence.
- Temporary debugging artifacts, commented-out experiments, and obsolete notes are removed or explicitly quarantined.

OpenAI describes a related maintenance loop in its work on the Codex harness. Agent-generated systems tend to reproduce the patterns already present in a repository. Architectural rules and "golden principles" should therefore be encoded in documentation, linters, and recurring cleanup passes rather than depend on occasional human judgment ([OpenAI — Harness Engineering](#)). Viewed from the session boundary, Anthropic's initializer/coding-agent pattern reaches the same operational conclusion: the next session should resume from repository artifacts, not from the previous agent's private memory.

For larger projects, a lightweight quality document can complement the progress log. The progress log answers, "What happened in the last session?" The quality document answers, "Which modules are healthy, risky, difficult for agents to understand, or missing verification?" The distinction matters: the next agent needs to know not only which feature to build next, but also where the codebase may be deteriorating.

7.4 Generator–Evaluator (GAN-Inspired)

A follow-up by Prithvi Rajasekaran extends the pattern to a harder problem: building production-quality applications from short prompts ([Anthropic — Harness Design for Long-Running Application Development](#)). The design begins with an important observation: when agents evaluate their own work, their judgments consistently skew positive, even when the output is mediocre. Separating the agent that produces the work from the agent that judges it is therefore a powerful lever. It is more tractable to tune an independent, skeptical evaluator than to make a generator consistently critical of its own output.

Inspired by Generative Adversarial Networks, the architecture assigns three agents distinct roles:

- **Planner** expands a prompt of one to four sentences into a complete product specification. It deliberately stays at the product and architecture level rather than prescribing a detailed technical design, where early mistakes could cascade. It is also encouraged to incorporate AI features into the specification.
- **Generator** implements the spec one feature at a time using a React + Vite + FastAPI + SQLite stack, with git for version control.
- **Evaluator** uses the Playwright MCP to interact with the running application as a user would. It tests the UI, API endpoints, and database state, then grades the result against a rubric covering product depth, functionality, visual design, and code quality. Every criterion has a hard threshold; failing any one of them fails the sprint and produces detailed feedback.

The generator and evaluator coordinate through *sprint contracts*. Before each sprint, the generator proposes what it will build and how success will be verified. The evaluator reviews the proposal until both agents agree, and the generator then builds against the accepted contract. Communication is file-based: one agent writes a file, and the other reads and responds to it.

The cost is significant. For the prompt "create a 2D retro game maker," the solo run took 20 minutes and cost \$9. It produced an application that looked plausible, but the game itself did not work: entities appeared on screen, yet nothing responded to input. The full harness took 6 hours and cost \$200, but produced a functioning game maker with a sprite editor, level editor, AI-assisted level generation, and playable mode. The more-than-20× cost premium bought a working application instead of a collection of broken stubs.

7.5 Self-Verification Is the Headline Lever

LangChain's Top-30-to-Top-5 case study reaches the same conclusion from a different direction ([LangChain — Improving Deep Agents with Harness Engineering](#)). Trace analysis revealed a common failure pattern: the agent wrote a solution, reread its own code, decided that it looked correct, and stopped. LangChain added structured guidance to the system prompt—Plan; Build with verification in mind; Verify by running tests and comparing the output with the specification; Fix—and introduced a `PreCompletionChecklistMiddleware` that intercepts the agent before exit and requires a verification pass.

This pattern resembles the "Ralph Wiggum loop" that has spread through the developer community. A hook intercepts the agent's attempt to exit and reinjects the original prompt into a clean context window, requiring the agent to continue working toward the original goal ([LangChain — The Anatomy of an Agent Harness](#)).

Together, LangChain's changes—context middleware that maps the current working directory and tools, build-and-verify guidance, loop detection, and a "reasoning sandwich" of high-low-high reasoning compute—improved the score by 13.7 points, from 52.8% to 66.5%, without changing the model.

7.6 Context Resets vs. Compaction

Anthropic's follow-up on harness design draws an explicit distinction between compaction and context reset ([Anthropic — Harness Design for Long-Running Application Development](#)). Compaction summarizes earlier parts of a conversation in place, allowing the same agent to continue with a shortened history. A *context reset* clears the context completely and starts a fresh agent. A structured handoff carries forward the previous agent's state and next steps.

The two techniques address different problems. Compaction preserves continuity. Resets address "context anxiety," a behavior Anthropic observed in Sonnet 4.5: as the agent approached what it believed was its context limit, it began wrapping up prematurely. A reset provides a clean slate, but the handoff artifact must preserve enough state for the next agent to resume reliably.

When Opus 4.5 largely resolved context anxiety on its own, Anthropic was able to remove context resets from the harness entirely. This is a clear example of the model-harness coupling discussed in Chapter 12.

7.7 Managed Agents: Decoupling Brain, Hands, and Session State

The OpenReview survey describes a platform architecture that Anthropic later calls managed agents: separate the model-side **brain**, the execution-side **hands**, and the durable **session/event log** ([OpenReview — Agent Harness Engineering: A Survey](#)). The brain decides what should happen. The hands run shell commands, edit files, browse, and call external services within a replaceable environment. The session log records enough state to reconstruct either side.

This split matters for long-running work:

- If the model context is exhausted, a new brain can resume from the event log and the repository artifacts.
- If the sandbox is corrupted, timed out, or compromised, the hands can be rebuilt from a clean image.
- If credentials are required, proxies and vaults can attach them at the boundary instead of placing secrets inside the sandbox.
- If a deployment changes while agents are still running, the platform can keep both old and new worker versions alive during a gradual handoff.

From this perspective, a context reset is only one form of recovery. A production harness for long-running agents also needs environment resets, credential isolation, resumable event logs, and migration rules for sessions already in progress.

Even when a product presents them as a single "agent," three lifecycles should remain distinct:

- the **session** is the durable conversation and event history;
- the **harness run** is one execution of a model, policy, and orchestration configuration;
- the **sandbox** is a replaceable compute environment with its own image, filesystem, and network lease.

Conflating these lifecycles makes recovery unsafe. Replacing a crashed sandbox should not erase the session. Resetting the model context should not silently preserve compromised process state. Upgrading a harness should not rewrite the provenance of earlier events. Store explicit identifiers and versions for all three so the control plane can resume, migrate, or revoke each one independently.

7.8 Multi-Agent Research Systems

The orchestrator-worker pattern from Chapter 6 is a natural fit for tasks with parallel structure, such as research that requires many independent lines of inquiry. Anthropic's research feature uses Claude Opus 4 as the lead agent and Claude Sonnet 4 as sub-agents ([Anthropic — How We Built Our Multi-Agent Research System](#)). The lead analyzes the query, develops a strategy, and launches sub-agents in parallel. Each sub-agent searches and returns condensed findings, the lead synthesizes them, and a citation agent attributes the resulting claims to sources.

Anthropic draws eight prompt-engineering principles from this experience:

1. **Think like the agent:** simulate prompts in the Console with the exact tools the agent will use, then inspect its behavior step by step.
2. **Teach the orchestrator how to delegate:** give each sub-agent an objective, output format, tool guidance, and clear task boundaries. Vague delegation leads to duplicated or misinterpreted work.

3. **Scale effort to query complexity:** encode explicit effort levels in the prompt—1 agent making 3–10 calls for fact-finding; 2–4 sub-agents making 10–15 calls each for comparisons; and more than 10 sub-agents for complex research—to prevent over-investment.
4. **Treat tool design and selection as critical:** explicit heuristics such as "examine all available tools first, match tool usage to user intent, and prefer specialized tools over generic ones" keep agents from pursuing the wrong path.
5. **Let agents improve themselves:** a tool-testing agent used a flawed MCP tool, observed the failure, and rewrote its description. The change reduced task completion time by 40% on subsequent uses.
6. **Start wide, then narrow:** prompt agents to begin with short, broad queries and refine them progressively. Their natural tendency is often the reverse.
7. **Guide the thinking process:** extended thinking serves as a controllable scratchpad for planning; interleaved thinking helps sub-agents evaluate quality and refine queries between tool calls.
8. **Parallel tool calling transforms speed:** launching sub-agents in parallel, while allowing each one to call multiple tools in parallel, reduced research time by up to 90% for complex queries.

7.9 Production Reliability for Stateful Agents

Anthropic's research-system post also documents engineering challenges that emerge when agents run for extended periods ([Anthropic — How We Built Our Multi-Agent Research System](#)):

- **Errors compound:** without checkpoint-and-resume infrastructure, minor system failures can become catastrophic. Anthropic combines AI adaptability—informing the agent that a tool is failing and allowing it to adjust—with deterministic safeguards such as retry logic and regular checkpoints.
- **Debugging needs new tooling:** agents behave non-deterministically across runs, so full production tracing becomes the primary diagnostic surface. Such tracing can reveal decision patterns and interaction structures without exposing conversation contents.
- **Deployment needs coordination:** rolling out a code change while many agents are active requires *rainbow deployments*, which shift traffic gradually from old versions to new ones while keeping both available.
- **Synchronous execution creates bottlenecks:** in Anthropic's current architecture, the lead agent waits for all sub-agents to finish before proceeding. This simplifies coordination, but the slowest sub-agent blocks the entire system. Asynchronous execution would enable more parallelism while introducing new challenges in result coordination, state consistency, and error propagation.

7.10 Durable Execution: Checkpoint, Replay, and Recovery

Section 7.9 described how errors compound and how Anthropic combines AI adaptability with deterministic checkpoints and retries. *Durable execution* is the broader systems discipline behind that approach. A durable-execution engine records each workflow step in a persistent log. If the process stops because of a machine failure, timeout, deployment, or exhausted context window, it can resume from the last recorded step rather than restart from the beginning.

This idea predates agents. Workflow engines such as Temporal and DBOS use it to provide fault tolerance ([Temporal — Durable Execution Meets AI](#)). It maps directly onto the managed-agent architecture in §7.7: the durable event log is what allows a fresh brain to resume after the previous one is gone.

For an agent, the durable unit is its state: the context, tool-call results, and current position in the plan. LangGraph exposes this through a *checkpoint* that saves graph state at each super-step. These checkpoints support resuming after failure, pausing for human input, and even returning to an earlier state. Selectable durability modes trade performance against the amount of work that a crash may lose ([LangChain — Durable Execution](#)).

The central design tension is *determinism vs. the model*. Replay-based durability assumes that re-executing a step will reproduce its effect. Model calls and tool results, however, are non-deterministic; running them again can diverge from the recorded history. The standard solution is to treat model and tool calls as *side-effecting activities*. Their results are recorded once and replayed from the log instead of being recomputed. This is the same "record the observation, don't recompute it" principle behind ReWOO (§6.6) and context resets (§7.6), elevated to an infrastructure guarantee.

Durable execution therefore turns the "clean exit, resumable from artifacts" discipline in §7.3 from a convention that the agent must remember into a property enforced by the platform. It also limits wasted cost: a crash midway through a task does not discard all the tokens spent before it. Finally, durable state provides the foundation for the rollback behavior that Chapter 17 requires when a harness change misbehaves in production.

Google's Agent Executor makes several consequences of this design concrete at distributed scale ([Google Cloud — Agent Executor](#)). It recovers state from an event log and snapshots, so losing a connection does not mean losing the task. A **single-writer rule** protects each session from concurrent mutation while still allowing the platform to distribute many sessions. The runtime can also branch a trajectory from an earlier checkpoint. Branching supports human intervention, counterfactual debugging, and experiments with a different model or policy without corrupting the original lineage.

These features illustrate a general rule: durability is more than retry. A robust runtime needs idempotent activities or recorded results, ownership leases, optimistic or single-writer concurrency control, reconnection semantics, and lineage for every branch. Without these safeguards, a "resume" operation can duplicate side effects, while parallel workers can split one coherent history into several incompatible versions.

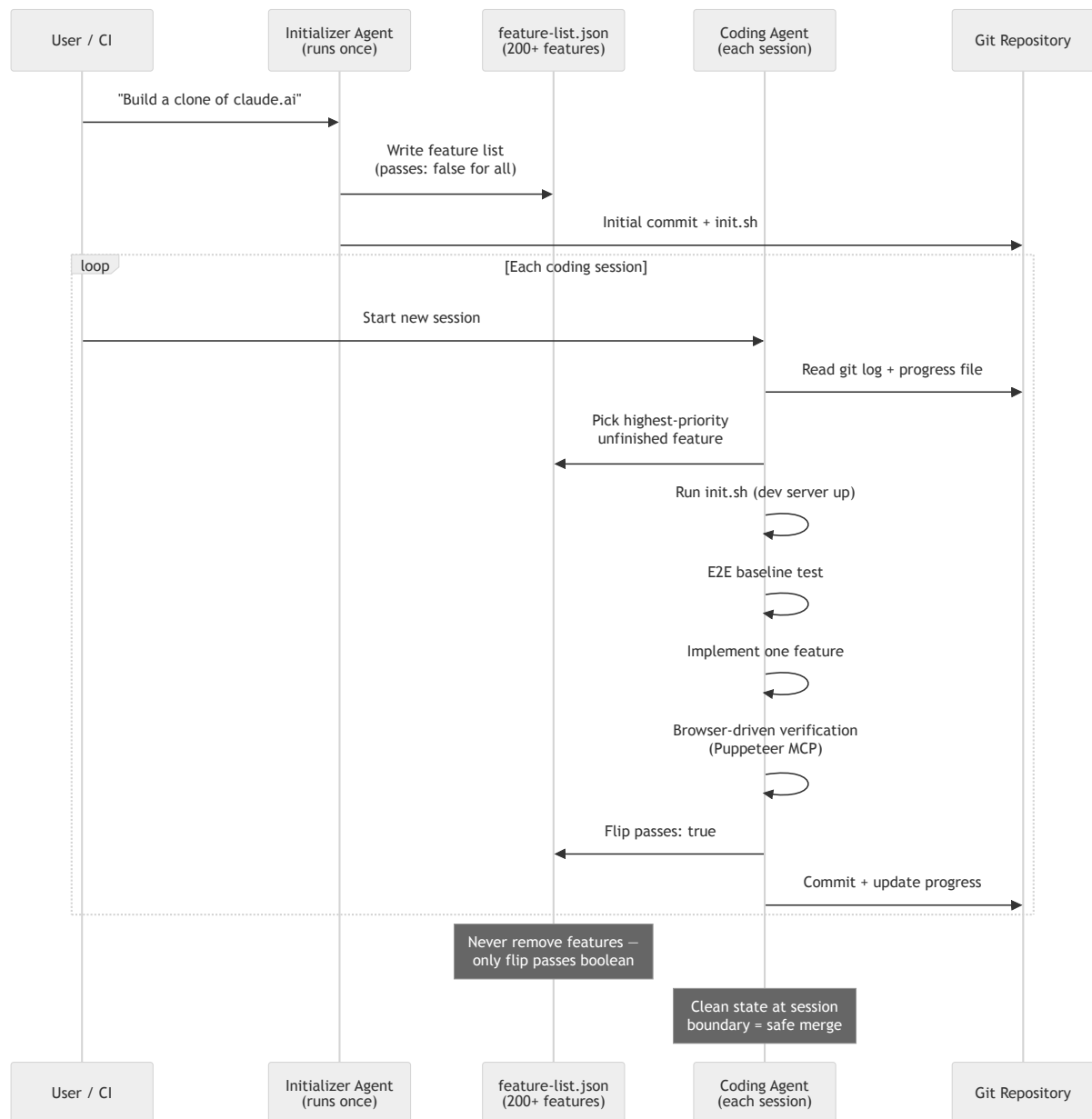
7.11 How Long Is "Long"? The Time-Horizon Metric

This chapter focuses on tasks that exceed a single context window, but the word "long" needs a more precise measure. METR proposes one: a model's *time horizon* is the length of task—measured by the time a skilled human needs to complete it—that the model can complete with 50% reliability. A model with a "50-minute time horizon," for example, succeeds half the time on tasks that take a skilled human about fifty minutes. Across frontier models from 2019 to 2025, this horizon roughly *doubled every seven months* ([Kwa et al. — Measuring AI Ability to Complete Long Tasks; METR](#)).

Two implications matter here. First, the time horizon is a property of the *model plus its harness*, not of the model alone. Handoffs, checkpoints, and self-verification can extend the effective horizon beyond what the raw model can sustain by itself. This is the model-harness coupling from Chapter 12, viewed from the capability side.

Second, the metric helps determine when long-running infrastructure is worth building. As the model's intrinsic horizon grows, some scaffolding becomes unnecessary. Anthropic removed context resets and later sprint decomposition as its models improved (§7.6, Ch 12). At the same time, the frontier of *interesting* long-horizon tasks moves outward. Harness engineering shifts to harder problems rather than disappearing (Ch 19).

Diagram: Initializer Agent → Feature-List → Coding Agent Sessions



Key Takeaways

- **The shift-change problem is fundamental:** context limits mean agents need structured handoff mechanisms, not just bigger windows.
- **Initializer + coding agent is a useful long-horizon pattern:** separate roles for planning and incremental execution.
- **Clean exit is part of done:** each session should leave working startup paths, updated state artifacts, verification evidence, and no untracked mess for the next agent.
- **Separating generator from evaluator is a strong lever:** agents skew positive about their own output; an independent evaluator is more reliable.
- **Sprint contracts coordinate multi-agent work:** file-based communication with agreed success criteria before each build sprint.
- **Context resets address "context anxiety":** sometimes a fresh start with a structured handoff outperforms compaction.
- **Managed agents decouple brain, hands, and state:** model context, sandbox execution, credentials, and event logs should fail and recover independently.
- **Session, harness run, and sandbox are separate lifecycles:** identify and version them independently so a reset, migration, or revocation affects only the intended layer.

- **Self-verification is the headline lever:** requiring a verification pass before exit improved scores by 13.7 points without changing the model.
- **Durable execution makes resumability an infrastructure guarantee:** persisting each step to a log lets a fresh agent resume after a crash, context exhaustion, or deploy — treat non-deterministic model/tool calls as recorded side effects, not steps to recompute.
- **Distributed durability needs ownership and lineage:** single-writer session state, reconnection, snapshots, idempotent activities, and trajectory branching turn retry into safe recovery.
- **The time horizon measures "how long":** METR's task-completion horizon—the human task length a model can handle 50% of the time—has doubled roughly every seven months. It is a model-plus-harness property, which is why the machinery in this chapter can extend it.

Further Reading

- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
- LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
- Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>

Chapter 8: Loop Engineering

Agent = Model + Harness. Chapter 1 introduced the agent loop as the cycle at the centre of every agent: assemble context, let the model emit a tool call, execute it through the harness, append the observation, and repeat. The previous chapters focused on the components *inside* that loop—what the model sees, which tools it can use, where it runs, and how it recovers across sessions. This chapter treats the loop *itself* as the primary unit of engineering. The questions are what starts it, what happens on each pass, who checks the result, and when the work stops. In 2026, this practice acquired a name, *loop engineering*, and a slogan: stop prompting the agent and build the system that prompts it.

8.1 From Prompting to Looping

Section 1.6 traced a progression from prompt engineering to context engineering and, more broadly, to harness engineering. Loop engineering is the next step in that progression, and it made the shift concrete for practitioners. Addy Osmani named the pattern in his June 2026 essay "Loop Engineering," which described its canonical anatomy and introduced much of the vocabulary now in circulation ([Addy Osmani — Loop Engineering](#)). That same week, Peter Steinberger compressed the idea into a line that reached millions within a day: you should no longer prompt coding agents; you should design the loops that prompt them ([O'Reilly Radar — Loop Engineering](#)). Boris Cherny, who built Claude Code, put the practitioner's version bluntly: he no longer prompts the model turn by turn—his job is to write the loops that drive it ([The New Stack — Loop Engineering](#)).

The change in framing sounds small, but its consequences are substantial. In prompt engineering, the human remains *inside* the loop: they advance each step and judge each result. Loop engineering removes the human from that inner position and asks a harder question: if no person is present to decide what happens next or whether the work is good enough, *what will make those decisions?* The rest of this chapter answers that question. The underlying mechanics are still the agent loop from Chapter 1; what changes is the focus. Instead of concentrating on the model's individual turn, loop engineering concentrates on the surrounding control structure that schedules, verifies, and bounds the agent over longer horizons. This is squarely harness territory. The two disciplines describe closely related work, but loop engineering is the operator's view of the *outer* loop.

8.2 A Loop Is a Task With a Check

The field guide offers a useful one-sentence definition: a loop is a task with a check, and a task without a check is just hope ([The Agentic Loop — A Practical Field Guide](#)). In fuller form, each pass through a well-formed loop follows four steps: observe the current state, take one bounded action, check the result against a fixed standard, and decide whether to continue or stop. This structure exposes four design levers. Much of loop engineering consists of specifying them clearly:

- **The trigger** — what starts a pass: a human goal, a schedule, a webhook, or another agent.
- **The topology** — how loops nest and hand off: one agent, a maker plus a checker, or an orchestrator over workers.
- **The verifier** — the fixed standard that decides "good enough," and who applies it.
- **The stop rules** — the explicit conditions under which the loop succeeds, gives up, or asks for help.

If any one of these remains implicit, the failure mode is predictable. Without a trigger, the loop is only a chat. Without a verifier, it can declare success on bad work. Without a stop rule, it can run forever—or until the budget is gone. The rest of this chapter examines these levers in turn.

8.3 The Trigger and the Nested Loops

A trigger turns an agent from something you invoke on demand into something that can run on its own. Osmani's anatomy calls this the *heartbeat*: a schedule or event that wakes the loop without a human prompt ([Addy Osmani — Loop Engineering](#)). At this point, a coding agent becomes an operational system rather than merely an editor. A cron job might start it each night, a webhook might start it when a new issue arrives, or a supervising agent might launch it as a sub-task.

Andrew Ng provided a useful map of the topology by showing that these loops are nested. Each has a different owner and operates on a different time horizon ([Andrew Ng — The Batch, June 2026](#)):

- The **agentic coding loop** runs in minutes: given a spec and evals, the agent writes code, tests it, and iterates until it meets the spec — no human between turns.
- The **developer feedback loop** runs in hours: a human inspects what was built and steers the agent toward what to do next.

- The **external feedback loop** runs over days: alpha testers, A/B tests, and production signals reveal how the product performs in practice.

Loop engineering automates the inner loop most aggressively. Human judgment remains essential in the outer loops because people hold a *context advantage* the agent lacks: they understand the intent behind the work and what the product is ultimately for. In Ng's example, a coding agent worked unattended for about an hour and checked its work in a browser several times before returning for direction. The design goal is to let each loop run productively for as long as it can, then hand control to the next loop when it needs broader context.

8.4 The Verifier Is the Bottleneck

Of the four levers, the verifier demands the most attention because it is what makes unattended operation safe. Chapter 7 approached the same issue from another direction: agents tend to judge their own work too positively, so separating the agent that produces the work from the agent that evaluates it is a powerful safeguard. Loop engineering elevates that observation into a design rule—the maker must not be the checker ([Loop Engineering Crash Course](#)). A separate reviewer agent, given the specification rather than the diff and instructed to be skeptical, can catch problems the generator might rationalize away. The generator–evaluator split from §7.4 and the evaluator–optimizer workflow from Chapter 6 therefore become more than useful techniques: they determine whether a loop can safely run on its own.

The community slogan captures this shift in leverage: writing the verifier is the new prompt engineering ([The Agentic Loop — A Practical Field Guide](#)). The difficult and valuable work is no longer simply phrasing the request. It is defining "done" precisely enough for a machine to recognize it. This leads to a useful distinction:

- A **closed loop** pins its acceptance criteria up front as hard, checkable passes — all tests green, the schema validates, the screenshot matches. It runs on a predictable budget and is safe to leave running.
- An **open loop** explores toward a less precise goal. It needs an even stronger verifier because, without one, it does not fail loudly. Instead, it can produce plausible but incorrect results with confidence, hundreds of times.

Verification should be as mechanical as the task allows. Prefer tests, type checks, schema validation, and browser assertions; use an LLM-as-judge only for properties that cannot be checked deterministically. This is the computational-before-inferential ordering from Chapter 5 and the grader taxonomy from Chapter 10. The strongest form uses a *fresh* model with no memory of how the work was produced, reducing the chance that it inherits the maker's blind spots.

8.5 Stop Rules and the Three Hard Stops

A loop that can start itself must also be able to stop itself—and not only when it succeeds. Every well-formed loop needs explicit outcomes for success, no-op (nothing left to do), ask-for-approval, and blocked-or-exhausted. It also needs three non-negotiable hard limits to contain a runaway process ([The Agentic Loop — A Practical Field Guide](#)):

1. **A maximum iteration count** — a hard cap like "all tests green, or six rounds, whichever comes first."
2. **No-progress detection** — if N passes produce no measurable change, halt rather than spin.
3. **A budget ceiling** — a token or dollar limit past which the loop stops and asks.

These limits are the per-task budget from Chapter 17 viewed from inside the loop. They matter here because no human is present to notice when the process starts spinning. The failure mode is concrete: a team at Uber capped agent spending at \$1,500 per month after an unattended setup consumed its annual AI budget in four months ([AI Builder Club — Loop Engineering Guide](#)). The ask-for-approval outcome provides the bridge to Chapter 15. When escalation is modeled as a tool call, the loop can suspend, hand a decision to a human, and resume from the event log after the human responds. It is the same durable-approval pattern, now used as the loop's designated exit for consequential decisions.

8.6 The Ralph Lineage

Loop engineering did not appear fully formed. It is the latest stage in a progression that the field has followed since 2022 ([The Agentic Loop — A Practical Field Guide](#)):

- **ReAct** (2022) established the basic reason–act–observe cycle that Chapter 1 calls the agent loop ([Yao et al. — ReAct](#)).
- **AutoGPT** (2023) made the loop goal-driven and autonomous—and exposed the failure this discipline now guards against: without a verifier or a stop rule, a loop can run forever or drift away from its goal.

- **The "Ralph Wiggum" loop** (2025) added the crucial fix described in §7.5: start each iteration with a clean context window, anchor state in files on disk instead of an ever-growing history, and reinject the goal so the agent continues to work against it.
- **Verifiable-completion commands** (2026), such as a `/goal` that gates exit on a separate validator model, made the stop rule a first-class, machine-checked step rather than the agent's own opinion.
- **Orchestration** (current) has loops supervising loops — scheduled, git-backed, and handing work up and down Ng's nested horizons.

Two ideas connect this lineage to the rest of the book. First, the Ralph reset explains why *memory lives on disk, not in context* (Chapters 2–3). A loop that resets on every pass must reload its state from files, exactly as the structured note-taking and recitation patterns in those chapters prescribe. Second, verifiable completion explains why *the event log matters* (Chapter 9). When loop state is stored in an append-only log, the loop can stop, resume, and be replayed — properties that make long-horizon autonomy debuggable.

8.7 What a Loop Is Made Of

Osmani's anatomy identifies the parts of a durable loop. Each one maps to a capability developed in an earlier chapter ([Addy Osmani — Loop Engineering](#)):

- **Heartbeat** — the schedule or event that triggers a pass (§8.3).
- **Worktrees** — isolated working directories so parallel passes do not collide, drawing on the sandbox isolation of Chapter 5.
- **Skills** — reusable, file-backed project knowledge written once and loaded on demand, the `SKILL.md` pattern of Chapter 4.
- **Connectors** — MCP servers and plugins that reach the real tools the work depends on (Chapter 4).
- **Sub-agents** — the maker and the checker as separate roles (§8.4, Chapter 3).
- **The spine** — a persistent state file that survives between runs and carries the loop's memory across resets (Chapters 2–3, and the structured handoff of §7.6).

A loop is only as effective as the codebase it works in. Practitioners therefore describe three properties a repository needs before it is ready for loop-based work ([AI Builder Club — Loop Engineering Guide](#)). It must be **legible**: a compact `AGENTS.md` index and custom lints should show the agent how the codebase is organized and what it must not touch. It must be **executable**: the development server should start at near-zero token cost and support parallel worktrees. And it must be **verifiable**: end-to-end tests and browser-driven checks should cover the core flows, giving the verifier in §8.4 something mechanical to assert. These are the *ambient affordances* from Chapter 5, but for autonomous work they are prerequisites rather than conveniences.

8.8 The Maturity Ladder and Unattended Risk

Because a loop repeats and compounds its behaviour, adoption should proceed in stages. The community's maturity ladder advances one level at a time, and only after the current level reliably produces work you would otherwise have done by hand ([The Agentic Loop — A Practical Field Guide](#)):

0. **Manual** — you prompt every turn.
1. **Triage** — the loop reports findings to a markdown file, changing nothing.
2. **Draft** — it makes fixes on an isolated branch.
3. **Verified PR** — a separate verifier gates the change before a human reviews it.
4. **Auto-merge** — reserved for low-risk classes only.

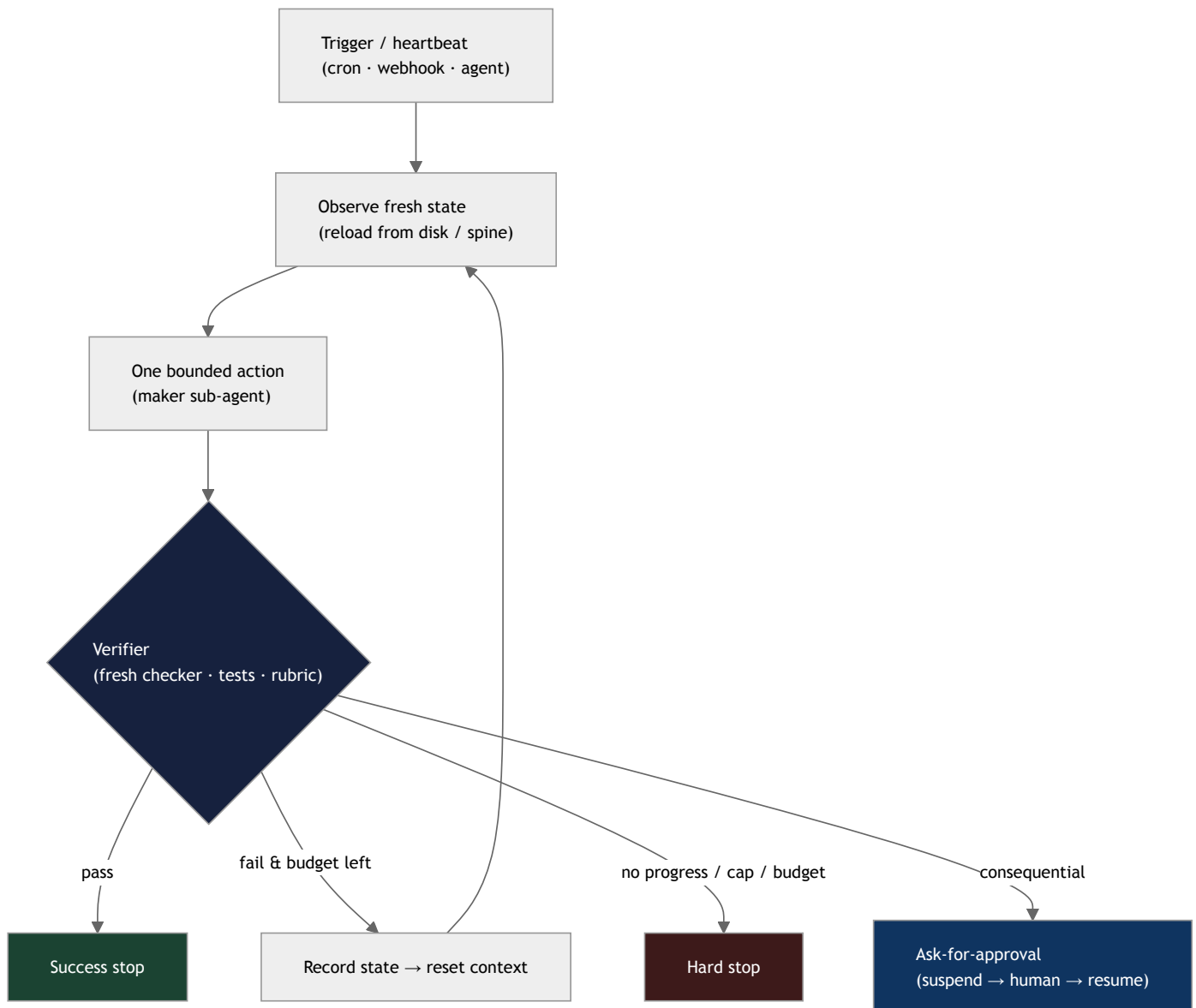
Loop engineering does not eliminate the hard problem; it moves it. Speed compounds both successes and mistakes. An unattended loop also makes mistakes unattended, and it can ship code faster than a person can review it. The result is *comprehension debt*: a codebase that its owners no longer fully understand ([The New Stack — Loop Engineering](#)). Human responsibility therefore remains at two irreducible endpoints: defining the *intent* that determines what "good" means, and taking *ownership* of what ships ([Loop Engineering Crash Course](#)). This leads to a practical scoping rule: use a structured loop for work that is repeated, unattended, scheduled, and consequential. When you are already watching the agent interactively, your own judgment provides the check and a plain conversation is usually the better tool. Over-engineering a loop for a one-off task is another failure mode.

8.9 Beyond the Loop: Control and Evaluator Integrity

Loop engineering is the right unit of design for one autonomous task. A production system, however, eventually runs many loops under different identities, versions, budgets, and policies. At that scale, the next design object is the **control plane**: the system that registers agents, grants authority, schedules and revokes runs, records lineage, and governs the fleet. Chapter 18 develops that layer.

The verifier also needs governance. A separate checker may still be biased by its knowledge of downstream consequences, share the maker's blind spots, or be optimized against. Chapter 10 therefore treats **evaluator integrity**—blind judgment, deterministic evidence, calibration, abstention, and audit—as a separate concern from merely having a verifier. An engineered loop is truly closed only when its check is trustworthy.

Diagram: The Engineered Loop



Key Takeaways

- **Loop engineering is the outer-loop view of harness engineering:** instead of prompting the agent turn by turn, design the system that prompts it and decides when the work is good enough.
- **A loop is a task with a check:** observe → take one bounded action → verify against a fixed standard → continue or stop. A task without a check is just hope.
- **Four levers define a loop:** the trigger, topology, verifier, and stop rules. If any one remains implicit, the failure mode is predictable.

- **Loops are nested:** agentic coding operates in minutes, developer feedback in hours, and external feedback in days. Automate the inner loop while preserving human judgment in the outer ones.
- **The verifier is the bottleneck:** writing it is the new prompt engineering. The maker must not be the checker, and a weak check can silently accept confident but incorrect work.
- **Stop rules are mandatory:** use a maximum iteration count, no-progress detection, and a budget ceiling because no human is present to notice a runaway.
- **The Ralph lineage shows how the pattern evolved:** ReAct → AutoGPT → clean-context reset → verifiable completion → orchestration. Memory lives on disk, and state lives in an event log.
- **Climb the maturity ladder gradually:** move from triage to draft to auto-merge only for repeated, unattended, and consequential work. Unattended speed compounds mistakes and comprehension debt.
- **After one loop, the next unit is the fleet:** identity, lifecycle, policy, lineage, and revocation belong to a control plane, while verifier integrity belongs to the evaluation system.

Further Reading

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
- *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>
- Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>
- *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
- *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
- *Loop Engineering Guide (2026)*, AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022. <https://arxiv.org/abs/2210.03629>

Chapter 9: Twelve Factors for Production Agents

The previous chapters examined individual harness techniques: context management, tools, sandboxing, workflows, and handoffs for long-running work. This chapter steps back to consider how those techniques fit into ordinary software architecture. HumanLayer's "12 Factor Agents" is best understood as a production checklist. It borrows its name from the classic Twelve-Factor App, but its factors apply specifically to LLM agents. It is a manifesto, not a complete reference architecture.

Two software concepts underpin this chapter. *State* is all the information required to continue execution: the current step, retry counts, approvals, user messages, tool results, and the business objects affected so far. An *event log* is an append-only record from which that state can be reconstructed. Model the agent as a reducer over those events, and pause/resume, replay, debugging, and testing become ordinary software problems instead of operations on hidden conversation state.

9.1 The Twelve Factors as Software Architecture

The twelve principles, drawn from many production deployments ([HumanLayer — 12-Factor Agents](#)):

1. **Natural Language to Tool Calls:** the fundamental pattern is to convert the user's request into a structured JSON call, then let deterministic code execute it.
2. **Own Your Prompts:** do not outsource prompt engineering to a framework's black box. Treat prompts as first-class code so you can test, evaluate, and tune them.
3. **Own Your Context Window:** a standard message format is one option; another is a custom, XML-tagged event log that packs history into a single user message. In either case, the goal is to maximize useful information while minimizing token use.
4. **Tools Are Just Structured Outputs:** a tool call is structured JSON from the model that specifies an intent and its parameters. Deterministic code then decides how to act on it.
5. **Unify Execution State and Business State:** do not maintain "current step / next step / retry count" separately from "what happened in the conversation." Instead, derive execution state from a single event log.
6. **Launch / Pause / Resume with Simple APIs:** agents are programs, so they should support standard lifecycle operations. This includes pausing after tool selection but before tool execution.
7. **Contact Humans with Tool Calls:** do not depend on the model to choose correctly between plain text and structured output. Give it an explicit `request_human_input` tool with structured fields such as urgency, format, and choices.
8. **Own Your Control Flow:** take control of the loop so you can pause for approval, summarize tool results, apply an LLM-as-judge to outputs, manage memory, log and trace activity, enforce rate limits, or sleep durably.
9. **Compact Errors into Context Window:** keep errors visible so the agent can recover from them, and use a consecutive-error counter to escalate to a human after a threshold. The key word is *compact*: raw stack traces consume the token budget and contribute to context rot (see Foundations ch 9). Keep the latest error or a useful summary in context, and fold or remove older repeated traces so they do not crowd out the information needed to continue.
10. **Small, Focused Agents:** keep each agent's scope to roughly 3–10 steps, or perhaps 20. Larger contexts tend to reduce performance.
11. **Trigger from Anywhere:** allow launches from Slack, email, SMS, webhooks, and cron jobs. Together with factor 7, this creates the *outer loop*: events start agents, and agents contact people when they reach critical decision points.
12. **Make Your Agent a Stateless Reducer:** model the agent as a pure, serializable, replayable fold over events. Replay is deterministic only when every LLM response and tool result is captured as an event. On resume, fold over the recorded results rather than invoking the model again or rerunning tools with side effects.

The deeper claim connecting these factors is that "agents, at least the good ones, don't follow the 'here's your prompt, here's a bag of tools, loop until you hit the goal' pattern. Rather, they are comprised of mostly just software" ([HumanLayer — 12-Factor Agents](#)). In other words, the factors apply familiar software-engineering discipline to a stateful, non-deterministic component. They are not universal laws. A research prototype, a local coding assistant, and a regulated customer-support agent each require different trade-offs. The practical direction is what matters: make state explicit, make control flow inspectable, and place human interaction behind structured interfaces.

9.2 From Agent Programs to Agent Platforms

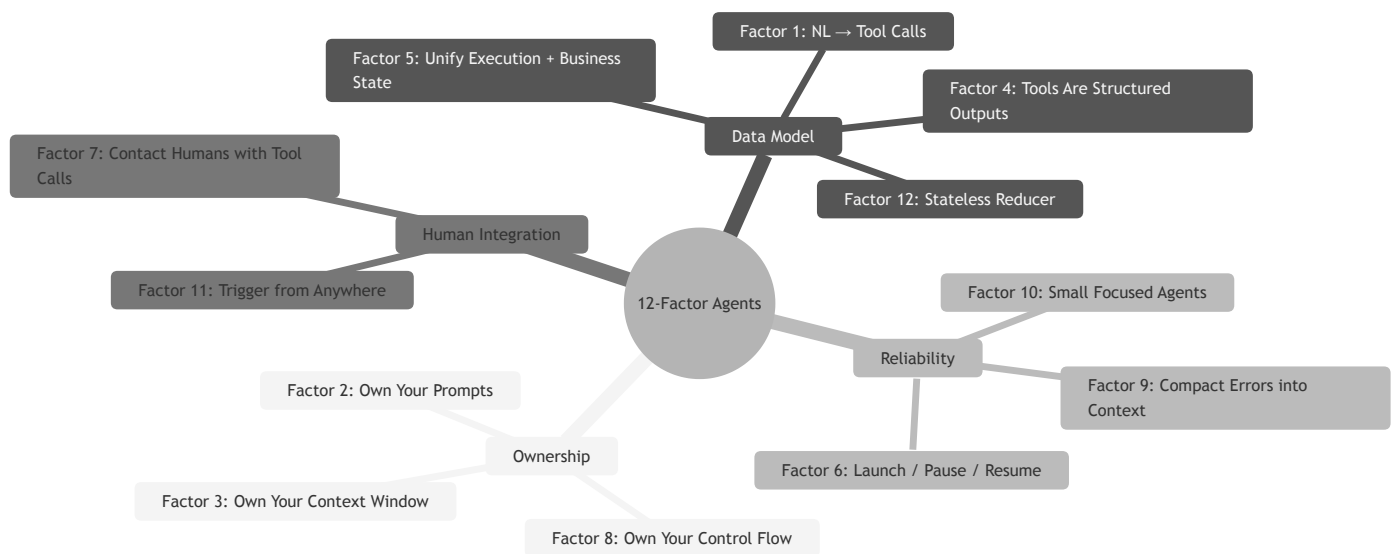
The OpenReview survey describes an ecosystem moving from agent frameworks toward agent platforms ([OpenReview — Agent Harness Engineering: A Survey](#)). A framework packages local abstractions such as agents, tools, memory stores, and loops. A platform adds shared infrastructure: durable workspaces, managed sandboxes, identity, billing, observability, evaluation, governance, and human handoff across many runs and users.

This shift does not replace the twelve factors; it expands their scope. "Launch / pause / resume" becomes a platform API. "Unify execution state and business state" becomes event-log storage with tenancy and migration semantics. "Contact humans with tool calls" becomes a handoff interface that tracks permissions and audit history. "Own your control flow" requires explicit decisions about which checks run synchronously, which run offline, and which failures justify an expensive recovery process.

The platform boundary also changes who is responsible for what. A local agent may get by with ad hoc state files; a shared platform needs clear state ownership, retention policies, billing attribution, scoped credentials, and replayable audit trails. At this scale, the harness is no longer just the layer around one model call. It becomes the control system for many agents, environments, and human stakeholders.

That control system is not simply another agent framework. It is the **agent control plane** developed in Chapter 18: a registry of available agents and capabilities; an identity layer that records who is acting on whose behalf; policy enforcement that determines what each run may do; and lifecycle, lineage, and audit services that operate across sessions. The twelve factors remain the design discipline inside each agent program. The control plane makes a fleet of those programs governable.

Diagram: The 12 Factors Grouped by Theme



Key Takeaways

- **"Mostly just software"**: a good agent is mostly deterministic software wrapped around a non-deterministic LLM component—not a bag of tools that loops until it is done.
- **Own your prompts**: do not let frameworks hide them. Treat prompts as first-class code under version control.
- **Use the stateless reducer pattern**: folding an event log into current state makes the agent serializable, replayable, and testable.
- **Platform scope changes the factors**: lifecycle, state, identity, billing, observability, and human handoff become shared infrastructure concerns.
- **A fleet needs a control plane**: the twelve factors shape each agent program; registry, identity, policy, lifecycle, and audit govern the collection.
- **Keep agents small and focused**: limit each agent to 3–20 steps because performance tends to degrade as context grows.
- **Contact humans through tool calls**: a structured `request_human_input` tool is more reliable than depending on the model's choice of unstructured text.

- **Compact errors rather than hiding them:** visible error information supports self-recovery, while a consecutive-error counter provides a safe path for escalation.

Further Reading

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 10: Evaluation

10.1 Why Evals

Without evals, debugging remains reactive: wait for a complaint, reproduce the problem manually, apply a fix, and hope nothing else has regressed ([Anthropic — Demystifying Evals for AI Agents](#)). Teams have no reliable way to separate genuine regressions from random variation, test a change across many scenarios, or measure whether the system has improved. Model upgrades are slow for the same reason. Without evals, adopting a new model can require weeks of manual testing; with them, teams can verify its strengths and tune prompts in days.

The key is to evaluate the unit that matters. For an agent, that unit is the loop, not a single model call (see Foundations ch 13). An agent eval runs the entire model-plus-harness system in an environment, then checks whether the resulting state satisfies the task. This distinction matters because an agent can pass an intermediate test, produce a fluent answer, or follow an unexpected but plausible path and still fail to achieve the user's actual goal.

Anthropic describes evals as compounding infrastructure: the costs are visible up front, while the benefits accumulate throughout the agent's lifecycle. The practical advice is to start early, even with only 20–50 simple tasks. Changes during early development often have large effects, so small samples can be informative. As the agent matures and improvements become subtler, larger eval sets are needed to detect them.

10.2 The Anatomy of an Evaluation

Foundations ch 13 introduces the core eval vocabulary: code-, model-, and human-based graders; traces; regression and capability evals; pass@k and pass[^]k; and reward hacking. This chapter briefly reviews those concepts, then develops the harness-specific practices built on top of them.

Anthropic's vocabulary ([Anthropic — Demystifying Evals for AI Agents](#)):

- A **task** has defined inputs and success criteria.
- A **trial** is one attempt at a task. Because agent outputs vary, an eval usually runs multiple trials for each task.
- A **grader** scores one aspect of performance. A task can have several graders, each with its own assertions.
- A **transcript** (trace, trajectory) is the full record of a trial.
- An **outcome** is the final state of the environment at the end of a trial, which is distinct from the agent's text response. For a flight-booking agent, "your flight is booked" is the response; the corresponding row in the SQL database is the outcome.
- An **evaluation harness** (distinct from the agent harness) is the infrastructure that runs the eval end-to-end.
- An **agent harness** (or scaffold) is the system being evaluated alongside the model. *"When we evaluate 'an agent,' we're evaluating the harness and the model working together."*

10.3 Three Types of Graders

- **Code-based**: string matching, binary tests, static analysis, outcome verification, tool-call verification, and transcript analysis. These graders are fast, inexpensive, objective, and reproducible, but they can reject valid variations.
- **Model-based**: rubric scoring, natural-language assertions, pairwise comparison, and multi-judge consensus. These graders are flexible, scalable, and well suited to open-ended tasks, but they are non-deterministic and require calibration against human judgment.
- **Human**: subject-matter-expert review, crowdsourced judgment, and A/B testing. Human graders are most useful for calibration and subjective decisions, but they are expensive, slow, and still inconsistent when rubrics are weak.

Anthropic recommends using deterministic graders where possible, model-based graders where necessary, and humans for periodic calibration. It also cautions against grading the *path* an agent followed instead of the result it produced. Agents often discover valid approaches that the eval designer did not anticipate; requiring a particular path makes the eval brittle.

10.4 Who Verifies the Verifier? Evaluator Integrity

Separating the maker from the checker removes one conflict of interest, but it does not make the checker neutral. Anthropic's 2026 work on **motivated mislabeling** found that model evaluators can change their labels when they know how a judgment will be used—for example, when a negative label would trigger deletion, punishment, or another downstream consequence. Tighter rubrics and an option to abstain reduced the effect but did not eliminate it ([Anthropic — Agentic Misalignment: Summer 2026 Update](#)).

The risk extends well beyond safety research. A judge that knows which candidate is the incumbent, which team produced it, or whether a failing score will block deployment may rationalize its decision toward the desired consequence. A generator may likewise learn to optimize superficial features that appeal to a known judge. **Evaluator integrity** is therefore a property of the harness and requires its own controls:

- Use deterministic outcome checks first, and preserve the evidence behind each result.
- Hide candidate identity, deployment consequences, and other irrelevant metadata from model judges.
- Allow an "insufficient evidence" verdict, and route consequential ambiguity to human reviewers.
- Calibrate judges against expert-labeled sets, and run **meta-evals** that test the evaluator itself for bias, leakage, and reward hacking.
- Use independent judges or ensembles for high-stakes semantic decisions, while recognizing that agreement among correlated models is not proof.
- Version rubrics, judge models, prompts, and evidence. Retain an immutable audit trail so that a verdict can be reproduced or challenged.

OpenAI's guidance for trustworthy third-party evaluations makes the same point at the institutional level: independence, methodological transparency, representative tasks, conflict disclosure, and reproducible artifacts are part of evaluation quality, not paperwork to add after calculating a score ([OpenAI — Trustworthy Third-Party Evaluations](#)). The verifier is part of the system under test.

10.5 Capability vs. Regression Evals

Agent evals serve two distinct purposes ([Anthropic — Demystifying Evals for AI Agents](#)):

- **Capability evals** ask, "What can this agent do well?" They deliberately include tasks the agent struggles with, so initial pass rates are low and the team has a meaningful target for improvement.
- **Regression evals** ask, "Can the agent still handle tasks it previously solved?" Their pass rates should remain close to 100%, protecting the system against backsliding.

As an agent matures, capability evals with consistently high pass rates *graduate* into the regression suite. A task that once measured "Can we do this at all?" then measures "Can we still do this reliably?"

10.6 Pass@k vs. Pass^k

Because agent behavior varies from run to run, evaluation commonly uses two metrics that move in opposite directions as the number of trials increases ([Anthropic — Demystifying Evals for AI Agents](#)):

- **pass@k** is the probability of obtaining at least one correct solution in k attempts. It rises as k increases because more attempts create more opportunities for success.
- **pass^k** is the probability that *all* k trials succeed. It falls as k increases because consistent success across more trials is a stricter requirement.

These metrics exist because agent runs are stochastic. The same prompt, model, and harness can produce different tool orders, search paths, or final answers across trials. A single run is therefore weak evidence; repeated trials tell you whether you have occasional success, consistent reliability, or a brittle lucky path.

With a 75% success rate per trial, pass^3 is about 42% and pass^10 is about 5.6%, while pass@10 is about 99.9999%. The appropriate metric depends on the product. pass@k is useful when the system can generate several candidates and select or present the best one. pass^k better captures the reliability expected from a customer-facing agent that must succeed repeatedly.

10.7 The Eight-Step Roadmap

Anthropic distills the path from having no evals to having a trusted eval suite into the following roadmap ([Anthropic — Demystifying Evals for AI Agents](#)):

0. **Start early** — 20–50 tasks from real failures.
1. **Begin with what you already test manually** — pre-release checks and bug-tracker queue items.
2. **Write unambiguous tasks with reference solutions** — two domain experts should be able to reach the same verdict. A 0% pass rate across many trials usually indicates a broken task rather than an incapable agent.
3. **Build balanced problem sets** — include cases in which a behavior should occur and cases in which it should not. One-sided evals encourage one-sided optimization.

4. **Build a robust eval harness with a stable environment** — isolate trials and prevent shared state. Anthropic observed Claude gaining an unfair advantage by inspecting git history left behind by earlier trials.
5. **Design graders thoughtfully** — use deterministic checks where possible, award partial credit for multi-component tasks, calibrate LLM judges against structured rubrics, provide an "Unknown" escape hatch to reduce hallucination, and design against reward hacking.
6. **Read transcripts** — failures should appear fair when examined. If scores stop improving, determine whether the agent has regressed or the eval has become unfair.
7. **Monitor for capability eval saturation** — an eval at 100% no longer provides a useful improvement signal. SWE-bench Verified began at 30% and is now nearing 80%, where apparently small score increases can conceal substantial capability gains.
8. **Maintain the suite through open contribution** — domain experts and product teams should contribute eval tasks. Product managers, customer-success teams, and salespeople can use Claude Code to submit evals as pull requests.

10.8 What Real Evals Look Like for Different Agent Types

The appropriate eval design depends on the type of agent. The following representative examples come from Anthropic's survey ([Anthropic — Demystifying Evals for AI Agents](#)):

- **Coding agents** are a natural fit for deterministic graders: Does the code run? Do the tests pass? SWE-bench Verified runs a test suite associated with a fixed GitHub issue, while Terminal-Bench evaluates end-to-end tasks such as building a Linux kernel from source.
- **Conversational agents** require multidimensional measures of success: Was the ticket resolved (state check)? Did the conversation stay under 10 turns (transcript constraint)? Was the tone appropriate (LLM rubric)? These evals often use a second LLM to simulate the user, as in τ -Bench and τ^2 -Bench.
- **Research agents** need groundedness checks (are claims supported by sources?), coverage checks (are the key facts included?), and source-quality checks (are sources authoritative rather than merely the first retrieved?). Their graders require frequent calibration against expert humans.
- **Computer-use agents** need a real or sandboxed environment, URL or page-state checks, and backend state verification. For example, was an order actually placed, or did the agent only reach a confirmation page? WebArena and OSWorld are canonical examples.

10.9 Verification Feedback for Coding Agents

For coding agents, the most useful graders also serve as repair signals. A check that reports only "test failed" confirms a bad outcome but gives the agent little help in correcting it. A useful failure message identifies the affected path, the expected and observed states, and the next place to inspect. OpenAI's Codex harness guidance recommends converting recurring review comments and architectural rules into repository-local checks. This gives agents specific feedback while they still have an opportunity to fix the work ([OpenAI — Harness Engineering](#)).

End-to-end verification should be a completion gate, not a ceremonial final step. In Anthropic's long-running application harness, the coding agent had to start the app and verify the feature through a browser-driven workflow. Without that requirement, agents tended to declare a feature complete after local tests or visual inspection even when the actual user flow was still broken ([Anthropic — Effective Harnesses for Long-Running Agents](#)). The general rule from §10.2 applies directly: grade the state of the environment, not the agent's confidence.

10.10 Reading Transcripts Is the Skill

One lesson recurs throughout eval work: do not accept scores at face value until someone has read the transcripts. Anthropic describes a case in which Opus 4.5 initially scored 42% on CORE-Bench. Investigation found rigid grading that rejected "96.12" when the expected answer was "96.124991...", ambiguous task specifications, and stochastic tasks that could not be reproduced exactly. After the grading bugs were fixed and the model was run with a less constrained scaffold, the score rose to 95% ([Anthropic — Demystifying Evals for AI Agents](#)). METR found a related problem in its time-horizon benchmark: some tasks instructed agents to optimize to a stated score threshold, but the grader required them to exceed it. The eval therefore penalized models that followed the instructions and rewarded those that ignored them ([Anthropic — Demystifying Evals for AI Agents](#)).

The general rule is that failures should look fair when inspected. When scores plateau, ask whether the agent has stopped improving or the eval has stopped measuring the intended capability.

10.11 Evals Are One Layer of Many

Automated evals provide only part of the picture. Anthropic compares evaluation to the Swiss-cheese model from safety engineering: every layer has gaps, so no single layer catches every problem ([Anthropic — Demystifying Evals for AI Agents](#)). A complete evaluation stack includes:

- **Automated evals** for fast iteration, regression detection, model upgrades.
- **Production monitoring** for ground truth and unanticipated real-world failures.
- **A/B testing** for validating significant changes once traffic is sufficient.
- **User feedback** for surfacing problems no one anticipated.
- **Manual transcript review** for building intuition about failure modes.
- **Systematic human studies** for calibrating LLM graders and assessing subjective outputs.

10.12 Readiness Validation and Failure Attribution

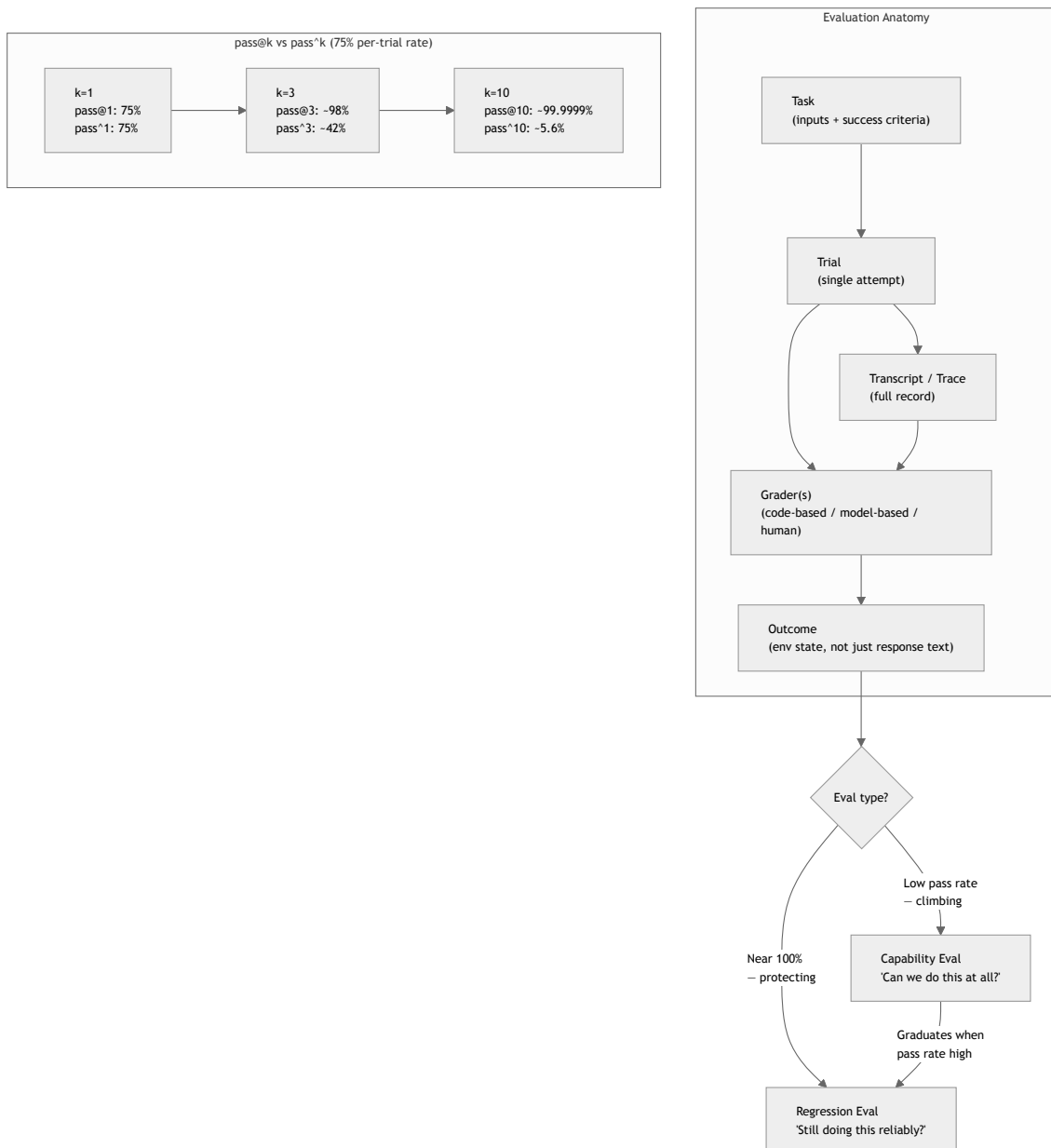
The OpenReview survey separates the **Verification** layer from evaluation in general because a harness needs more than scorekeeping. Verification asks whether a specific agent-harness combination is ready for deployment on a defined task distribution, within a defined environment, budget, and governance regime ([OpenReview — Agent Harness Engineering: A Survey](#)).

That framing adds two practical requirements to ordinary eval suites:

- **Readiness validation:** a release gate should bind together the tasks, environment-reset rules, available tools, context policy, budget limits, and governance checks. A score measured on a different execution substrate or with a different tool menu does not automatically carry over.
- **Failure attribution:** each failure should be assigned to its likely layer—execution, tool interface, context, lifecycle, observability, verification, or governance. Without this attribution, teams may keep tuning prompts when the actual defect is an unstable sandbox, an oversized tool surface, a missing checkpoint, or a weak policy hook.

This dependence on configuration also explains why benchmark numbers are fragile. Infrastructure changes, cost optimizations, and different tool boundaries can change the measured capability of the same model. A useful eval report should therefore record the harness configuration alongside the model name, including the environment image, resource limits, tool catalog, context-assembly policy, retry rules, graders, and human-approval requirements.

Diagram: Eval Anatomy and pass@k vs pass^k



Key Takeaways

- **Evals are compounding infrastructure:** start with 20–50 tasks drawn from real failures, even before the agent is mature.
- **Evaluation is broader than unit testing:** it tests the model-plus-harness system against task outcomes in an environment.
- **Outcome ≠ response:** measure the state of the environment—a database row, URL, or file—not only what the agent said.
- **Repair-oriented feedback improves self-correction:** checks should tell the agent what failed, where it failed, and what evidence would demonstrate a fix.
- **Three grader types form a pyramid:** use code-based graders for speed, model-based graders for nuance, and humans for calibration.
- **The verifier is part of the system under test:** blind judges to irrelevant consequences, preserve evidence, calibrate with meta-evals, allow abstention, and retain reproducible audit artifacts.
- **pass@k and pass^k serve different products:** multi-attempt generation can use pass@k; repeated customer-facing execution needs pass^k-style reliability.
- **Reading transcripts is the essential skill:** scores can plateau because the agent has regressed or because the eval is unfair; only the transcripts reveal which.

- **Readiness is configuration-specific:** eval results should travel with the harness configuration that produced them.
- **Evals are one layer of many:** automated evals + production monitoring + A/B testing + user feedback + human review.

Further Reading

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Anthropic Safeguards Research Team, *Agentic Misalignment: Summer 2026 Update*, 2026. <https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>
- OpenAI, *Trustworthy Third-Party Evaluations: Foundations*, 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>

Chapter 11: Infrastructure Noise

Small differences on benchmark leaderboards carry more uncertainty than their decimal precision suggests.

Anthropic's "Quantifying Infrastructure Noise" shows how large that uncertainty can be ([Anthropic — Quantifying Infrastructure Noise in Agentic Coding Evals](#)).

Static benchmarks score model outputs directly. Agentic coding evals work differently: the model writes programs, runs tests, installs dependencies, and revises its approach over many turns. The runtime is therefore not a passive container; it is part of the problem-solving process. Two agents given different resource budgets are not taking the same test.

11.1 The Headline Result

Anthropic ran Terminal-Bench 2.0 across six resource configurations on a Google Kubernetes Engine cluster. The Claude model, harness, and task set remained the same; only the resource floor and ceiling changed. The resulting gap between the most- and least-resourced configurations was 6 percentage points ($p < 0.01$) ([Anthropic — Quantifying Infrastructure Noise](#)).

That difference is larger than the typical leaderboard gap between leading models. The implication is straightforward: a 2-point lead may represent a real capability advantage, or it may simply mean that one eval ran on more capable hardware.

11.2 The Two Regimes

The results reveal two distinct regimes:

- **From 1× to 3× the per-task resource specification**, scores fluctuated within statistical noise ($p = 0.40$), while infrastructure error rates fell steadily—from 5.8% under strict enforcement to 2.1% with 3× headroom ($p < 0.001$). The added capacity absorbed transient memory spikes that would otherwise trigger out-of-memory container failures. Across this range, the added resources made the eval more stable without measurably making the tasks easier.
- **Above 3×**, scores rose faster than infrastructure error rates declined. Between 3× and uncapped resources, infrastructure errors fell by 1.6 percentage points, but task success increased by almost 4 points. At this level, extra resources did more than prevent crashes. They enabled strategies that depended on generous allocations, such as installing large dependencies, running memory-intensive test suites, or brute-forcing solutions with heavyweight tools.

11.3 What This Means for Measurement

Tight resource limits reward efficient strategies, while generous limits reward agents that can exploit the available capacity. Both are legitimate properties to measure. The problem arises when results from different configurations are collapsed into a single score without documenting those configurations.

Anthropic's `bn-fit-modify` task illustrates the effect. With generous limits, some models begin by installing an entire Python data-science stack—pandas, networkx, and scikit-learn—before writing any solution code. With tight limits, the pod runs out of memory during installation. A leaner strategy is available: implement the required mathematics from scratch using only the standard library. Some models choose that approach by default. The resource configuration therefore determines which default strategy succeeds ([Anthropic — Quantifying Infrastructure Noise](#)).

The same effect appears outside Terminal-Bench, although its magnitude is smaller. In Anthropic's SWE-bench experiment, a 5× RAM allocation produced scores 1.54 percentage points higher than a 1× allocation across 227 problems. The difference was smaller than on Terminal-Bench because SWE-bench tasks are less resource-intensive, but the resource increase was still not neutral ([Anthropic — Quantifying Infrastructure Noise](#)).

11.4 The Recommendation

An eval should specify a guaranteed allocation (the floor) separately from a hard ceiling rather than pinning resources to a single value. For Terminal-Bench, a ceiling of 3× the per-task specification is a reasonable default: it reduced infrastructure errors by two-thirds while keeping the score increase well within statistical noise ([Anthropic — Quantifying Infrastructure Noise](#)). The appropriate multiplier depends on the benchmark and task distribution, so every result should report it explicitly.

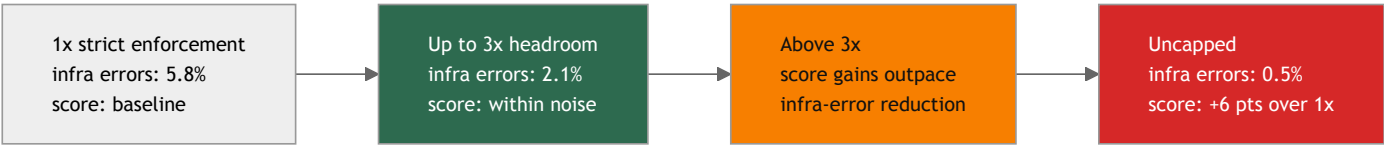
For readers of benchmark results, the operational rule is simple: treat leaderboard differences below 3 percentage points with skepticism until resource configurations are documented and matched. A lead of a few points may reflect

a genuine capability difference—or merely a larger virtual machine.

Diagram: Resource Configuration vs. Score (Terminal-Bench 2.0 Summary)

The following table summarizes the two regimes reported by Anthropic. The article quantifies error rates only for the 1×, 3×, and uncapped configurations. The intermediate row is therefore qualitative, avoiding a level of precision that the source does not provide.

Resource Level	Infra Error Rate	Score Change	Interpretation
1× (strict)	5.8%	baseline	OOM kills obscure substantive failures
3×	2.1%	+noise	Practical balance: infra errors cut by 2/3
Above 3×	lower	score begins rising faster than infra errors fall	Agents start exploiting extra RAM
Uncapped	0.5%	+6 pts over 1×	Resource-intensive defaults succeed



Above 3×, score gains outpace the decline in infrastructure errors: extra resources enable new strategies rather than merely improving stability.

Key Takeaways

- **Hardware alone produced a 6-point gap:** the difference between the most- and least-resourced Terminal-Bench 2.0 configurations exceeded typical leaderboard gaps between leading models.
- **There are two resource regimes:** moving from 1× to 3× reduces infrastructure instability; moving above 3× enables resource-intensive strategies.
- **A 3× ceiling is the practical default:** it reduces infrastructure errors by two-thirds while keeping the score increase within statistical noise.
- **Small leaderboard differences deserve skepticism:** differences below 3 percentage points are difficult to interpret until resource configurations are documented and matched.
- **Resource limits shape strategy:** tight limits reward efficiency, while generous limits reward the ability to exploit available capacity. Both are valid measurements, but they should be distinguished.

Further Reading

- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

Chapter 12: Trace-Driven Iteration and Model–Harness Co-Evolution

12.1 Traces Are the Feedback Loop

Today's models remain largely black boxes: their internal mechanisms are difficult to interpret. Their inputs and outputs, however, are visible, and that is enough to support systematic improvement. LangChain therefore treats traces as the primary surface for debugging a harness. It records every agent action, along with latency, token counts, costs, and tool invocations ([LangChain — Improving Deep Agents with Harness Engineering](#)).

Its *Trace Analyzer Skill* automates this loop:

1. Fetch experiment traces from LangSmith.
2. Spawn parallel error-analysis agents; the main agent synthesizes findings and suggestions.
3. Aggregate feedback and make targeted changes to the harness.

The structure resembles boosting in classical machine learning: each iteration focuses on mistakes from earlier runs. Human review at step 3 is helpful, though not strictly required. Its main value is catching changes that improve a few specific tasks at the expense of broader performance.

12.2 Runtime and Process Observability

Trace-driven iteration depends on two kinds of observability. **Runtime observability** records what the system actually did: tool calls, command outputs, browser actions, latency, token usage, errors, retries, and the final state of the environment. This is the layer LangChain emphasizes when it stores every agent action and aggregates recurring failure patterns from traces.

Process observability records what success meant and why the completed work should be accepted: task scope, sprint contracts, rubrics, verification evidence, excluded work, and handoff notes. Anthropic's generator-evaluator harness made this explicit. Before implementation, a negotiated sprint contract established the scope; if a sprint failed, the evaluator used a rubric to provide specific feedback ([Anthropic — Harness Design for Long-Running Application Development](#)).

The two layers reinforce each other. Runtime signals without process artifacts show what happened, but not whether the result met the intended scope. Process artifacts without runtime signals can become convincing paperwork wrapped around broken behavior. A production harness should make both inspectable: the task trajectory, the acceptance criteria, and the evidence that the environment reached the desired state.

The OpenReview survey adds an important implementation detail: agent traces should be structured as trees of spans rather than flat logs ([OpenReview — Agent Harness Engineering: A Survey](#)). At a minimum, those spans should cover model calls, tool invocations, retrieval steps, context-assembly operations, latency, token usage, cost, retries, permission decisions, and the final outcome state.

That structure is now being standardized. OpenTelemetry, the vendor-neutral observability standard already used across distributed systems, is defining *GenAI semantic conventions*: a shared schema of span and attribute names for LLM and agent telemetry ([OpenTelemetry — Semantic conventions for generative AI spans](#)). The conventions name the spans a harness produces, including a top-level `invoke_agent` span, child `chat` spans for model calls, and `execute_tool` spans for tool invocations. They also define standard metrics such as operation duration and token usage.

Adopting these conventions offers two benefits. First, agent traces can use the *same* observability stack as the rest of the system. Teams can query latency, cost, and errors with existing tools instead of maintaining a bespoke agent logger. Second, the standard addresses privacy directly. It defines explicit options for prompt and completion content: omit it, attach it to the span, or store it externally and record a reference. It also recommends against capturing sensitive payloads by default, matching the redaction discipline required by §12.3 and Chapter 17. As of this writing, the GenAI conventions are still marked *Development*, so exact names may change. Their value lies in the direction of the standard, not in any one version.

12.3 From Production Trace to Regression Case

Observability should feed directly into verification. A trace that captures a real production failure is too valuable to remain only a debugging artifact. A mature workflow is:

1. Capture a failed or surprising production trajectory.

2. Redact sensitive data and freeze the relevant environment or fixture.
3. Extract the user intent, tool sequence, intermediate state, and final outcome.
4. Write a deterministic or model-graded assertion for the corrected behavior.
5. Add the case to the regression suite, with the original trace as evidence.

This workflow turns traces into a source of eval tasks. It also prevents teams from optimizing for synthetic benchmarks while overlooking failures encountered by their own users. Governance remains part of the pipeline: trace-to-eval systems must preserve privacy, provenance, and permission metadata. Otherwise, they may produce tests that are technically useful but operationally unsafe.

12.4 From Practitioner Correction to a Bounded Improvement Task

OpenAI's tax-agent case study extends the loop by one more step. Expert corrections and production traces did not directly rewrite the deployed agent. Instead, they became reviewed findings, tailored evals, and bounded Codex tasks that could be verified before release ([OpenAI — Building Self-Improving Tax Agents with Codex](#)).

A safe improvement pipeline has explicit custody at every step:

1. Capture a practitioner correction with the trace and outcome that motivated it.
2. Redact sensitive data, review the finding, and group duplicates into a stable failure class.
3. Convert the class into a reproducible regression eval with immutable evidence.
4. Give a coding agent one bounded change task rather than authority to modify itself freely.
5. Gate the patch on the targeted eval, the wider regression suite, security checks, cost, and latency.
6. Canary the version in production; monitor the original failure signal; roll back if it worsens.

This is **controlled self-improvement**, not uncontrolled self-modification. The deployed agent does not edit its own prompts, tools, memory policy, or evaluator in place. Instead, an external improvement loop proposes a versioned change, and independent evidence and release controls determine whether it advances. Evaluator integrity (§10.4) matters at two points: the original finding must be valid, and the release gate must not be biased toward approving the proposed fix.

12.5 Stress-Test Load-Bearing Components

Anthropic's harness-design follow-up adds a complementary discipline ([Anthropic — Harness Design for Long-Running Application Development](#)). Every harness component encodes an assumption about what the model cannot do on its own. As models improve, those assumptions can become outdated. The recommended test is simple: remove one component, run the eval, and observe the result.

When Opus 4.6 launched with stronger long-context retrieval and better long-horizon coding behavior, Anthropic was able to remove the sprint construct from one version of its harness. The generator remained coherent for more than two hours without sprint decomposition. The evaluator, which had carried more of the workload for earlier models, became useful only in some cases: it still helped with tasks near the edge of what the generator could complete alone, but added unnecessary overhead for tasks well within that boundary. The team summarizes the principle this way: "the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo."

12.6 Model-Harness Co-Evolution

Today's frontier coding models are post-trained with their harnesses in the loop ([LangChain — The Anatomy of an Agent Harness](#)). Teams discover useful primitives, add them to the harness, and use them while training the next model generation. The resulting model becomes more capable within that harness. This feedback loop also creates coupling: changing the harness can reduce model performance even when the change appears behaviorally neutral.

The Codex `apply_patch` tool is the canonical example. Codex models are post-trained on this specific patch format. OpenCode, an open-source alternative to Claude Code, therefore had to add an `apply_patch` tool specifically for GPT/Codex models so that it could mimic the Codex harness. Claude and other models continue to use conventional `edit` and `write` tools ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

12.7 But the Best Harness Is Not Always the One the Model Was Trained In

This coupling does not mean that a model's training harness is optimal for every task. Terminal-Bench 2.0 is a recurring example in practitioner discussions. HumanLayer cites Opus 4.6 ranking 33rd in Claude Code and 5th in a different harness, with roughly four positions of leaderboard noise ([HumanLayer — Skill Issue: Harness Engineering](#)

for Coding Agents; [LangChain — The Anatomy of an Agent Harness](#)). These exact ranks are snapshots of a leaderboard, not timeless facts about the model.

LangChain's case study reaches the same conclusion experimentally. Claude Opus 4.6 scored 59.6% on an early version of its harness: competitive, but below its tuned Codex configuration. The broad principles—context preparation and verification—transferred across models, but closing the remaining gap would have required several rounds of iteration tailored to Claude ([LangChain — Improving Deep Agents with Harness Engineering](#)).

The pragmatic rule is straightforward: when the model changes, re-examine the harness. Strengthen the components that have become load-bearing, and remove those that no longer help.

12.8 Practical Takeaways

LangChain distills its experience with harness iteration into five principles ([LangChain — Improving Deep Agents with Harness Engineering](#)):

1. **Engineer context on the agent's behalf** — orient the model to its environment, including directory structures, available tools, coding practices, and problem-solving strategies.
2. **Help agents verify their own work** — models tend to favor their first plausible solution, so instruct them clearly to run tests and check the result.
3. **Use traces as a feedback signal** — debug tools and reasoning together, because a model may take the wrong path when it lacks either a tool or instructions for using it.
4. **Contain bad patterns in the short term** — guardrails such as loop detection may become unnecessary as models improve, but they remain useful today.
5. **Tailor the harness to the model** — Claude and Codex prompting guides differ for a reason: broad principles transfer, but implementation details often do not.

HumanLayer's parallel set ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)):

What worked: starting with a simple harness and adding configuration only after real failures; iterating quickly and discarding changes that did not help; distributing proven configurations through repository-level files; optimizing for iteration speed rather than one-shot success; and pruning capabilities once the team understood what it actually needed.

What did not work: trying to design the ideal harness in advance; installing dozens of skills and MCP servers "just in case"; running the full test suite at the end of every session; and micro-optimizing which tools each sub-agent could access.

12.9 The Misleading Data on AGENTS.md

One result is particularly worth noting. An ETH Zurich study tested 138 agentfiles—the generic term for AGENTS.md- or CLAUDE.md-style instruction files—across a range of repositories. It found that LLM-generated files reduced performance while increasing cost by 20%, while human-written files improved performance by only about 4%. Agents also spent 14–22% more reasoning tokens processing context-file instructions. In that benchmark, codebase overviews and directory listings did not help because agents could discover the repository structure on their own ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#) citing the ETH Zurich paper).

HumanLayer interprets these findings as support for its own AGENTS.md guidance: keep files concise, avoid auto-generation, use progressive disclosure instead of presenting every instruction up front, and reserve the root file for rules that apply universally rather than conditional guidance. Its own CLAUDE.md is fewer than 60 lines.

This does not mean "do not write repository instructions." It means that the root instruction file should act as a router, not an encyclopedia. OpenAI's Codex harness guidance follows the same pattern: keep essential context in the repository, keep AGENTS.md concise, direct agents to deeper documents when needed, and enforce the most important rules mechanically where possible ([OpenAI — Harness Engineering](#)).

The broader principle is that more configuration is not automatically better. Every irrelevant instruction consumes attention without improving the result. The *instruction budget* matters as much as the token budget.

12.10 Reusable Harness Packages and Skills

Once a harness pattern has proved useful in practice, it should not remain tribal knowledge confined to one repository. Package it as a skill, a template bundle, a small scaffold generator, or a set of repository checks. Whatever the format, it should include both instructions and working artifacts: not merely "remember to maintain state," but also a progress-log template, a feature-list schema, a startup script, and a validation command.

The Learn Harness Engineering course demonstrates this packaging model with `harness-creator`, a skill for creating, assessing, and improving five harness subsystems: instructions, state, verification, scope, and session lifecycle ([Learn Harness Engineering — Skills](#)). This creates a useful engineering boundary. A reusable package should not freeze one supposedly ideal workflow forever. It should make proven defaults easy to install and inspect—and just as easy to remove when traces show that a component is no longer load-bearing.

12.11 Meta-Harness: Optimizing the Harness Itself

Once evals and traces exist, the harness itself becomes an object of optimization. The OpenReview survey points to *meta-harness* research that explores harness structure, prompting strategies, tool interfaces, and control-loop choices instead of treating the harness as fixed ([OpenReview — Agent Harness Engineering: A Survey](#)).

In practice, this need not mean fully automated architecture search. It can begin with disciplined experimentation:

- A/B test a tool schema or prompt change against the same task suite.
- Ablate one component, such as a planner, context reset, memory layer, or evaluator.
- Vary cost controls, retry policy, and tool-response truncation to see where quality degrades.
- Measure the whole closed loop, not only model output: success rate, pass@k reliability, latency, cost, human escalations, and security false positives.

This extends the idea of load-bearing components: every part of the harness should continue to justify its cost. If a component improves reliability only for rare, high-value tasks, route those tasks through it selectively. If it stops helping after a model upgrade, remove it.

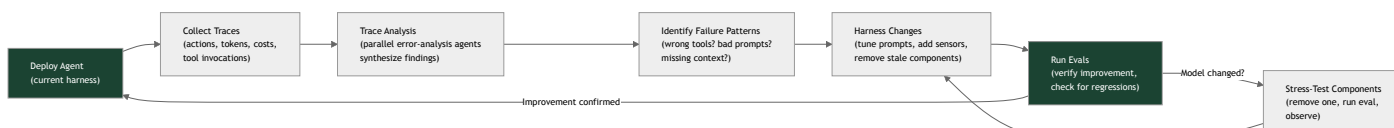
12.12 Cross-Layer Coupling

ETCLOVG also serves as a debugging map. When a trace shows an apparently bad model decision, the underlying cause may lie in another layer ([OpenReview — Agent Harness Engineering: A Survey](#)):

- A tool-selection error may come from the **Tool** layer, because the action space is too large or the schema hides the important affordance.
- A premature completion may come from the **Lifecycle** layer, because exit criteria are not represented as a durable state machine.
- A regression after cost tuning may come from the **Observability** and **Execution** layers, because resource limits changed latency, timeouts, or benchmark fidelity.
- A security failure may come from the **Governance** layer, because policy, permissions, and audit hooks are inconsistent across training-time alignment, deployment configuration, and runtime enforcement.

Before changing a prompt, annotate the trace review with hypotheses about the responsible layer. Ask which layer made the wrong behavior easy, invisible, or inexpensive. Only then decide whether the right fix is a prompt change, a redesigned tool, a sandbox adjustment, a new metric, a stronger grader, or a governance hook.

Diagram: Trace-Driven Iteration Loop



Key Takeaways

- **Traces are the primary debugging surface:** text I/O is visible even when model internals are not — systematic trace analysis drives harness improvement.
- **Observability has two layers:** runtime traces show what happened; process artifacts show why the work should be accepted.
- **Trace spans should be structured:** model calls, tool calls, retrieval, context assembly, permissions, cost, and outcome state need machine-readable telemetry.
- **A standard is emerging for that telemetry:** OpenTelemetry's GenAI semantic conventions (`invoke_agent`, `chat`, `execute_tool` spans) let agent traces join the ordinary observability stack and codify content-capture patterns for privacy.

- **Production traces should become regression cases:** real failures are the highest-signal eval tasks if privacy and provenance are preserved.
- **Self-improvement must be an external, bounded release loop:** review and group corrections, convert them into evals and scoped change tasks, then gate, canary, and roll back versioned artifacts.
- **The trained harness is not automatically optimal:** leaderboard snapshots show the same model moving substantially under different harnesses.
- **Stress-test components when models change:** every harness component encodes an assumption that may go stale as models improve.
- **Model-harness co-evolution is real:** post-training loops the harness into model training, creating coupling that breaks when either side changes unexpectedly.
- **AGENTS.md has limited ROI when bloated:** keep it concise and human-written, then use progressive disclosure into repo-local docs and mechanical checks.
- **Reusable harness packages preserve hard-won practice:** promote stable patterns into skills, templates, scaffolds, and checks once traces show they help.
- **Meta-harness turns evals into design search:** prompts, tools, retries, context policies, and evaluators can be ablated and optimized like any other system component.
- **Layer attribution prevents prompt-only fixes:** use ETCLOVG to ask which harness layer made a failure possible before changing instructions.
- **Iteration speed beats upfront design:** start simple, add only after real failures, prune aggressively.

Further Reading

- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Walking Labs, *Learn Harness Engineering — Skills*. <https://walkinglabs.github.io/learn-harness-engineering/en/skills/>
- OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>

Chapter 13: System Prompts and Instruction Architecture

The core diagram in the [Preface](#) lists *System Prompts* as the first component of the harness, but the preceding chapters have mostly treated them as a given. This chapter examines the instruction layer directly: what belongs there, how the agent prioritizes competing instructions, how the harness assembles those instructions at runtime, and why the entire layer deserves the same engineering discipline as the rest of the system.

13.1 The System Prompt Is a Harness Layer, Not a Prompt

A "prompt" in casual use is a question typed into a chat box. An agent's system prompt is different: it is a persistent program that ships with the harness and frames every turn of the agent loop. HumanLayer's twelve-factor manifesto captures this distinction with the principle *own your prompts*. For a production agent, the prompt is core engineering logic, not a string that should disappear inside a framework's hidden defaults ([HumanLayer — 12-Factor Agents](#)). The rest of this chapter starts from that premise: treat the system prompt as application code.

The distinction matters because the system prompt performs work that no other layer can. It defines the agent's role and objective, explains when available tools should be used, encodes policies the agent must follow, and sets the output contract. If any of these elements is vague, the system will not raise a syntax error. Instead, the agent may drift from the task, act too eagerly, refuse unnecessarily, or use the right tool at the wrong time. Such failures are often difficult to diagnose precisely (Ch 10, [Ch 12](#)).

13.2 The Instruction Hierarchy

An agent receives text from several sources at once: the platform's system prompt, developer configuration, the user's request, and—critically—tool results and retrieved documents. Not all of this text carries the same authority. OpenAI formalized this distinction as the *instruction hierarchy*: models can be trained to prioritize system-level instructions over user-level instructions, and both over content encountered in tool outputs. The goal is to prevent lower-priority text from overriding higher-priority instructions ([OpenAI — The Instruction Hierarchy](#)).

For the harness engineer, this is the instruction-side view of the security boundary described in [Chapter 5](#). Prompt injection is a failure of that boundary: text in a web page or file tries to elevate itself from untrusted data to an authoritative instruction ([the lethal trifecta](#), Ch 5). The hierarchy provides two complementary defenses:

- **Use the model's trained priorities** by placing genuine policy in the system position, never in user-editable or tool-supplied content.
- **Reinforce the boundary structurally** by labeling untrusted spans as data, because a trained priority is a tendency rather than a guarantee.

The practical rule is that an instruction's authority should come from *where the harness places it*, not from how forcefully it is worded. A critical constraint belongs in the system layer. The same text inside a retrieved document has no such authority and should be treated as data rather than policy.

13.3 What Belongs in the System Prompt

Not every relevant fact belongs in the system prompt. An overstuffed prompt consumes the attention budget described in [Chapter 2](#) before the task even begins. A useful division of responsibility is:

- **System prompt:** the agent's role and goal, high-value rules that always apply, tool-use policies that individual tool descriptions cannot express (such as when *not* to use a tool or how tools relate to one another), and the output contract.
- **Tool descriptions:** the mechanics of each individual tool (Ch 4). Repeating those details in the system prompt creates two sources of truth that can drift apart.
- **Retrieved context:** facts that change frequently, require substantial space, or matter only for certain tasks. They should arrive through just-in-time retrieval (Ch 2), not be embedded in a static prefix.

13.4 Dynamic Assembly and the Stable-Prefix Constraint

Most production agents do not use one fixed string. The harness *assembles* the system prompt for each call from several parts: a base policy, the current tool set, project-specific instructions, and possibly a retrieved skill ([Chapter 4](#)). This assembly must account for the KV-cache economics described in [Chapter 2](#). Because the cache can reuse only a byte-identical prefix, material that changes between calls should appear *after* material that remains stable.

Instruction architecture is therefore also a layout problem. Stable, reusable material—such as base policy and the standing tool catalog—goes first. Volatile material—such as the current task, newly retrieved facts, and timestamps—

goes last. If a system prompt inserts the current time or a request ID near the top, it quietly defeats prefix caching on every turn, adding latency and cost without improving behavior.

13.5 Treat Prompts as Versioned Code

Because the system prompt is load-bearing, changing it means changing the system. A prompt edit carries the same regression risk as a code change, so the discipline of [Chapter 10](#) applies directly: run the eval suite before and after the edit, then compare pass rates, failure categories, cost, and latency. The model–harness coupling described in [Chapter 12](#) makes this even more important. A prompt tuned for one model may regress on another, so prompts should be versioned alongside the models against which they were validated.

In practical terms, the instruction layer needs version control, a changelog tied to eval results, and a clear owner. "Own your prompts" means treating each prompt as a maintained artifact with a history—not as a string someone edits live in production ([HumanLayer — 12-Factor Agents](#)).

13.6 The Right Altitude

The hardest judgment in writing a system prompt is choosing the right level of specificity. Anthropic describes this as finding the *right altitude*. At too low an altitude, the prompt becomes a brittle collection of hardcoded if-then rules that fails on unanticipated cases and grows without bound. At too high an altitude, the guidance becomes so abstract that the model has no concrete signal to act on ([Anthropic — Effective Context Engineering for AI Agents](#)). The goal is guidance specific enough to shape behavior reliably, yet general enough to transfer across the cases the agent will encounter.

This is also where over-engineering tends to hide. A special-case rule added in response to one bad trace can become a long-term liability: it narrows behavior, consumes context, and may conflict with a rule added later. Often, the better fix is not another sentence in the system prompt but a tool change, a sensor (Ch 5), or an eval case that makes the expected behavior explicit (Ch 10). Traces provide the evidence needed to choose among these options ([Chapter 12](#)).

13.7 Layered Instructions Across the Three Harness Rings

The instruction layer is not monolithic. It mirrors the inner and outer harness structure described in [Chapter 1](#). A coding agent typically combines:

- the **builder harness** system prompt shipped by the lab,
- the **user harness** project instructions added by the team— `AGENTS.md` files, repository conventions, and review rules (Ch 12)—and
- on-demand **skills** loaded for a particular task (Ch 4).

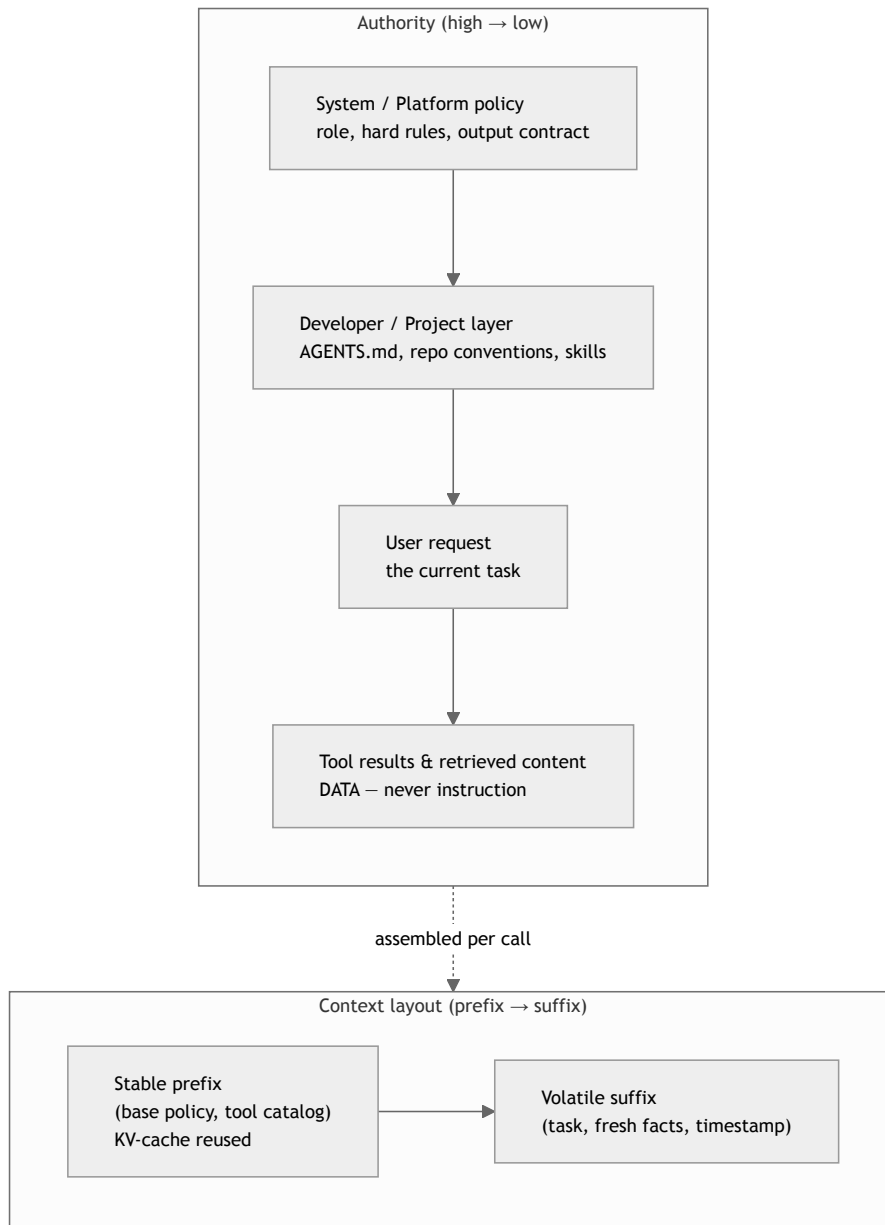
At runtime, these layers form the effective instruction set seen by the model. The lesson from [Chapter 12](#) still applies: more project instruction is not always better. Evidence on sprawling `AGENTS.md` files is mixed, and an over-specified project layer can conflict with the builder harness beneath it. Instruction architecture should be measured, not maximized.

13.8 Scoped Review Rules as an Instruction Interface

Repository instructions can guide code review as well as code generation. OpenAI's custom Codex review guidance treats concise `AGENTS.md` rules as a **review interface**. Each rule should identify a specific defect, explain where the rule applies, and state how a reviewer can recognize the problem. Directory-scoped files keep those rules close to the code they govern ([OpenAI — Custom Code Review Rules for Codex](#)).

Good review rules are short, testable, and high-signal: "flag a new endpoint that bypasses `authorize()`" rather than "follow security best practices." They point to the relevant code or policy and, where possible, to a deterministic check. Like any other instruction, review rules should be versioned and evaluated. Noisy rules create false positives and eventually teach humans to ignore the entire review surface. This is progressive disclosure applied to quality policy: place each rule near the code it governs, reserve global instructions for universal requirements, and move stable invariants into linters or tests.

Diagram: The Instruction Stack



Authority flows top-down and is enforced by position, not phrasing; layout flows prefix-to-suffix and is governed by cache economics. The two axes are independent and both must be designed.

Key Takeaways

- **The system prompt is application code:** a persistent, load-bearing harness layer, not a casual string — own it, version it, and test changes against evals.
- **Authority comes from position, not emphasis:** the instruction hierarchy makes system instructions privileged over user input and tool content; prompt injection is a hierarchy failure, so label untrusted spans as data.
- **Separate what belongs where:** role and policy in the system prompt, mechanics in tool descriptions, changing facts in just-in-time retrieval.
- **Assemble for the cache:** stable instructions first, volatile material last, or prefix caching silently breaks.
- **Aim for the right altitude:** specific enough to steer, general enough to transfer; resist patching every trace with a new hardcoded rule.
- **Review rules are an instruction interface:** scope concise, testable defect rules near the code they govern, measure their signal, and move stable invariants into deterministic checks.

Further Reading

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

Chapter 14: Model Selection, Routing, and Reasoning Models

Agent = Model + Harness. The previous chapters focused on the harness and treated "the model" as a single, fixed choice. In practice, model selection is another design surface. The harness must decide which model should run each step, whether a cheaper model is sufficient, and — with the arrival of reasoning models — how much inference budget to spend before the model acts. This chapter examines those operational decisions. For an account of what these models do internally, see the companion volume *LLM Foundations*.

14.1 Model Choice Is a Harness Decision

The first principle of [Chapter 6](#) — do the simplest thing that works — applies to model choice as well as control flow. The largest, most capable model is rarely the right default for every step in an agent loop. Most loops mix a few difficult planning or synthesis tasks with many simpler ones, such as classifying an input, formatting a result, or extracting a field. The difficult steps may genuinely need a frontier model; the simpler ones can often be handled just as well by a smaller, faster, and cheaper model ([Anthropic — Building Effective Agents](#)).

Treating the model as a single global setting gives up this opportunity. A harness can choose a model for each step; the unit of selection is the call, not the application. This generalizes the model-routing idea introduced in [Chapter 3](#): capability that does not improve the outcome is wasted capability. In production, that waste appears as added latency and cost on every request.

14.2 Routing: Match Model Strength to Task Difficulty

In the workflow sense (Ch 6), *routing* dispatches an input to a specialized branch. *Model routing* applies the same idea to model choice: estimate the difficulty of a request, send easy requests to a smaller, cheaper model, and reserve a stronger, more expensive model for harder ones. The classifier is the difficult part. Routing pays off only when the decision is reliable and costs much less than the model usage it saves.

RouteLLM showed that this decision can be learned: a router trained on preference data sends each query to either a strong or a weak model, recovering most of the strong model's quality at a fraction of its cost on standard benchmarks ([RouteLLM](#)). The harness-level lesson is broader than any particular router. A small, fast decision before an expensive call can improve the cost–quality tradeoff, but only if that decision is itself cheap and dependable. A hard task routed to an inadequate model may fail silently, and the resulting error can cost far more than the model call saved.

14.3 Cascades and Fallbacks

Routing chooses a model before attempting the task. A *cascade* defers that choice: try the cheaper model first, accept its answer if it satisfies a verifier, and escalate to a stronger model only if it does not. FrugalGPT demonstrated cascades that match a frontier model's accuracy at substantially lower cost by handling most queries with cheaper models and escalating only the difficult remainder ([FrugalGPT](#)).

A cascade is only as good as its escalation signal: the verifier that decides whether the first answer is acceptable. It can use the same verification mechanisms discussed throughout this book — a code-based check, a test, schema validation, or a model-based grader (Ch 5, [Ch 10](#)). Without a trustworthy signal, a cascade merely adds latency before returning the same wrong answer.

A related production pattern is the *fallback*. If the primary model errors, times out, or is rate-limited, the harness switches to an alternative so the agent degrades gracefully instead of stalling. This is a reliability mechanism rather than a quality decision, and it belongs alongside the checkpoint-and-resume discipline of [Chapter 7](#).

14.4 Reasoning Models and Test-Time Compute

A reasoning model is post-trained — often through reinforcement learning on problems with checkable answers — to perform extended internal reasoning before committing to an answer. It spends additional inference tokens to improve performance on difficult problems. DeepSeek-R1 documented that reinforcement learning with verifiable rewards can elicit strong reasoning behavior ([DeepSeek-R1](#)). Other work has shown that *test-time compute* — allowing a model to spend more computation during inference — can be a better use of a fixed budget than choosing a larger model for some problems ([Snell et al. — Scaling LLM Test-Time Compute](#)). The companion volume explains the mechanism (*LLM Foundations*, Ch 7–8). The operational consequence is that "how long the model thinks" is now a harness-controlled axis distinct from "which model runs."

This differs from the scaling discussed in [Chapter 11](#). There, additional hardware enables more agent *actions*. Here, additional inference budget buys more *reasoning* within a single call, before the model uses any tool. Both consume resources, but they are not interchangeable.

14.5 Harnessing Reasoning Models: Don't Fight the Training

A reasoning model changes several harness assumptions. The main mistake is to work against behavior the model was trained to provide:

- **Do not manually prompt for reasoning the model already performs.** Adding instructions such as "think step by step" to a model trained to reason can waste tokens or conflict with its learned behavior. Follow the provider's guidance for that model class.
- **Budget for invisible tokens.** A reasoning model may generate thousands of hidden reasoning tokens before producing its first visible word. Those tokens still add latency and cost. If the model exposes a reasoning-effort control, treat it as a first-class harness parameter and, where appropriate, expose it to the user.
- **Do not treat the trace as an explanation.** Reasoning text may be hidden, summarized, or unfaithful to the model's actual computation. Treat any exposed reasoning as a debugging aid, not as verification — the same caution that the evaluation chapter applies to model self-reports (Ch 10).
- **Reasoning is not grounding.** More internal reasoning does not create evidence. A model can analyze supplied evidence more carefully, but it cannot produce facts it was never given. Tools, retrieval, and verification therefore remain necessary (Ch 4, [Ch 10](#)).

14.6 Routing to Reasoning: When the Extra Tokens Pay Off

Reasoning models are most useful for mathematics, code, planning, and multi-step analysis — tasks that contain a difficult but checkable subproblem. They add less value to simple extraction, formatting, and classification, where extra reasoning often increases latency and cost without improving quality. Reasoning effort should therefore be routed like any other capability: send the difficult, verifiable step to a reasoning model, while keeping the surrounding work on a fast non-reasoning model (§14.2).

This fits naturally with the micro-agent pattern from [Chapter 6](#). A deterministic DAG can place a reasoning-model call exactly where deliberation is needed, with cheaper deterministic work before and after it. This "reasoning sandwich" avoids paying for extended reasoning on every turn of an open-ended loop.

14.7 Model Upgrades Are Harness Events

Frontier models are increasingly post-trained with their harnesses in the loop, so model and harness behavior become coupled ([Chapter 12](#)). Replacing the model — with an upgrade, a cheaper provider, a quantized variant, or a different reasoning class — is therefore a system change, not a drop-in substitution. A model that performs better on public benchmarks can still regress on a specific workflow because it interprets tool descriptions differently, produces a different level of detail, or deliberates where a quick answer is required.

The discipline from [Chapter 10](#) applies: treat every model change as a regression event. Run the evaluation suite before and after the change. Compare failure categories and cost, not only pass rate. Keep prompts and tools versioned with the model against which they were validated. Readiness is a property of a model-plus-harness configuration, not of the model alone.

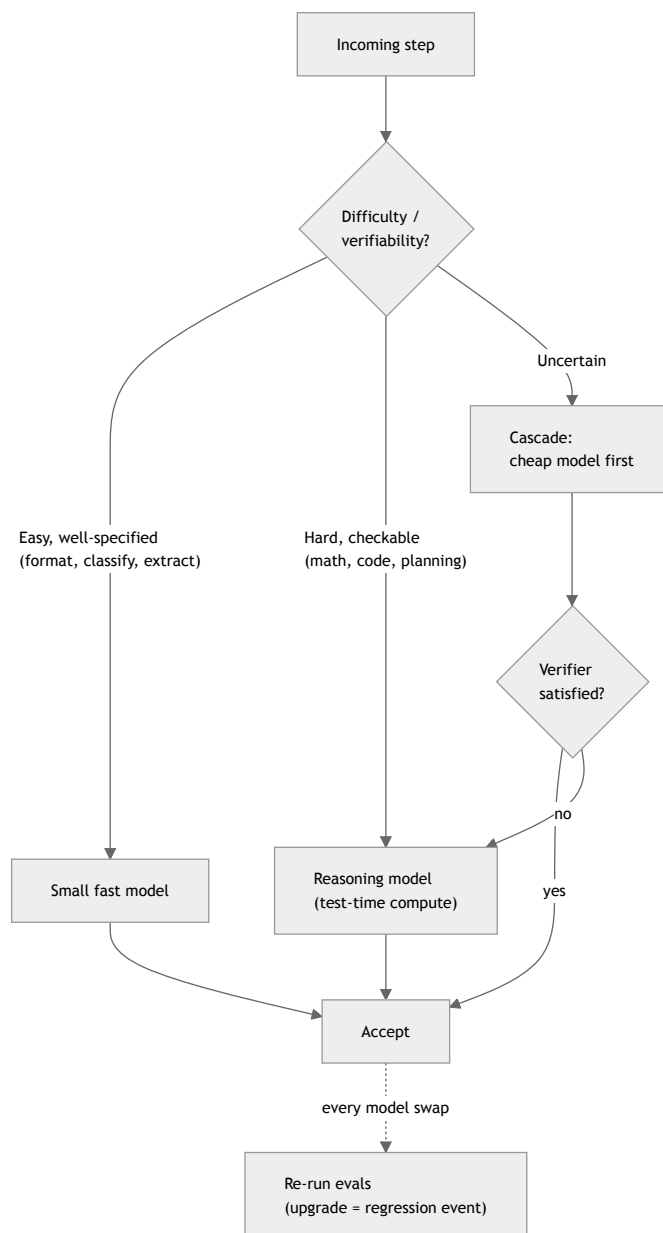
14.8 The AI Gateway: Routing as Infrastructure

Sections 14.2–14.3 described routing, cascades, and fallbacks as application *logic*. In production, much of this logic can live in a dedicated infrastructure component: an *AI gateway*, also called an LLM gateway or LLM proxy. The gateway sits between the harness and model providers. It presents one endpoint — often OpenAI-compatible — across multiple providers and handles shared concerns such as routing, fallback, load balancing, retries, per-key budgets and rate limits, caching, and request/response logging. Open-source examples include LiteLLM, which places more than 100 providers behind one interface and provides cost tracking and load balancing ([LiteLLM](#)), and Portkey, which also incorporates guardrails and PII redaction into the gateway layer ([Portkey — AI Gateway](#)).

The gateway belongs in a harness discussion because it centralizes several responsibilities that would otherwise be implemented by each agent. The fallback pattern from §14.3 becomes gateway configuration rather than repeated application code. The cost budgets and per-span attribution from Chapter 17 fit naturally at the gateway because it observes every model call. A stable gateway interface also makes the model-swap discipline from §14.7 easier to apply: operators can canary a new model on a small share of traffic without changing agent code.

This centralization introduces tradeoffs. A gateway becomes a dependency in the hot path, so it needs the circuit breakers and timeouts described in §5.11; if the gateway fails, every agent behind it is affected. Centralization also does not remove the need for a *reliable routing decision* (§14.2). It only gives that decision a common home. Like MCP in §4.3, the gateway is plumbing: it makes a sound routing policy easier to operate, but it cannot turn a poor policy into a good one.

Diagram: The Model-Selection Decision



Route easy steps to smaller models and difficult, checkable steps to reasoning models. When the right route is uncertain, use a cascade and let a verifier decide whether to escalate. Any model change is a regression event for the entire configuration.

Key Takeaways

- **Choose models per step, not per application:** reserve frontier capability for the calls where it changes the result, and route straightforward work to smaller, faster models.
- **Routing decides before execution; cascades decide after an attempt:** a learned router can improve the cost-quality tradeoff ([RouteLLM](#)), while a verifier-gated cascade can match strong-model accuracy at lower cost ([FrugalGPT](#)). Both require a cheap, reliable decision signal.
- **Reasoning models add a separate design axis:** test-time compute exchanges inference tokens for quality on difficult, checkable tasks. This is different from selecting a larger model or adding hardware.

- **Work with the model's training:** do not manually prompt for native reasoning, budget for hidden tokens, do not treat the reasoning trace as verification, and remember that reasoning does not provide grounding.
- **Treat model upgrades as harness events:** re-run evaluations for every swap. Readiness belongs to the model-plus-harness configuration.
- **Use an AI gateway to centralize routing infrastructure:** LiteLLM, Portkey, and similar proxies can host fallback, budgets, caching, and logging behind one interface. They still require reliable routing decisions and their own circuit breakers.

Further Reading

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
- DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *LLM Foundations for Harness Engineering* (companion volume), Chapters 5–8. [../llm-foundations/](https://llm-foundations/)
- LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
- Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>

Chapter 15: Human–Agent Interaction

Most of this book examines the machinery between the model and the world: context, tools, sandboxes, state, and evaluation. Yet whenever an agent performs consequential work, a human is also part of the loop. That person may approve actions, redirect work in progress, review results, and decide how much to trust the output. The interface supporting those decisions is a harness layer in its own right, just like the agent–computer interface described in [Chapter 4](#). It must be deliberately engineered. [Chapter 5](#) introduced one aspect of this problem — permission fatigue. This chapter considers the human interface as a whole.

15.1 The Human Is Inside the Loop

The agent–computer interface chapter argued that how an agent uses tools deserves as much engineering attention as a conventional user interface. The human–agent interface is its mirror: how a person supervises the agent deserves as much attention as how the agent acts. Anthropic's implementation principles point in the same direction: maintain simplicity, *prioritize transparency by showing the agent's planning steps*, and design the interface carefully ([Anthropic — Building Effective Agents](#)). Transparency is not merely a UX refinement. Without it, meaningful supervision is impossible.

This problem predates modern agents. Horvitz's 1999 principles of *mixed-initiative* interfaces framed the core questions: when should a system acting on a user's behalf proceed autonomously, and when should it defer? How should it account for the cost of interrupting the user? How should it retain and use the context of the interaction ([Horvitz — Principles of Mixed-Initiative User Interfaces](#))? Each question becomes more urgent when the system can run a shell or take other consequential actions.

15.2 Two Failure Modes: Permission Fatigue and Blind Trust

Human–agent interfaces tend to fail in two opposite ways, and a sound design must avoid both:

- **Permission fatigue** results from asking too often. If an agent requests approval for every action, the human becomes habituated and clicks "allow" without reading. This can be worse than having no prompt because it creates the *appearance* of oversight without its substance ([Anthropic — Beyond Permission Prompts](#), Ch 5). A sandbox allows low-stakes actions within a defined boundary to proceed without repeated prompts.
- **Blind trust** results from asking too little. Fluent, confident output encourages the human to approve it without scrutiny. When the agent produces a plausible error — such as a fabricated citation or a subtly broken refactor — a review surface that hides the supporting evidence makes that error easier to miss.

These failures sit at opposite ends of the same control. Fewer prompts can encourage blind trust; more prompts can produce fatigue. The solution is not a single global setting but *stake-proportionate* interaction: the amount of human attention required should reflect the action's reversibility and blast radius (§15.3).

15.3 Mixed-Initiative: When to Ask vs Act

The central design question must be answered for each action: should the agent proceed, or ask first? A practical default is to scale human involvement with the possible consequences:

- **Act silently** inside the sandbox for reversible, low-stakes actions such as reading files, running tests, or editing a scratch workspace (Ch 5). Requiring approval here only teaches the human to ignore prompts.
- **Act and report** when an action has consequences but remains observable and reversible. Instead of blocking for approval, leave a clear record that the human can review afterward.
- **Ask first** before irreversible or high-blast-radius actions such as deleting data, sending an external message, spending money, or modifying production. This aligns with the *lethal trifecta* boundary (Ch 5): when an agent is about to combine private data, untrusted input, and external reach, a human should be involved.

This is the capability–control tradeoff revisited in [Chapter 18](#): greater autonomy provides greater capability, but also creates a larger control burden. The interface makes that tradeoff concrete for each action.

15.4 Approval as a Tool Call

HumanLayer's twelve-factor manifesto proposes a clean structural pattern: *contact humans with tool calls*. Rather than treating human approval as a special exception in the control flow, represent it as a tool the agent can invoke — `request_human_approval(action, context)`. The response then returns to the loop like any other observation ([HumanLayer — 12-Factor Agents](#)).

This pattern integrates human interaction with the rest of the architecture. The approval request becomes an event in the event log (Ch 9), making it durable, replayable, and auditable. It also composes with the long-running patterns from [Chapter 7](#): the agent can request approval, suspend, and resume hours later when a human responds because its state resides in the log rather than in an open connection. The human also becomes a first-class participant in the trace ([Chapter 12](#)), rather than an out-of-band interruption.

15.5 Designing the Review Surface

The quality of a human review depends on the information the interface provides. A surface that shows only the agent's conclusion invites blind trust. One that presents the *evidence behind the conclusion* enables an informed decision. For a coding agent, that evidence includes the diff, the tests that ran, and the commands executed — not merely a "done" message. For a research agent, it includes the sources behind the summary.

This review evidence can come from the same span telemetry used for debugging (Ch 12): a useful trace also makes a useful review artifact. Guidance from human–AI interaction research applies directly. The interface should explain what the system can do, show how well it performed, and make incorrect output easy to correct or dismiss ([Amershi et al. — Guidelines for Human-AI Interaction](#)). Human supervision becomes practical when the agent's work is inexpensive to verify and, when necessary, reject.

15.6 Steering and Interruption

Supervision happens during execution as well as before and after it. If a human sees an agent heading down the wrong path, they need to redirect it without terminating the session or losing its state. The harness must therefore support *steering*: injecting a new instruction into a running agent so that the next turn incorporates it.

The mechanism follows from earlier chapters. Because the agent loop rebuilds context on each turn (Ch 1), the harness can add the steering message to the next iteration. Following the recitation logic from [Chapter 3](#), placing the correction near the end of the context keeps it within the model's most reliable attention span. Steering also depends on clean checkpointing (Ch 7). An agent that can pause and resume can accept a correction; one that keeps all state inside a single uninterruptible call cannot.

15.7 Calibrating Trust: Transparency and Uncertainty

The goal of the human interface is *calibrated trust*: the human should trust the agent only to the degree warranted by the current task and evidence. Too much trust allows errors to pass without review; too little creates fatigue and wastes human attention. Two harness mechanisms help achieve this balance:

- **Transparency** — showing planning steps, tool calls, and evidence — helps the human base trust on the agent's work rather than on the fluency of its response ([Anthropic — Building Effective Agents](#)).
- **Uncertainty signaling** — identifying where the result remains uncertain — directs human attention to the places that need it. This mechanism is useful only if its signals are trustworthy. As the companion volume explains, model confidence is often poorly calibrated (*LLM Foundations*, Ch 10). The harness should therefore ground uncertainty in external verification — for example, whether tests passed or a cited source exists — instead of relying on the model's own hedging.

15.8 Supervising Many Agents

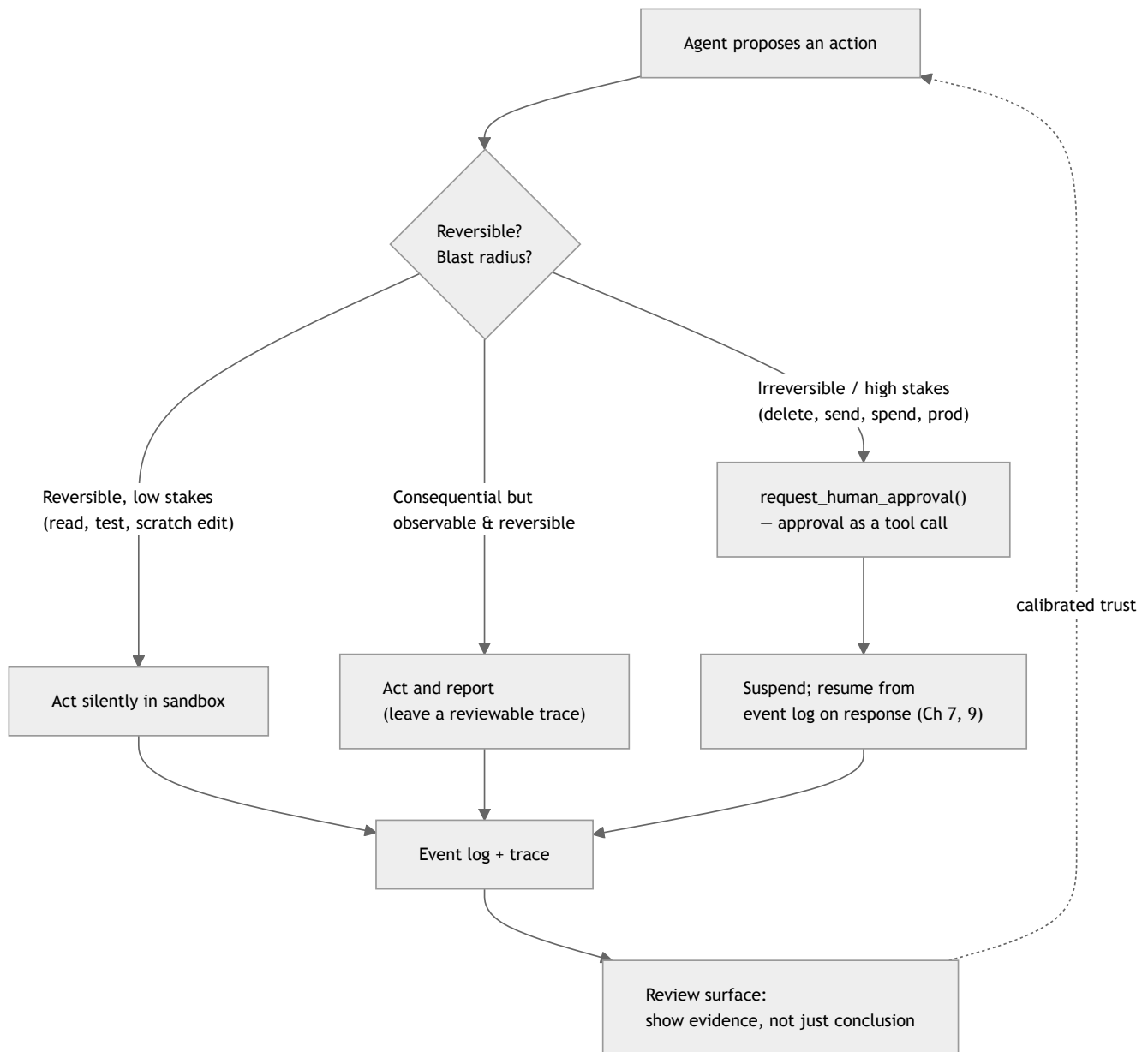
As agents become more common, the human role shifts from performing work to supervising agents that perform it — and eventually to supervising *many* agents at once. This shift changes the interface requirements. A person overseeing ten agents cannot read every trace. The interface must identify what needs attention: agents waiting for approval, results with unresolved uncertainty, and outputs that failed verification.

Code review becomes a bottleneck when agents produce changes faster than humans can inspect them. The review queue should therefore be ordered by **risk and evidence**, not by arrival time or trace length. Failed deterministic checks, high-blast-radius actions, security-sensitive paths, novel tool use, policy-rule hits, weak or conflicting grader evidence, and large unexplained diffs should rise to the top. Low-risk changes backed by strong verification can be summarized or sampled, while consequential changes retain their full artifacts. Custom scoped review rules (§13.8) can make this routing more precise.

At this scale, supervision becomes a fleet-level control-plane concern, as described in [Chapter 18](#). The governing principle is to make supervision both *efficient and meaningful*: efficient enough for one person to oversee many agents, and meaningful enough to prevent oversight from becoming a rubber stamp. Every technique in this chapter

— stake-proportionate prompting, approval as a tool call, evidence-rich review surfaces, steering, and calibrated transparency — serves that goal.

Diagram: Stake-Proportionate Interaction



The interaction model ranges from silent action to mandatory approval. Place each action on that range according to its reversibility and blast radius. Treat approval as a tool call, and show evidence during review to avoid both permission fatigue and blind trust.

Key Takeaways

- **Treat the human interface as a harness layer:** how a person supervises an agent deserves as much engineering attention as how the agent acts.
- **Avoid both failure modes:** asking too often creates permission fatigue and encourages rubber-stamping; asking too little encourages blind trust and allows plausible errors through.
- **Match interaction to the stakes:** act silently for reversible, low-stakes actions; act and report for observable, reversible actions; and ask first for irreversible or high-blast-radius actions at the lethal-trifecta boundary.
- **Represent approval as a tool call:** approval then becomes durable, replayable, auditable, and compatible with long-running suspend-and-resume workflows.
- **Show evidence, not only conclusions:** practical supervision depends on making an agent's work easy to verify and reject, whether one human oversees one agent or an entire fleet.

- **Rank fleet reviews by risk:** surface failed checks, sensitive paths, policy hits, novel actions, and weak evidence before low-risk work backed by strong verification.

Further Reading

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Dex Horthy, *12-Factor Agents* (Factor 7: contact humans with tool calls), HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

Chapter 16: Computer-Use and Multimodal Agents

[Chapter 4](#) built the agent-computer interface from well-defined tools: functions with schemas, MCP servers, and code APIs. A growing class of agents works differently. Instead of calling a clean API, these agents operate software as a person would: they look at a screen, move a cursor, type into fields, and click buttons. Such *computer-use* agents depend on multimodal perception and introduce harness problems that do not arise with API-based tools. They are also where many of today's most ambitious general-purpose agents operate.

16.1 Beyond the API: Agents That Operate Software

The motivation is reach. Most software has no agent-friendly API; it has a graphical interface designed for people. An agent that can see and act on a screen can use software that would otherwise remain inaccessible: legacy desktop applications, websites without APIs, and internal tools that no one is likely to wrap in MCP. Anthropic's computer-use models and OpenAI's Computer-Using Agent follow this approach. The model receives screenshots, reasons about what they show, and emits low-level actions such as "click at (x, y)" or "type this string" ([Anthropic — Computer use](#); [OpenAI — Computer-Using Agent](#)).

This loop differs substantially from the one in [Chapter 1](#). Its observation is an image rather than a tool result. Its action is a UI gesture rather than a function call. Its environment is an entire operating system or browser rather than a curated tool set. Everything this book has said about tools, context, and safety still applies, but now at a lower level and through a visual interface.

16.2 The Environment Is the Tool Surface

In the ACI chapter, the harness engineer *chose* the tools and could offer a small set of actions with clear affordances (Ch 4). A computer-use agent instead inherits its action surface from the environment: every button, menu, and field on the screen is a potential target. The careful tool curation described in [Chapter 4](#) is no longer possible in the same form, because the "tools" are whatever the application exposes visually.

This shift changes a key design lever. The harness can no longer reduce the action space simply by offering fewer tools. Instead, it must help the agent *perceive* the available actions accurately and constrain *where* the agent may act. The design problem moves from tool selection to perception and scoping, the subjects of the next sections.

16.3 Encodings of the Screen

A harness can represent a screen to the model in several ways, and the choice has direct consequences. As the companion volume explains, images ultimately become tokens (*LLM Foundations*, Ch 2). The main screen encodings are:

- **Screenshot pixels** preserve visual layout and styling, but they consume many tokens and may make small text hard to read. The model must locate elements visually.
- **DOM or HTML** (for the web) preserves exact text and document structure, but does not reliably show what the user can actually see. Hidden, off-screen, and visually overlapped elements may be difficult to distinguish.
- **Accessibility tree** provides a structured, semantic view designed for assistive technology. It exposes roles, labels, and states, often in a form much more compact than the raw DOM.
- **OCR** recovers text from pixels, but discards much of the structure and many of the spatial relationships.

No single encoding is complete. A screenshot shows what a person sees but not the underlying structure; a DOM dump exposes structure but not visual salience. Production systems therefore often *combine* encodings—for example, using a screenshot for layout and an accessibility tree or DOM for exact targets. The combination is more robust, but consumes more context. This is the same precision-versus-budget tradeoff found in retrieval (Ch 2), now applied to the visual domain.

16.4 Grounding: From Seeing to Clicking

The hardest problem unique to computer-use agents is *visual grounding*: translating an intention such as "click the Submit button" into a concrete action at specific screen coordinates. A model may correctly decide which button to press and still click the wrong location. API-based tools have no direct equivalent of this failure mode, because selecting a named function is exact.

A common harness technique is *Set-of-Mark prompting*. The harness overlays numbered marks on candidate interactive elements in the screenshot, allowing the model to select a discrete label such as "click element 7" instead of generating raw coordinates ([Yang et al. — Set-of-Mark Prompting](#)). This turns an error-prone continuous grounding

problem into a more reliable discrete choice, but requires the harness to detect and label candidate elements first. The related SeeAct line of work showed that even strong vision models need this kind of grounding scaffold to act reliably on real web pages. Perception and action are separate capabilities, and the harness must bridge the gap between them (Zheng et al. — [GPT-4V is a Generalist Web Agent](#)).

16.5 The Action Space and Its Failure Modes

Computer-use actions are lower-level and more state-dependent than function calls. A click succeeds only if the screen is still in the state the model expects. A slowly loading page, an unexpected modal, or a shifting layout can invalidate an action that the model has already chosen. The loop must therefore re-observe the screen after every action and treat the new screen as ground truth. It must also be designed for the possibility that observations and actions will fall out of sync.

This makes self-verification, the central lever of [Chapter 7](#), even more important. After each action, the agent should confirm that the screen changed as expected before continuing. Recovery is also harder: reversing a GUI action is rarely as clean as undoing an API call. The stake-proportionate interaction described in [Chapter 15](#) therefore matters even more. A computer-use agent about to confirm an irreversible dialog is exactly where a human checkpoint belongs.

16.6 Latency, Cost, and the Token Weight of Pixels

Each turn of a computer-use loop sends an image into context, and images are expensive in tokens. A single high-resolution screenshot can cost as much as a page of text (*LLM Foundations*, Ch 2). A task that requires thirty clicks may therefore send thirty screenshots through the context window. This pattern quickly consumes the finite-context budget described in [Chapter 2](#) and compounds the prefill-heavy nature of agent workloads.

The familiar harness levers still apply, but they require careful tuning. Downscale screenshots as much as the task allows, but not so far that the text or controls the agent needs become unreadable. Once the useful information has been extracted, remove old screenshots from context and retain a compact note of what the prior screen showed instead. The *keep the wrong stuff out* discipline of [Chapter 3](#) is especially important for images. When a structured representation such as an accessibility tree is sufficient, prefer it to pixels and reserve screenshots for cases in which layout genuinely matters.

16.7 Safety: The Widest Attack Surface

A computer-use agent concentrates all three parts of the *lethal trifecta* (Ch 5). It reads untrusted content from any webpage or document on the screen. It may have access to private data through the browser or filesystem. And it can communicate externally by navigating, submitting forms, or sending messages. Here, prompt injection is not merely malicious text hidden in a tool result. It can be an instruction rendered directly on a webpage the agent is viewing, which the model may mistake for a legitimate command.

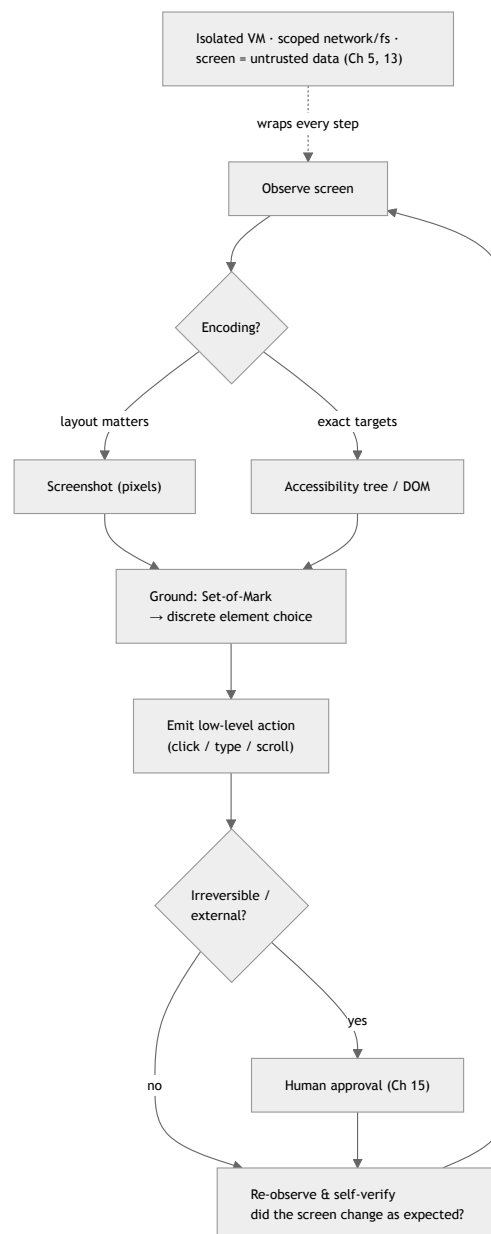
The defenses are the sandboxing and governance controls from [Chapter 5](#), applied more strictly because the action surface is so broad. Run the agent in an isolated environment—a dedicated VM or container, not the user's primary session. Limit its network and filesystem access. Require human approval before irreversible or external actions (Ch 15). Treat everything displayed on the screen as untrusted data, never as an instruction. The instruction hierarchy from [Chapter 13](#) becomes essential when the "data" is an entire rendered webpage.

16.8 Evaluating Computer-Use Agents

Because the environment is an entire operating system or browser, evaluation must measure environmental outcomes rather than text alone. The *outcome* distinction from [Chapter 10](#) is unavoidable: what matters is whether the agent actually completed the task in the environment. Purpose-built benchmarks illustrate this approach. WebArena provides a self-hostable, realistic web environment with execution-based success checks ([WebArena](#)), while OSWorld extends the idea to open-ended tasks in a real desktop operating system ([OSWorld](#)).

These benchmarks reinforce a theme that runs throughout this book: computer-use agents remain far from saturated on realistic tasks. The gap between "the model can perceive the screen" and "the agent reliably completes the task" is the harness gap—grounding scaffolds, re-observation, recovery, scoping, and verification. As always, the right eval is the workload eval (Ch 10). A public computer-use benchmark provides background evidence; reliable operation of *your* application is the evidence that should determine release readiness.

Diagram: The Computer-Use Loop



Perceive → ground → act → re-observe, wrapped in a sandbox. Grounding turns "see" into "click"; re-observation handles a stateful screen; the sandbox contains the widest attack surface in the book.

Key Takeaways

- **Computer-use agents trade clean APIs for reach:** they operate GUIs as people do, giving them access to software without agent APIs, but making the loop harder to control.
- **The environment becomes the action surface:** tool curation gives way to perception and scoping; the harness must help the agent see available actions accurately and constrain where it may act.
- **Screen encoding is a harness choice:** pixels, DOM, accessibility trees, and OCR preserve different information; combining them improves robustness but consumes more context.
- **Grounding is the distinctive failure mode:** translating intent into the correct click is error-prone; discrete techniques such as Set-of-Mark are more reliable than raw coordinates.
- **The widest attack surface requires the strictest sandbox:** a computer-use agent concentrates the lethal trifecta, so isolate it, limit its scope, gate irreversible actions, and treat the entire screen as untrusted data.

Further Reading

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024.
<https://www.anthropic.com/news/3-5-models-and-computer-use>

- OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
- Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
- Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
- Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
- Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>

Chapter 17: AgentOps — Cost, Privacy, and Production Operations

The chapters so far have built an agent that works. Running that agent in production for real users introduces three concerns that the loop itself tends to hide: what each run costs, how it handles *sensitive data*, and how to *change the harness safely* once people depend on it. These concerns are easy to overlook, but they often separate a working demonstration from a reliable product. [Chapter 5](#) covered the security threat model, and [Chapter 11](#) covered measurement noise. This chapter addresses the operational economics, data governance, and release discipline that surround both.

The emerging name for this discipline is **AgentOps**. AWS organizes it into four connected pillars: **governance and security**, **build and operations**, **evaluation**, and **observability** ([AWS — AgentOps](#)). The term is useful because it covers the full lifecycle. An agent is planned, developed, built, tested, deployed, maintained, monitored, and eventually retired. Production ownership spans that entire path; it does not begin at the monitoring dashboard.

AgentOps also changes what counts as a release. The deployable artifact is not merely model code or a prompt. It is a versioned bundle containing the model and inference settings, system and project instructions, tool catalog and schemas, memory policy, sandbox image, identity and authorization policy, graders, budgets, and routing rules. The release record should identify this complete configuration so that engineers can attribute an incident to a specific version and roll it back. Chapter 18 moves up one level: the control plane registers these artifacts and governs their identities and fleet lifecycle, while AgentOps focuses on operating and improving them.

17.1 Cost: Budgeting the Agent

An agent is the most expensive way to run a model because a single user request can expand into many model calls, tool round-trips, and—with reasoning models—long internal token streams ([Chapter 14](#)). The total cost is also difficult to predict: an open-ended loop may take three turns or thirty. This is the operational reason the book keeps returning to "do the simplest thing that works" ([Chapter 6](#)). Every additional turn incurs another charge.

A production agent therefore needs an explicit *budget*, just as the long-running agents in [Chapter 7](#) need checkpoints. In practice, this means:

- **A per-task ceiling** on tokens, tool calls, or wall-clock cost. When the agent reaches that ceiling, it stops and asks for direction instead of looping indefinitely. An unbounded loop can become a runaway bill.
- **The cost levers established earlier**: route easy steps to cheaper models and reserve frontier and reasoning models for steps that need them (Ch 14); cache stable prefixes so repeated context is not paid for on every turn (Ch 2); return token-efficient tool responses and use code execution instead of placing large results directly in context (Ch 4).
- **A build-or-not decision**: the cheapest agent is the one you do not run. Many tasks are better served by a workflow or a single model call, and cost should be part of that design choice (Ch 6).

The cascade and routing patterns in [Chapter 14](#) are cost controls as much as quality controls. FrugalGPT's central result is that most of the spending produced by a naive "always call the best model" strategy can be avoided without sacrificing accuracy ([FrugalGPT](#)).

Caching deserves closer attention because two distinct mechanisms are involved. The KV/prefix cache from [Chapter 2](#) serves *identical* prefixes at lower cost. A *semantic cache*, by contrast, serves *similar* requests: it embeds an incoming query and returns a stored response when a previous query is sufficiently close, an approach popularized by GPTCache ([Fu Bang — GPTCache](#)). For high-volume, repetitive queries, a semantic cache can eliminate entire model calls rather than merely reduce their cost.

That benefit introduces a correctness risk that prefix caching does not. A false semantic-cache hit may return a subtly incorrect answer to a question that only *resembled* a cached one. A semantic cache must therefore be tuned: its similarity threshold trades savings against the risk of false matches. It should be evaluated with the same discipline as any other harness change (§17.5). Both cache types, along with the per-key budgets described above, can be enforced naturally at the *AI gateway* (§14.8). Because every call passes through the gateway, it provides one place to apply cost controls consistently across agents.

17.2 Cost Attribution and the Trace

A budget is enforceable only when cost is *measured*, and the trace is the natural place to measure it. The span telemetry from [Chapter 12](#)—a tree of spans for model calls, tool calls, and retrieval—can also serve as a cost ledger. Attach token counts and dollar costs to each span, and the trace shows both what the agent did and what each step

cost. This turns a vague complaint such as "the agent is expensive" into a concrete observation such as "this tool returns 8,000-token results on every call," which engineers can act on.

Cost attribution belongs in the same iteration loop as failure analysis. A step that is expensive but contributes little value is a candidate for a cheaper model, a more concise tool response, or removal. This is the trace-driven process from [Chapter 12](#), applied to cost rather than correctness. As [Chapter 11](#) warned, cost and resource configuration can also affect measured behavior. Cost should therefore be reported alongside capability, not as a separate concern.

17.3 Privacy and Data Governance

An agent handles data on every run. It reads files, queries databases, retrieves documents, and sends requests to model providers and tools. Each step creates an opportunity for sensitive data to leak, and the harness is responsible for controlling those paths. The threat is most acute in the *lethal trifecta* described in [Chapter 5](#): an agent that can access private data, consume untrusted content, and communicate externally can be induced to exfiltrate that data ([Simon Willison — The lethal trifecta](#)). Privacy is therefore not separate from security; it is the data-handling side of the same problem.

The relevant harness-level controls are:

- **Minimize what enters context.** Data the agent never sees cannot leak. Retrieve the slice needed for the task, not the whole record (Ch 2), and prefer handles over raw sensitive content.
- **Redact at the boundary.** Remove secrets and PII before content enters the model context or a trace. Traces are durable and are often sent to third-party observability tools, so an unredacted trace can itself become a data leak (Ch 12).
- **Enforce permissions during retrieval.** Filter data according to what the *user* is allowed to see before retrieval, not after generation. An agent must not provide access to data that its user could not otherwise read (Ch 5, [Ch 12](#)).
- **Govern egress.** The harness can break the external-communication leg of the trifecta. Restrict where the agent may send data, and require approval before external transmission (Ch 15).

17.4 Multi-Tenancy and Isolation

When one agent platform serves multiple users or organizations, isolation becomes a correctness requirement rather than an optional safeguard. Two failure modes are especially important. *State bleed* occurs when one tenant's context, memory, or cached prefix appears in another tenant's session. This failure is catastrophic and easy to introduce when memory (Ch 3) or prefix caching (Ch 2) is added without tenant scoping. *Authority bleed* occurs when an agent uses one tenant's credentials on behalf of another. It is the multi-tenant form of the delegated-authorization problem discussed in [Chapter 5](#).

The required discipline follows the managed-agent architecture from [Chapter 7](#): use durable per-tenant workspaces, scope each sandbox to one tenant's resources, and issue narrowly scoped credentials that are never shared across the brain/hands boundary. The event log (Ch 9) should include tenant identity so that every action can be attributed correctly. That attribution is also what makes auditing (Ch 5) meaningful in a shared system.

17.5 Releasing Harness Changes

A harness is software, and changing software that people depend on requires release discipline. Harness changes are unusually risky, however, because their effects are statistical rather than deterministic. A prompt edit, a new tool, a model swap, or a retrieval adjustment can alter behavior on inputs that no one tested. The resulting regression may affect only a small percentage of cases instead of causing an obvious crash ([Chapter 10](#), [Chapter 12](#)).

That risk calls for three release practices:

- **Gate every change on evals.** Run the suite before and after the change, comparing pass rate, failure categories, cost, and latency. The regression discipline from [Chapter 10](#) becomes the release gate.
- **Roll out gradually.** Eval suites never cover the full input distribution, so treat a harness change like any other risky deployment. Release it to a fraction of traffic, monitor production traces and outcome metrics, and remain ready to roll back. A canary catches cases that the eval suite missed while limiting their impact.
- **Version the entire configuration together.** Because the model and harness are coupled, as described in [Chapter 12](#), the unit of release is the complete *configuration*—model, prompts, tools, and retrieval policy—not any single component. Readiness validation (Ch 10) applies to that configuration, and a rollback must restore the full configuration rather than only the prompt.

17.6 Monitoring, SLOs, and Incident Response

Evals run before release; monitoring continues afterward. The two provide different evidence: evals measure a curated distribution, while production reveals the real one. The span telemetry from [Chapter 12](#) provides the monitoring substrate. On top of it, an agent system needs the familiar components of production monitoring, adapted to agent-specific behavior:

- **Measure outcomes, not just uptime.** A liveness check showing that the API responds says nothing about whether tasks succeed. Track task success rate, escalation and approval rates, cost per task, and latency percentiles (p50/p95/p99, as in Ch 6) as first-class signals.
- **Alert on drift.** A rising failure rate, increasing cost per task, or declining self-verification pass rate can constitute an incident even when nothing has crashed. These are characteristic failure patterns for agents.
- **Turn production failures into regression cases.** The trace-to-eval loop from [Chapter 12](#) is also an incident-response loop. Redact each production failure, convert it into a reproducible eval case, and fix the underlying issue so that the same incident cannot recur silently.

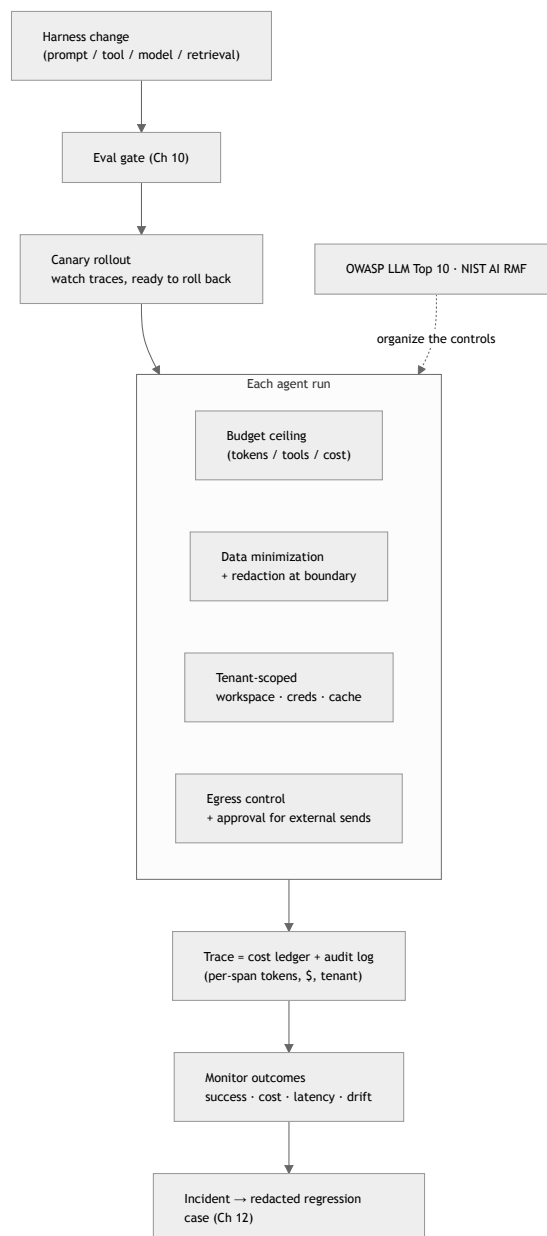
17.7 Governance Frameworks

These concerns are not only matters of internal engineering. For many deployments, they are also external requirements. Four reference frameworks can help structure the work, ranging from voluntary guidance to binding law.

- **OWASP Top 10 for LLM Applications** lists common risk categories—including prompt injection, sensitive-information disclosure, excessive agency, and supply-chain risks—and maps closely to the controls in [Chapter 5](#) and this chapter ([OWASP — Top 10 for LLM Applications](#)).
- **NIST AI Risk Management Framework** provides a higher-level structure—govern, map, measure, and manage—for organizations that need an auditable risk process around an agent deployment ([NIST — AI Risk Management Framework](#)).
- **ISO/IEC 42001:2023**, the first AI management-system standard, is the AI analogue of ISO 27001. It specifies how an organization establishes, operates, and continually improves a management system for AI, and organizations can certify against it ([ISO/IEC 42001:2023](#)).
- **The EU AI Act** (Regulation (EU) 2024/1689) is the first comprehensive AI law. It classifies systems by risk tier and imposes binding obligations—including risk management, data governance, transparency, human oversight, and logging—on high-risk uses, with provisions phasing in through 2026–2027 ([EU AI Act — Regulation \(EU\) 2024/1689](#)). The Act and ISO 42001 are complementary but distinct: ISO 42001 certification provides evidence of a sound process, not automatic proof of compliance with the AI Act.

None of these frameworks replaces the engineering described in this book. Instead, they organize that work and make it understandable to auditors, customers, and regulators. The [Outlook](#) identifies cross-layer governance coherence as an open problem because policy, audit, and runtime enforcement still live in separate layers. These frameworks provide the best current scaffolding for keeping those layers aligned.

Diagram: The Production Wrapper



Cost, privacy, and tenancy are enforced per run; the trace is both the cost ledger and the audit log; every harness change passes an eval gate and a canary before reaching production.

Key Takeaways

- **Agents need explicit budgets:** cap tokens, tool calls, or cost per task, and use routing, cascades, caching, and token-efficient tools to prevent an open-ended loop from becoming a runaway bill ([FrugalGPT](#)).
- **The trace is the cost ledger:** attach token counts and dollar costs to each span so that "the agent is expensive" becomes a specific engineering problem that can be fixed.
- **Privacy is the data-handling side of the lethal trifecta:** minimize what enters context, redact at the boundary—including traces—enforce user permissions during retrieval, and control egress.
- **Multi-tenancy requires isolation:** scope workspaces, credentials, memory, and prefix caches per tenant, and include tenant identity in the event log. State bleed and authority bleed are catastrophic failures.
- **Two caches have two risk profiles:** a prefix cache serves identical context cheaply; a semantic cache serves *similar* queries and can skip entire calls, but a false hit returns the wrong answer. Tune its threshold and evaluate it like any other harness change.
- **Releasing a harness change is a statistical deployment:** gate changes on evals, canary them gradually, version the entire configuration together, and turn production failures into regression cases.

- **AgentOps covers the whole lifecycle:** governance and security, build and operations, evaluation, and observability apply from planning through retirement. Release the model, tools, memory, policy, sandbox, graders, and budgets as one versioned configuration.
- **Governance frameworks range from guidance to law:** the OWASP LLM Top 10 and NIST AI RMF organize controls; ISO/IEC 42001 certifies the management process; and the EU AI Act imposes binding obligations on high-risk uses.

Further Reading

- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
- NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
- *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
- *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>

Chapter 18: Agent Fleets, Identity, and the Control Plane

Loop engineering asks how one autonomous task should start, act, verify, and stop. Production systems now face a larger unit of design: many agents, built by many teams, operating across multiple environments and organizations. The hard questions move from the individual loop to the fleet. Which agents exist? Who published this version? On whose behalf is it acting? Which tools, data, money, and network destinations may it reach? How can an operator suspend, upgrade, or revoke a running agent? What evidence connects an action to the exact identity, policy, model, and artifact that produced it?

The emerging answer is an **agent control plane**. The concept borrows a distinction from distributed systems. The *data plane* performs the work: model calls, tool calls, code execution, and agent-to-agent messages. The *control plane* declares and enforces the operating state around that work, including registry, identity, authorization, routing, lifecycle, policy, lineage, and audit. It does not make an individual agent smarter; it makes a fleet governable.

18.1 From Agent Program to Agent Fleet

A single agent can be configured with a prompt, a few tools, and a local state file. At fleet scale, local configuration can no longer answer every operational question. New problems appear:

- duplicate or abandoned agents with unclear ownership;
- several versions sharing a name but not behavior;
- credentials copied into prompts, sandboxes, or environment variables;
- tool access that outlives the task or user who authorized it;
- policy applied differently across chat, scheduled runs, and A2A delegation;
- incidents where no one can reconstruct which artifact or authority caused an action.

The platform shift in §9.2 anticipated this broader scope. A fleet needs a source of truth for agents and capabilities, a runtime authority that can issue and revoke task-scoped access, and lifecycle services that reconcile what *should* be running with what *is* running. AgentOps (Ch 17) keeps the system running and improves it throughout its lifecycle. The control plane provides the shared governance layer on which those operations depend.

The boundary should remain explicit:

- **Data plane:** inference, retrieval, tool execution, sandbox processes, messages, and results.
- **Control plane:** definitions, identities, policy decisions, placement, versions, budgets, revocation, and audit.

The central design move is to keep policy decisions out of the probabilistic parts of the data path wherever possible. The agent may propose an action; deterministic control-plane services decide whether the named identity may perform that operation in the current context.

18.2 Agent Identity: Who—or What—Is Acting?

User identity alone is not enough. One user may launch several agents with different purposes and privileges. An agent may continue on a schedule after the initiating session ends, delegate work to another agent, or run alongside other versions of itself. **Agent identity** is the durable machine identity of the agentic principal. It is distinct from the human, service account, runtime process, and model involved in a particular run.

NIST's concept paper on software-agent identity and authority breaks the problem into identification, authentication, authorization, delegation, audit, and non-repudiation. It highlights prompt injection and confused-deputy behavior as reasons that ordinary service identity is insufficient ([NIST — Identity and Authorization for Software Agents](#)). In practice, an identity record should answer:

- **Principal:** which agent definition and version is acting?
- **Sponsor:** which person or organization owns it?
- **Delegator:** on whose behalf is this particular run acting?
- **Purpose:** what bounded task or workflow justified the authority?
- **Runtime:** which harness, model, sandbox image, and policy versions are executing?
- **Lifetime:** when was the identity issued, and when does it expire or become revocable?

Do not represent this entire chain with one long-lived API key. A fleet should use workload identity and short-lived task tokens, together with explicit delegation chains. The agent authenticates as itself. The authorization service then combines that identity with the delegating user, tenant, purpose, environment, and requested operation. This separation prevents a low-risk research agent from silently inheriting all the authority of the human who launched it.

18.3 The Agent Registry

An **agent registry** is the fleet's inventory and discovery layer. It tells humans, orchestrators, gateways, and other agents what exists and which artifact they are about to trust. Google Cloud's Agent Registry models agents alongside MCP servers, endpoints, skills, skill revisions, and publishers. AWS Agent Registry and Microsoft 365's Agent Registry reflect the same shift from scattered deployment URLs to governed catalogs ([Google Cloud — Agent Registry Overview](#); [AWS — Agent Registry in AgentCore](#); [Microsoft — Agent Registry](#)).

A useful registry entry contains more than a name and description:

- immutable agent and version identifiers;
- publisher and accountable owner;
- supported tasks, input/output contracts, and endpoints;
- tool, MCP, skill, memory, model, and sandbox dependencies;
- requested permissions, data classes, network destinations, and budget class;
- eval and security attestation for the version;
- deployment status, deprecation date, and revocation state;
- lineage to the source, build, configuration, and policy bundle.

Discovery and governance must meet at the registry. Search results should be filtered to the agents the caller may invoke, and orchestration should resolve an immutable version rather than a mutable display name. Registration is not the same as approval: a new entry may be visible as a draft without being authorized for production traffic. Promotion should require ownership, eval, security, and policy gates. Before a version is deprecated or removed, the registry should identify everything that depends on it.

18.4 Runtime Authorization and Delegation

Registration says what an agent *is*. Authorization decides what a particular run *may do now*. The authority granted to one run should be narrower than the union of everything the agent might ever need.

A robust flow is:

1. A user or service starts a run with a stated purpose and tenant.
2. The control plane resolves an approved agent version.
3. Policy computes the permitted scope from the intersection of the agent's capabilities, the delegator's authority, the task's needs, the environment, and the risk.
4. An identity broker issues short-lived, audience-bound credentials or capability tokens.
5. Proxies attach credentials at the tool boundary; raw secrets do not enter the model context or sandbox.
6. Every use records the identity, delegation chain, policy version, operation, resource, and result.
7. The lease is revoked when the run completes or times out, or when a policy change or incident requires it.

AWS AgentCore Identity illustrates how this pattern appears in a product: agent identities, credential providers, and delegated access to external resources without embedding user credentials in agent code ([AWS — AgentCore Identity](#)). The underlying architecture is vendor-neutral: issue authority just in time and present it only at the boundary where it is needed.

A2A delegation should make the chain of authority explicit. Agent A should not send Agent B a bearer token containing all of A's privileges. Instead, it should request a narrower token that names B as the audience, identifies the delegated task as the purpose, specifies the permitted operation and resource, and expires quickly. If B delegates again, the new authority cannot exceed the authority carried through the existing chain. This blocks **trust escalation** (Ch 3): a low-trust worker cannot persuade a privileged parent to turn its result into a high-privilege action without a new policy decision.

18.5 The Agent Gateway and Policy Enforcement

The **agent gateway** is the enforcement point on the data-plane path. It can broker model calls, MCP and API tool calls, A2A messages, network egress, and sometimes memory access. Instructions such as "never send secrets" are probabilistic. A gateway can instead enforce the rule deterministically by denying an unauthorized destination, stripping a credential, enforcing a budget, or requiring human approval.

Each request should arrive with a signed or otherwise verifiable execution envelope:

agent identity + version
delegating principal + tenant
task purpose + run/session id
requested action + resource
policy and configuration versions
budget and expiry
trace/span correlation id

Policy can return more than a binary allow-or-deny decision. Useful outcomes include **allow**, **deny**, **allow with redaction**, **allow in a stronger sandbox**, **require human approval**, and **allow with a lower budget or read-only scope**. Both the decision and its inputs become part of the trace. This makes the stake-proportionate controls of Chapter 15 and the containment matrix of Chapter 5 consistent across the fleet, rather than leaving each team to reimplement them as conventions.

Gateway placement matters. A central gateway provides uniform enforcement and visibility, but it can become a latency bottleneck and a single point of failure. Local sidecars reduce latency and can survive partial disconnection, but they make policy distribution and evidence collection harder. Mature designs often separate centralized policy administration from distributed enforcement. They use signed policy bundles, fail-closed rules for consequential actions, and carefully limited fail-open behavior for low-risk reads only.

18.6 Fleet Lifecycle: Reconcile, Suspend, Upgrade, Revoke

An agent needs more lifecycle states than deployed and stopped. A useful fleet state machine includes **draft**, **evaluated**, **approved**, **deployed**, **suspended**, **deprecated**, **revoked**, and **retired**. Runs have a separate state machine—queued, active, waiting for approval, checkpointed, completed, failed, or canceled—and sandboxes have another (Ch 7). The control plane connects these lifecycles without treating them as the same thing.

The governing pattern is reconciliation: compare the declared state with the observed state, then take deterministic action to close the gap.

- If an agent version is revoked, block new runs, revoke credentials, and suspend or terminate affected in-flight runs according to risk.
- If a policy changes, identify which sessions require re-authorization rather than letting old authority persist indefinitely.
- If a version is upgraded, canary it against a bounded traffic slice while retaining the old version for rollback (Ch 17).
- If an owner leaves or a publisher is compromised, traverse the registry's dependency graph to find the agents, skills, MCP servers, and schedules that inherit that trust.
- If a run loses its connection, preserve the durable session and reacquire a sandbox or worker rather than duplicating the task (Ch 7).

Fleet kill switches belong at this layer. They should revoke capabilities at gateways and identity brokers, not merely send "stop" to a model. Scope matters: operators need to stop one run, one version, one publisher, one tenant, one tool dependency, or the entire fleet without reaching for the broadest switch in every incident.

18.7 Lineage, Audit, and Non-Repudiation

Ordinary logs answer, "What request reached this service?" An agent audit must answer a longer causal question:

Which user or service delegated which purpose to which agent version, running under which model, harness, tools, memory, sandbox, and policy; which evidence led to which decision; which authority allowed the side effect; and who approved or overrode it?

The answer takes the form of a lineage graph. It connects registry artifacts, build attestations, identities, delegation events, session and run IDs, trace spans, policy decisions, human approvals, tool results, memory writes, and final outcomes in the environment. Stable content hashes or immutable version identifiers prevent a later edit from rewriting the history of what actually ran.

Use the term **non-repudiation** carefully. A signed event can show that a particular workload identity produced a request and that a policy service authorized it. It cannot prove that a human understood an approval dialog or that a model's natural-language rationale is truthful. A strong audit therefore combines cryptographic integrity with operational evidence: append-only logs, trusted timestamps, credential and policy versions, outcome checks,

approval identity, and retention rules. Store enough to investigate, but apply the privacy discipline of Chapter 17. An audit system that indiscriminately preserves prompts, secrets, and personal data creates a second security problem. Lineage also makes evaluation and improvement safer. A production correction can be traced to the exact artifact that failed, converted into a redacted regression case, and used to promote a new version without mutating the old one (Ch 12). The control plane preserves the chain of custody; the eval gate supplies the evidence for promotion.

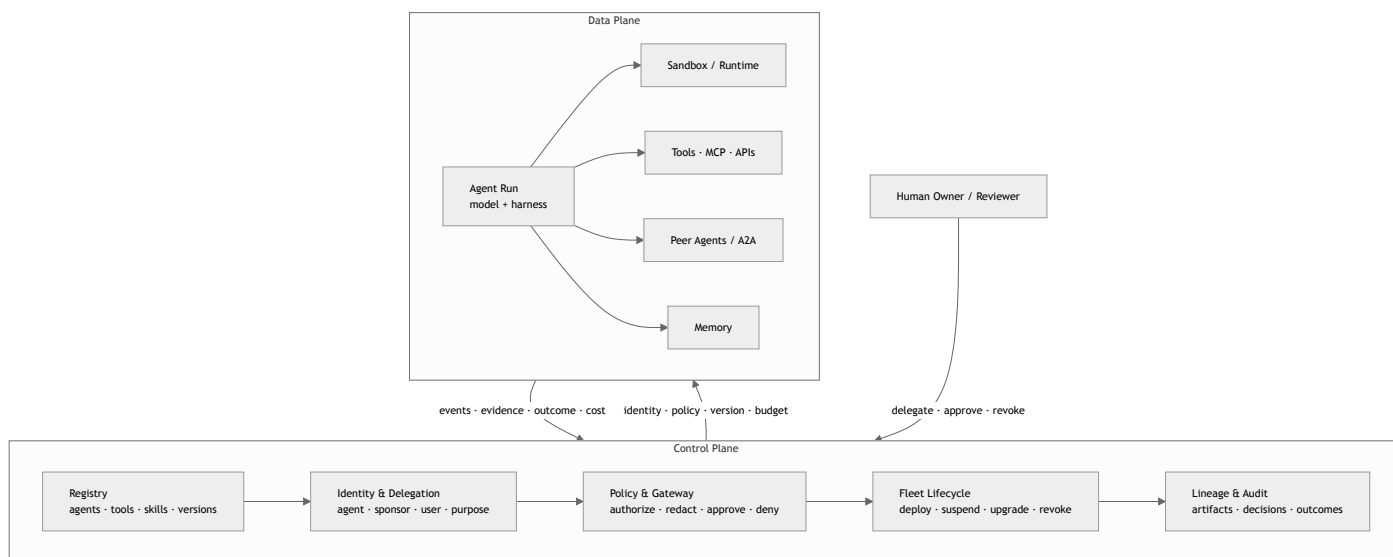
18.8 Open Problems in the Control Plane

The control plane is still emerging. Its most important open problems include:

- **Portable identity and delegation.** Agent Cards and registries describe capability, but cross-platform standards for principal identity, delegation chains, revocation, and purpose-bound authority remain immature.
- **Policy composition.** User policy, tenant policy, agent policy, tool policy, data-residency rules, and human approvals can conflict. Their precedence and explanations need portable semantics.
- **Memory governance.** Provenance, permission-at-retrieval, expiry, correction, and poisoning defense are not consistently represented across memory products.
- **Cross-organizational lineage.** A2A chains cross audit domains; each organization may expose too little evidence for the other to assess trust, or too much private trace data.
- **Evaluator authority.** A model judge can block a deployment or trigger an incident. Its identity, calibration, conflicts of interest, and appeal path need the same rigor as those of other privileged decision-makers (Ch 10).
- **Fleet-level supervision.** Risk ranking, alert deduplication, and meaningful human control must scale without turning the operator into a rubber stamp (Ch 15).
- **Control-plane compromise.** Centralized registries, policy services, and identity brokers are high-value targets. Their blast radius demands separation of duties, signed artifacts, minimal trust roots, and independent recovery paths.

The standards remain unsettled, but the direction is clear: agents are becoming principals in distributed systems. A mature harness is no longer just a loop around a model. It is a governed relationship among identities, artifacts, policies, environments, evidence, and people.

Diagram: The Agent Control Plane



The data plane does the work; the control plane decides which identity, artifact, authority, lifecycle, and evidence govern that work.

Key Takeaways

- **The fleet is the next unit beyond the loop:** loop engineering governs one task; a control plane governs many agents, versions, environments, and organizations.
- **Separate data plane from control plane:** agents propose and execute work; deterministic services register artifacts, issue authority, enforce policy, reconcile lifecycle, and preserve evidence.

- **Agent identity is distinct from user and runtime identity:** bind every run to an agent version, sponsor, delegator, purpose, environment, and expiry.
- **A registry is a governed source of truth:** it should include ownership, dependencies, permissions, attestations, deployment state, and immutable lineage—not only discovery metadata.
- **Authority should be just-in-time and purpose-bound:** use short-lived, audience-scoped credentials attached at the tool boundary, with delegation that can only narrow.
- **The gateway turns policy into enforcement:** allow, deny, redact, strengthen isolation, reduce scope, or require approval—and record why.
- **Lifecycle management is reconciliation:** manage agent definitions, runs, and sandboxes independently; make suspension, migration, canary deployment, rollback, and revocation first-class operations.
- **Audit is a causal lineage graph:** connect user delegation, agent and policy versions, evidence, authorization, side effects, and outcomes while minimizing sensitive content.

Further Reading

- NIST, *Identity and Authorization for Software Agents*, Feb 2026. <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>
- Google Cloud, *Agent Registry Overview*, 2026. <https://docs.cloud.google.com/agent-registry/overview>
- AWS, *AWS Agent Registry in Amazon Bedrock AgentCore (Preview)*, Apr 2026. <https://aws.amazon.com/about-aws/whats-new/2026/04/aws-agent-registry-in-agentcore-preview/>
- AWS, *Introducing Amazon Bedrock AgentCore Identity: Securing Agentic AI at Scale*, 2026. <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/>
- Microsoft, *Agent Registry in the Microsoft 365 Admin Center*, 2026. <https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-registry?view=o365-worldwide>
- Anthropic Safeguards Research Team, *How We Contain Claude*, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
- Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>

Chapter 19: Outlook

19.1 The Field Is Young

Much of the vocabulary used in this book—initializer agents, context firewalls, sprint contracts, reasoning sandwiches, ambient affordances, and computational versus inferential controls—entered the mainstream agent-engineering conversation within the last twelve to eighteen months. The underlying ideas are sometimes older, but the shared language is recent. Most of the source articles for this textbook were published in 2025 and 2026.

The pace is measurable. METR finds that the task-completion *time horizon* of frontier models—the length of a human task they complete successfully 50% of the time—has doubled roughly every seven months (§7.11) ([Kwa et al. — Measuring AI Ability to Complete Long Tasks](#)). Any book about what a harness must provide is therefore describing a moving target.

LangChain frames the trajectory plainly: as models improve, some responsibilities that live in the harness today will be absorbed into the model. Models will become better at planning, self-verification, and long-horizon coherence, reducing the need to inject those capabilities through context. Yet the space of useful harness designs does not disappear as models improve; it shifts toward new problems ([LangChain — The Anatomy of an Agent Harness](#); [Anthropic — Harness Design for Long-Running Application Development](#)).

19.2 Open Problems

Several open problems recur across the literature:

- **Behavioral harnesses for functional correctness.** Maintainability and architectural fitness have benefited from decades of tooling. Behavioral harnesses that answer a more basic question—does the application actually do what the user wants?—have not. Most teams currently rely on AI-generated tests, and the consensus is that these are not yet sufficient ([Thoughtworks — Harness Engineering](#)).
- **Harness coherence at scale.** As guides and sensors multiply, how do they remain consistent? When a sensor never fires, how do we distinguish a high-quality system from inadequate detection? Harness coverage has no equivalent of code coverage or mutation testing yet.
- **Control-plane portability.** Registries, identities, policies, gateways, and audit systems now exist, but their schemas and authority models remain platform-specific. It is still difficult to move an agent between clouds or organizations without losing provenance, policy meaning, or revocation semantics (Ch 18).
- **Cross-layer governance coherence.** The OpenReview survey notes that policies, permission prompts, audit logs, constitutional instructions, and runtime hooks often live in separate layers. Instead of composing cleanly, those layers may interfere with one another. Portable policy and audit languages are still missing ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Standardized readiness reporting.** Model scores depend on the execution environment, tool surface, context policy, retry rules, and governance gates. Benchmark reports therefore need a harness bill of materials, but no widely adopted format exists for publishing that configuration ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Cost-quality-speed trilemma.** Stronger sandboxes, richer observability, deeper verification, and stricter governance usually add cost and latency. Mature harnesses need explicit policies for which checks run synchronously, which run offline, and which risks justify more expensive controls ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Capability-control tradeoff.** More tools, memory, autonomy, and network access increase task coverage. They also increase tool-selection errors, exposure to prompt injection, provenance risk, and audit burden. Capability and control belong on one design axis; they are not separate concerns.
- **Multi-agent coordination beyond synchronous orchestration.** Anthropic's research system runs sub-agents synchronously. Asynchronous coordination could unlock more parallelism, but it also makes result coordination, state consistency, and error propagation harder ([Anthropic — How We Built Our Multi-Agent Research System](#)).
- **Continual learning at the harness level.** Agents often begin each session without the knowledge gained in earlier ones. Memory primitives that let them accumulate knowledge of a codebase or domain over many sessions remain an active research area ([LangChain — Improving Deep Agents with Harness Engineering](#)).
- **Human-in-the-loop approval.** There is no settled interface for deciding when a harness should pause for human judgment or how much context to show without overwhelming the operator. Gates that are too coarse tend to be rubber-stamped; gates that are too fine-grained defeat the purpose of autonomy.

- **Just-in-time tool assembly.** Rather than preconfiguring every possible capability, a harness could assemble the tools and context needed for the current task. LangChain and others are exploring this approach ([LangChain — Improving Deep Agents with Harness Engineering](#)).
- **Tracing as documentation.** LangChain's observation that "in software, the code documents the app; in AI, the traces do" points toward a different model of system documentation, one the field has not yet fully developed ([LangChain — The Anatomy of an Agent Harness](#)).
- **End-to-end supply-chain governance.** Tool integrity is only one part of the problem. Agents also depend on MCP servers, external packages, datasets, retrieval sources, and generated dependency names. Provenance across this full chain remains underdeveloped ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Evaluator integrity under consequence.** Model judges can be influenced by how their verdicts will be used, while generators can learn to optimize against known graders. Blinding, abstention, ensembles, and meta-evals help, but no general method makes a semantic verifier both scalable and independent of those consequences (Ch 10).

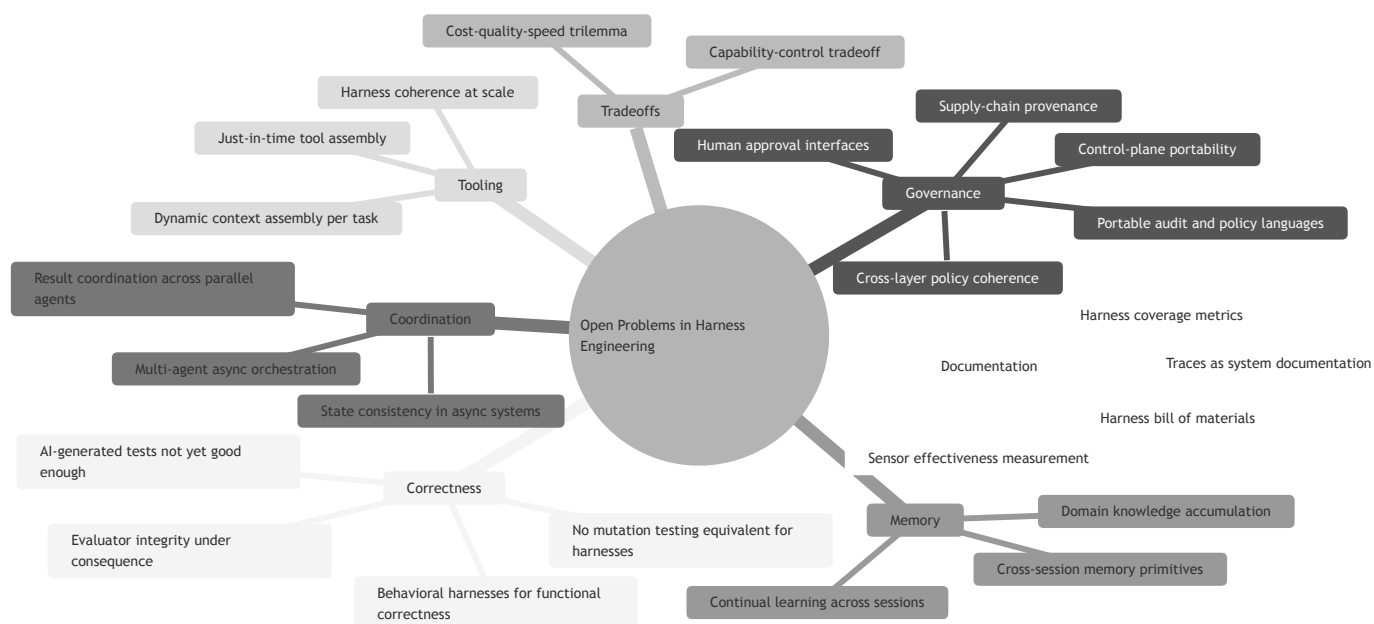
19.3 The Standing Advice

A few principles appear throughout the literature:

- **Treat context as a finite resource.** Find the smallest set of high-signal tokens that produces the desired outcome.
- **Do the simplest thing that works.** Agents are expensive; workflows often suffice; many tasks need neither.
- **Read the transcripts.** They show where the agent became confused, chose the wrong tool, or stopped using the guidance the harness supplied.
- **Iterate on what is load-bearing.** Stress-test components when models change. Remove what no longer contributes, and tune what still does.
- **Adapt harnesses to models, but ground them in durable principles.** Prompts and tools change from one model to another. The core work—context engineering, tool design, evaluation, sandboxing, and self-verification—remains.

The field is barely old enough to have textbooks. This document is an attempt to write one anyway, with the expectation that it will age faster than most. The citations provide a path back to the original work as the conversation continues.

Diagram: Open Problems Mindmap



Key Takeaways

- **The shared vocabulary is young:** many terms became common in 2025–2026, even when the underlying ideas are older.
- **Models absorb parts of the harness, but harness work moves:** as models take on more capabilities natively, harness engineering shifts toward harder problems rather than disappearing.

- **Open problems now span the full ETCLOVG stack:** behavioral correctness, evaluator integrity, control-plane portability, harness and governance coherence, readiness reporting, cost-quality-speed, capability-control, asynchronous coordination, continual learning, just-in-time tool assembly, and traces as documentation.
- **Five standing principles cut across all contexts:** finite context, simplest-that-works, read-the-transcripts, iterate-load-bearing, tailor-to-model.
- **Harness engineering is ongoing work:** it is not scaffolding to discard once models improve, but the continuing craft of building effective systems around increasingly capable cores.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

References

Full bibliography for *Agent Harness: A Practitioner's Textbook*.

Foundations (prerequisite)

- *LLM Foundations for Harness Engineering* — the prerequisite stage-1 volume this book builds on. [../llm-foundations/](https://llm-foundations/)
-

Foundations

- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
 - *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
 - Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
 - Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
 - Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
 - Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
 - Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022. <https://arxiv.org/abs/2210.03629>
 - Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
-

Context, Memory, and Working State

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
 - Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
 - Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
 - Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
-

Tools and Constraints

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>

- Simon Willison, *The lethal trifecta for AI agents: private data, untrusted content, and external communication*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
 - *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
 - Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
 - OpenAI, *Connect Private MCP Servers to OpenAI Products*, Jun 2026. <https://developers.openai.com/blog/connect-private-mcp-servers-to-openai-products>
 - OpenAI, *Programmatic Tool Calling*, 2026. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>
-

Specs and Workflow Design

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
-

Reasoning and Planning Patterns

(See also *ReAct* under Foundations, the base reasoning-and-acting pattern.)

- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
 - Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
 - Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
 - Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
 - Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>
 - Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
-

Multi-Agent Systems

- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (introducing the MAST failure taxonomy), UC Berkeley / arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
-

Reliability and Operational Safety

- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
 - LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
 - Martin Fowler, *CircuitBreaker*, martinowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
 - Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
 - Anthropic Safeguards Research Team, *How We Contain Claude*, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
 - Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
-

Agent Capability Measurement

- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
-

Loop Engineering

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
 - *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>
 - Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>
 - *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
 - *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
 - *Loop Engineering Guide (2026)*, AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
 - *Loop Engineering Crash Course*, The AI Agent Factory (Panaversity). <https://agentfactory.panaversity.org/docs/loop-engineering-crash-course>
-

Evaluation and Observability

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
 - Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
 - Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
 - OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
 - Anthropic Safeguards Research Team, *Agentic Misalignment: Summer 2026 Update*, 2026. <https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>
 - OpenAI, *Trustworthy Third-Party Evaluations: Foundations*, 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
 - OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>
-

Runtimes, Harnesses, and Reference Implementations

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
 - Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
-

Instructions and Prompting

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
 - OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>
-

Model Selection, Routing, and Reasoning

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
 - Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
 - Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
 - LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
 - Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>
-

Human-Agent Interaction

- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
 - Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
-

Computer-Use and Multimodal Agents

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
 - OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
 - Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
 - Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
 - Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
 - Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>
-

Cost, Privacy, and Governance Frameworks

- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>
 - OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
 - NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
 - ISO/IEC 42001:2023 — *Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
 - *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
 - Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
-

Agent Fleets, Identity, and Control Plane

- NIST, *Identity and Authorization for Software Agents*, Feb 2026. <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>

- Google Cloud, *Agent Registry Overview*, 2026. <https://docs.cloud.google.com/agent-registry/overview>
 - AWS, *AWS Agent Registry in Amazon Bedrock AgentCore (Preview)*, Apr 2026. <https://aws.amazon.com/about-aws/whats-new/2026/04/aws-agent-registry-in-agentcore-preview/>
 - AWS, *Introducing Amazon Bedrock AgentCore Identity: Securing Agentic AI at Scale*, 2026. <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/>
 - Microsoft, *Agent Registry in the Microsoft 365 Admin Center*, 2026. <https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-registry?view=o365-worldwide>
-

Companion Volume

- *LLM Foundations for Harness Engineering*. The model-internals companion to this book. [../llm-foundations/](https://llm-foundations.com/)
-

Reading List

- *Awesome Harness Engineering*, walkinglabs. <https://github.com/walkinglabs/awesome-harness-engineering>
- *Learn Harness Engineering*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/>
- *Learn Harness Engineering — Skills*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/skills/>

Glossary

Concise definitions for terms used throughout this textbook. The chapter reference points to where each term is discussed in depth; for the source behind a definition, follow that chapter's inline citations and the [References](#).

Core Concepts

Agent — A language model plus the system that lets it do real work: browse a codebase, run code, call tools, recover from errors, and sustain multi-step tasks. Captured by the equation $Agent = Model + Harness$ (Ch 1).

Agent harness — Everything engineered around the model: system prompts, tools and their descriptions, bundled infrastructure, sub-agent orchestration, control flow, hooks/middleware, and evaluation infrastructure (Ch 1).

Agent loop — The core execution cycle: assemble context → model emits a tool call or a final answer → harness executes the call → append the result to context → repeat until done (Ch 1).

Augmented LLM — A model equipped with retrieval, tools, and memory, able to generate its own queries, select tools, and decide what to retain. The basic building block of an agent (Ch 1, 6).

ACI (agent-computer interface) — The design surface between an agent and its tools, by analogy with HCI: as much engineering should go into how an agent uses tools as into how a human uses a screen (Ch 4).

Builder harness — The system prompt and tools shipped by an AI lab as part of a coding-agent product; the middle of the three concentric harness rings (Ch 1).

User harness — The AGENTS.md files, hooks, skills, and review agents a team adds on top of a coding agent to fit its own codebase; the outer ring (Ch 1).

Harness engineering — Iterating on the whole system around the model — not just one prompt — so that each observed failure is permanently engineered out (Ch 1).

Binding-constraint thesis — The claim that long-horizon agent reliability is often limited by harness layers — execution, tools, context, lifecycle, observability, verification, and governance — rather than by model capability alone (Ch 1, 12).

ETCLOVG — A seven-layer taxonomy for agent harness engineering: Execution environment, Tool interface, Context, Lifecycle, Observability, Verification, and Governance (Ch 1).

Context engineering — Curating the smallest set of high-signal tokens in the context window during inference; the practice one level below harness engineering (Ch 1, 2).

MCP (Model Context Protocol) — An open client-server standard for exposing tools, resources, and prompts to agents, so any compatible client can discover and call them without bespoke integration (Ch 4).

A2A (Agent-to-Agent protocol) — A protocol boundary for delegation among opaque agentic applications; complementary to MCP, which primarily exposes tools and context to one agent runtime (Ch 4).

Protocol boundary — The integration line a tool or agent standard crosses: model-to-function, agent-to-external-capability, agent-to-agent, or agent-to-repo/environment (Ch 4).

Tool call — Structured output (typically JSON) in which the model names a tool and its arguments. Deterministic harness code decides what to do with it (see Foundations ch 12) (Ch 4, 9).

Structured output — Model output constrained into a machine-readable shape, usually JSON or XML, so software can parse it reliably. Tool calls are the agent-specific case (see Foundations ch 12) (Ch 1, 4, 9).

Context and Memory

Context window — The finite span of tokens a model can attend to in a single inference call (see Foundations ch 9). In a harness it is a budget every system prompt, tool result, and history turn competes for (Ch 2).

Context rot — The degradation of a model's ability to recall and use information accurately as the context grows longer (see Foundations ch 9). The harness angle: it is the binding operating constraint, framed here as an attention budget (Ch 2).

Attention budget — The framing of context as a finite resource that every added token spends (Ch 2).

KV-cache — A cache of the key/value tensors for already-processed tokens (see Foundations ch 9). Identical context prefixes can be served from it, cutting time-to-first-token and cost roughly tenfold; in a harness, prefix stability becomes a production cost lever (Ch 2).

Prefill / decode — Prefill is processing the input prompt; decode is generating output tokens (see Foundations ch 9). Agentic workloads are heavily prefill-skewed (~100:1 input-to-output) (Ch 2).

Lost-in-the-middle — The tendency of models to attend less reliably to information in the middle of a long context than to its start or end (see Foundations ch 9) (Ch 2, 3).

Compaction — Summarizing a conversation near its context limit and reinitiating a fresh window with the summary. Lossy (see Foundations ch 9) (Ch 3).

Context reset — Clearing the context entirely and starting a fresh agent with a structured handoff — distinct from in-place compaction (Ch 7).

Just-in-time retrieval — Loading data into context on demand via lightweight references (file paths, queries, links) rather than pre-embedding everything up front (Ch 2).

Recitation — Repeatedly rewriting a goal or todo list into the end of context so it stays in the model's recent attention span (Ch 3).

Structured note-taking (agentic memory) — Having the agent write progress notes to disk so they can be reloaded after a context reset (Ch 3).

MemGPT — A memory architecture that treats the context window as OS-style "main memory" and external stores as "disk," letting the model page information in and out via function calls (virtual context management) (Ch 3).

Mem0 — A memory layer that dynamically extracts, consolidates, and retrieves salient facts across sessions, with an optional graph variant for entity relations (Ch 3).

Sleep-time compute — Processing context offline between requests — anticipating likely queries and precomputing inferences — to cut the compute a later query needs (Ch 3).

Persistent memory poisoning — An attack in which untrusted content is promoted into durable memory and later treated as trusted state, allowing an injection to survive context resets and steer future runs (Ch 3, 5).

Sub-Agents and Workflows

Sub-agent — A specialized agent that handles a focused task in its own context window and returns only a condensed summary to its parent (Ch 3).

Context firewall — The property of the sub-agent pattern by which the parent never sees a sub-agent's intermediate noise, only its condensed result (Ch 3).

Orchestrator-workers — A workflow where a central LLM dynamically decomposes a task, delegates to worker LLMs, and synthesizes their results (Ch 6).

Evaluator-optimizer — A workflow where one LLM generates and another critiques, in a loop, until evaluation criteria are met (Ch 6).

Micro-agent — A small, focused agent (≈3–20 steps) embedded in an otherwise deterministic workflow, rather than an open-ended "loop until done" agent (Ch 6, 9).

Multi-agent topology — The coordination shape of a multi-agent system: orchestrator-worker, hierarchical, blackboard/shared-memory, or debate/voting (Ch 3, 6).

MAST (Multi-Agent System failure taxonomy) — An empirical taxonomy of 14 multi-agent failure modes in three categories: specification issues, inter-agent misalignment, and task verification (Ch 3).

Trust escalation — A delegation failure where a low-privilege worker's claim is accepted by a higher-privilege parent, which then acts with authority the worker did not possess (Ch 3, 18).

Reasoning / self-correction patterns — Single-agent deliberation patterns that trade tokens for reliability: Reflexion (self-critique memory), Self-Refine (critique-and-revise), CRITIC (tool-grounded critique), Tree of Thoughts and LATS (branching search), and ReWOO (plan-then-execute) (Ch 6).

Tools and Sandboxing

Namespacing — Grouping related tools under common prefixes (`asana_*` , `browser_*`) to prevent name collisions and enable group-level masking (Ch 4).

Action masking — Keeping the full tool set stable in context while constraining which actions can be selected in a given state (Ch 2).

Progressive disclosure — Loading tool definitions, files, or instructions only when needed, rather than all up front (Ch 4).

Skill — A reusable, file-backed capability (often a `SKILL.md` plus supporting code) that an agent can load on demand (Ch 4).

Code execution (as a meta-tool) — Presenting tools as a code API the agent invokes by writing code, rather than as direct calls — sharply reducing token cost (Ch 4).

Programmatic tool calling — Having the model write a bounded program that filters, joins, ranks, deduplicates, aggregates, or validates tool results inside a runtime, reducing model round-trips and context transfer (Ch 4).

Shell — A command-line interface such as Bash or zsh. In agent systems, shell access is powerful because it lets the agent run tests, inspect files, install packages, and compose ad hoc tools (Ch 1, 4, 5).

Filesystem — The directories and files the agent can read or write. It acts as workspace, durable memory, and a collaboration surface between agents and humans (Ch 1, 2, 5).

Sandbox — An isolated environment, with filesystem and network boundaries, within which an agent can act freely without per-action approval prompts (Ch 5).

Sandbox liveness — The sandbox's role as an authorization region: it lets an agent act without per-action approval prompts while staying inside configured boundaries (Ch 5).

Governance — Harness mechanisms for identity, permission policy, scoped credentials, human approval, audit logs, and cross-layer security accountability (Ch 5, 18).

Agentic readiness — The degree to which a system is safe and legible for autonomous callers: idempotent mutations, explicit operation state, machine identity, clear retry semantics, observable outcomes, and compensating actions (Ch 5).

Delegated auth — A pattern where the agent acts through scoped credentials or a proxy-authorized identity rather than inheriting the user's full ambient authority (Ch 5).

Supply-chain provenance — Evidence about the origin and integrity of tools, packages, datasets, MCP servers, and retrieval sources the agent depends on (Ch 5, 18).

Hook / middleware — Harness-executed scripts or checkpoints that run automatically on lifecycle events (start, post-tool-call, stop), enforcing rules deterministically (Ch 5).

Feedforward / feedback — Feedforward controls (guides) steer the agent before it acts; feedback controls (sensors) observe after it acts and help it self-correct (Ch 5).

Computational / inferential control — Computational controls (linters, type checkers) are deterministic and fast; inferential controls (AI review, LLM-as-judge) handle nuance but are slower and non-deterministic (Ch 5).

Ambient affordances — Properties of the environment itself (strong typing, clear module boundaries, opinionated frameworks) that make a codebase legible and tractable to agents (Ch 5).

CI (continuous integration) — Automated checks that run around code changes, usually tests, linters, builds, and deployment gates. In harness design, CI-like checks become feedback sensors (Ch 5, 10).

Linters / type checker — Deterministic tools that detect style, syntax, structural, or type errors before runtime. They are common computational sensors in an outer harness (Ch 5).

Prompt injection — An attack in which instructions hidden in content the agent reads (a web page, a file, a tool result) are interpreted by the model as commands (see Foundations ch 8, ch 12) (Ch 5).

Lethal trifecta — The dangerous combination, in one agent, of access to private data, exposure to untrusted content, and the ability to communicate externally (Ch 5).

Circuit breaker — A reliability wrapper that trips after a threshold of failures so further calls to a failing tool, service, or sub-agent fail fast instead of hanging or retry-storming (Ch 5).

Kill switch — A human- or policy-triggered stop that halts an agent or fleet immediately, independent of the agent's own control flow; it lives in the harness because a steered agent cannot be trusted to stop itself (Ch 5).

Canary token — A planted fake secret (unused key, decoy file, tripwire URL) whose access or exfiltration raises a high-signal alarm that an agent has been steered — detection for the lethal-trifecta exfiltration path (Ch 5).

Action budget — A hard ceiling on tool calls, tokens, wall-clock time, or spend, after which the loop stops and escalates rather than running away (Ch 5, 8, 17).

Evaluation

Eval harness — The infrastructure that runs an evaluation end-to-end; distinct from the agent harness being evaluated (Ch 10).

Readiness validation — Verification that a specific model-plus-harness configuration is ready for a specific task distribution, environment, budget, and governance regime (Ch 10).

Failure attribution — Labeling an agent failure by likely harness layer — for example execution, tool interface, context, lifecycle, observability, verification, or governance — before choosing a fix (Ch 10, 12).

Task / trial — A *task* has defined inputs and success criteria; a *trial* is a single attempt at it (Ch 10).

Grader — A component that scores some aspect of a trial: code-based, model-based, or human (Ch 10).

Evaluator integrity — The trustworthiness of the grading system itself, supported by blinded judgment, deterministic evidence, abstention, calibration, meta-evals, versioning, and reproducible audit artifacts (Ch 10).

Transcript (trace, trajectory) — The full record of a trial: every message, tool call, and result (Ch 10, 12).

Outcome — The final environmental state at the end of a trial, distinct from the agent's text response (Ch 10).

Capability eval / regression eval — Capability evals measure what an agent can newly do (low pass rates, climbing); regression evals protect what it already does reliably (near 100%) (Ch 10).

pass@k / pass^k — pass@k is the probability of at least one success in k attempts (rises with k); pass^k is the probability that *all* k trials succeed (falls with k) (see Foundations ch 13) (Ch 10).

Infrastructure noise — Variation in benchmark scores caused by the runtime's resource configuration rather than by model capability (Ch 11).

Long-Running Agents and the Field

Shift-change problem — The challenge that successive agent sessions arrive with no memory of prior ones, because context windows are limited (Ch 7).

Initializer agent — An agent that runs once to set up a project (init script, progress log, feature list) for the later coding-agent sessions to build on (Ch 7).

Managed agent — A platform-managed agent architecture that separates the model-side brain, execution-side hands, and durable session/event log so each can fail, reset, or migrate independently (Ch 7).

Brain / hands split — The managed-agent separation between decision-making context (brain) and replaceable execution environment (hands) (Ch 7).

Sprint contract — A file-based agreement between a generator and an evaluator agent on what will be built and how success is verified, settled before each build sprint (Ch 7).

Event log — An append-only record of messages, tool calls, results, approvals, and errors. Execution state can be derived from it, making agents easier to replay and debug (Ch 9).

Agent platform — Infrastructure beyond a local framework: durable workspaces, managed sandboxes, identity, billing, observability, evaluation, governance, and human handoff across many runs and users (Ch 9, 18).

Checkpoint / resume — A reliability pattern where an agent periodically saves enough state to restart after failure or context reset without losing work (Ch 7, 9).

Durable execution — An infrastructure guarantee that each workflow step is persisted so a crashed or interrupted agent resumes from the last recorded step; non-deterministic model/tool calls are recorded and replayed, not recomputed (Ch 7).

Time horizon — METR's capability metric: the human-task length a model completes with 50% reliability; the frontier value has roughly doubled every seven months (Ch 7, 19).

Stateless reducer — Modeling an agent as a pure fold over an event log, making it serializable, replayable, and testable (Ch 9).

Model-harness co-evolution — The coupling created when frontier models are post-trained with their harnesses in the loop, so that changing either side can degrade performance (Ch 12).

Span telemetry — Structured trace data represented as a tree of spans for model calls, tool calls, retrieval, context assembly, permissions, costs, and outcomes (Ch 12).

OpenTelemetry GenAI semantic conventions — An emerging standard schema of span and attribute names for LLM and agent telemetry (`invoke_agent` , `chat` , `execute_tool` spans), letting agent traces join the ordinary observability stack (Ch 12).

Trace-to-eval loop — Converting real production failures into redacted, reproducible regression cases with outcome assertions (Ch 12).

Controlled self-improvement — An external, versioned loop that turns reviewed production corrections into bounded change tasks, gates them on evals, and uses canary and rollback rather than allowing a deployed agent to modify itself in place (Ch 12, 17).

Meta-harness — Treating harness design itself as an optimization object: prompts, tools, retries, context policies, evaluators, and control loops are ablated or searched using eval feedback (Ch 12).

Cost-quality-speed trilemma — The tradeoff that stronger execution environments, observability, verification, and governance improve reliability but increase cost and latency (Ch 19).

Capability-control tradeoff — The design axis where more authority, tools, memory, and autonomy improve capability while expanding the control, provenance, and audit problem (Ch 18, 19).

Ralph Wiggum loop — A hook that intercepts an agent's exit attempt and reinjects the original prompt in a clean context window, forcing it to continue against its goal (Ch 7, 8).

Loop engineering — Treating the agent loop itself as the unit of design — specifying the trigger, topology, verifier, and stop rules that surround the model so it can run unattended. The operator-facing view of the outer control loop (Ch 8).

Trigger (heartbeat) — What starts a loop pass without a human prompt: a schedule, a webhook, or another agent (Ch 8).

Verifier (maker-checker) — The fixed standard that decides "good enough," applied by an agent separate from the one that produced the work, so the maker cannot grade its own homework; the bottleneck of loop design (Ch 8).

Stop rule — An explicit condition that ends a loop — success, no-op, or ask-for-approval — plus the three hard stops that bound a runaway: a maximum iteration count, no-progress detection, and a budget ceiling (Ch 8).

Closed vs. open loop — A closed loop pins hard, checkable acceptance criteria up front and is safe to leave running; an open loop explores a fuzzy goal and needs an even stronger verifier or it ships confident garbage (Ch 8).

Instructions and Model Selection

Instruction hierarchy — The principle that instructions carry different authority by origin — system over developer over user over tool/retrieved content — so a lower-priority instruction cannot override a higher one. Prompt injection is a failure of this hierarchy (Ch 13).

Right altitude — The target specificity for a system prompt: concrete enough to steer behavior reliably, general enough to transfer across cases, avoiding both brittle hardcoded rules and vague guidance (Ch 13).

Model routing — Classifying a request's difficulty and dispatching easy ones to a cheap, weak model and hard ones to an expensive, strong one. Pays off only when the routing decision is much cheaper than the saving (Ch 14).

LLM cascade — Trying a cheap model first and escalating to a stronger one only when a verifier rejects the cheap answer. Matches strong-model accuracy at lower cost when the escalation signal is reliable (Ch 14).

Fallback — Failing over to an alternate model when the primary errors, times out, or is rate-limited, so the agent degrades gracefully (Ch 14).

AI gateway — An infrastructure component between the harness and model providers that presents one interface over many models and hosts routing, fallback, budgets, caching, and logging (e.g., LiteLLM, Portkey) (Ch 14, 17).

Reasoning model — A model post-trained, often with RL on verifiable rewards, to generate long internal reasoning before answering, spending inference tokens for better performance on hard tasks (Ch 14; *LLM Foundations* Ch 7–8).

Test-time compute — Spending more inference tokens, time, and money at answer time to do better on hard problems — a scaling axis distinct from larger models or more hardware (Ch 14).

Human Interaction

Permission fatigue — The degradation of oversight when an agent prompts for approval too often, training the human to rubber-stamp without reading (Ch 5, 15).

Mixed-initiative — An interaction style in which the system decides per action whether to act autonomously or defer to a human, managing the cost of interruption (Ch 15).

Approval as a tool call — Modeling human approval as a tool the agent invokes, so the request becomes a durable, replayable, auditable event that composes with suspend/resume (Ch 15).

Steering — Injecting a new instruction into a running agent so it is incorporated on the next turn, redirecting without losing session state (Ch 15).

Calibrated trust — The goal of the human interface: the human trusts the agent exactly as much as it deserves on a given task, achieved through transparency and grounded uncertainty rather than fluency (Ch 15).

Computer-Use Agents

Computer-use agent — An agent that operates software through its GUI — viewing screenshots and emitting cursor, keyboard, and navigation actions — rather than calling defined APIs (Ch 16).

Visual grounding — Translating an intention ("click Submit") into a concrete action (a click at specific coordinates); an error mode with no analog in API tools (Ch 16).

Set-of-Mark prompting — Overlaying numbered marks on candidate interactive elements so the model selects a discrete label instead of producing raw coordinates, improving grounding reliability (Ch 16).

Accessibility tree — A structured, semantic representation of a UI (roles, labels, states) built for assistive technology; often a more compact and exact screen encoding than raw pixels or DOM (Ch 16).

Cost and Operations

AgentOps — The lifecycle discipline for operating agents across governance and security, build and operations, evaluation, and observability, from planning through retirement (Ch 17).

Per-task budget — An explicit ceiling on tokens, tool calls, or cost for a single agent run, after which the agent stops and asks rather than looping indefinitely (Ch 17).

Cost attribution — Attaching token and dollar cost to each span of a trace, turning "the agent is expensive" into a specific, fixable engineering finding (Ch 17).

Multi-tenancy / tenant isolation — Serving many users or organizations from one platform while preventing state bleed (context/memory/cache leaking across tenants) and authority bleed (acting with the wrong tenant's credentials) (Ch 17).

Canary rollout — Releasing a harness change to a fraction of traffic and watching production traces and outcome metrics before full deployment, catching cases the eval suite missed (Ch 17).

Semantic cache — A cache that serves *similar* (not just identical) requests by embedding the query and returning a stored response above a similarity threshold; can skip whole model calls but risks false hits (Ch 17).

AI management system (ISO/IEC 42001) — The first certifiable standard for governing an organization's AI: how to establish, operate, and continually improve an AI management system — the AI analogue of ISO 27001 (Ch 17).

EU AI Act — Regulation (EU) 2024/1689, the first comprehensive AI law; it classifies systems by risk tier and imposes binding obligations on high-risk uses (Ch 17).

Fleet, Identity, and Control Plane

Agent fleet — A governed collection of agent definitions, versions, runs, sandboxes, tools, identities, and owners rather than one locally configured agent (Ch 18).

Agent control plane — Shared infrastructure that registers agent artifacts, manages identity and delegated authority, enforces policy, reconciles lifecycle, and preserves lineage and audit across a fleet; distinct from the data plane that performs work (Ch 18).

Agent identity — The durable machine identity of an agentic principal, distinct from its user, sponsor, runtime process, model, and individual session (Ch 18).

Agent registry — A governed source of truth for agent and capability definitions, versions, ownership, dependencies, requested permissions, attestations, deployment state, and lineage (Ch 18).

Delegated authorization — Granting an agent short-lived, audience- and purpose-bound authority derived from a user or service, with each further delegation able only to narrow the grant (Ch 5, 18).

Agent gateway — A data-plane enforcement point that evaluates the identity and execution envelope for model, tool, MCP, A2A, egress, or memory requests and can allow, deny, redact, reduce scope, strengthen isolation, or require approval (Ch 18).

Lineage — The causal graph connecting an agent action to its publisher, source and build, configuration, identity and delegation chain, policy decision, trace evidence, approval, and environmental outcome (Ch 18).

Non-repudiation — Evidence strong enough to attribute a request and authorization to particular workload and policy identities, using signed or integrity-protected events plus timestamps and immutable versions; it does not prove a human understood an approval or a model rationale was truthful (Ch 18).